

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

**Факультет прикладної математики
Кафедра системного програмування і
спеціалізованих комп'ютерних систем**

«На правах рукопису»
УДК 004.7:681.518

До захисту допущено:
Завідувач кафедри
_____ Віталій РОМАНКЕВИЧ
«___» _____ 2025 р.

Магістерська дисертація

на здобуття ступеня магістра

**за освітньо-професійною програмою «Системне програмування та
спеціалізовані комп'ютерні системим»**

зі спеціальності 123 «Комп'ютерна інженерія»

**на тему: «Спосіб оптимізації енергоживлення IoT-системи
моніторингу кліматичних показників»**

Виконав (-ла):

студент (-ка) II курсу, групи КВ-41 мп

Стецюренко Ілля Станіславович _____

Науковий керівник:

Доцент кафедри СПіСКС, канд. техн. наук, доцент

Петрашенко Андрій Васильович _____

Рецензент:

доцент кафедри ПЗКС, канд. техн. наук, доцент,

Заболотня Тетяна Миколаївна _____

Засвідчую, що у цій магістерській
дисертації немає запозичень з праць
інших авторів без відповідних
посилань.

Студент (-ка) _____

Київ – 2025 року

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет прикладної математики

Кафедра системного програмування і спеціалізованих комп'ютерних систем

Рівень вищої освіти – другий (магістерський)

Спеціальність – 123 «Комп'ютерна інженерія»»

Освітньо-професійна програма «Системне програмування та спеціалізовані комп'ютерні системи»

ЗАТВЕРДЖУЮ

Завідувач кафедри

_____ Віталій РОМАНКЕВИЧ

«___» _____ 2025 р.

ЗАВДАННЯ
на магістерську дисертацію студенту
Стецюренко Ілля Станіславович

1. Тема дисертації «Спосіб оптимізації енергоживлення IoT-системи моніторингу кліматичних показників», науковий керівник дисертації Петрашенко Андрій Васильович, канд. техн. наук, доцент, затверджені наказом по університету від «6» листопада 2025 р. №4836-с

2. Термін подання студентом дисертації 12 грудня 2025

3. Об'єкт дослідження: процес енергоспоживання в розподілених автономних IT-системах моніторингу кліматичних показників.

4. Вихідні дані:

- розроблений програмно-апаратний комплекс моніторингу кліматичних показників на базі ESP32;
- адаптивний алгоритм керування енергоспоживанням;
- результати експериментальних досліджень та порівняльного аналізу енергоефективності протоколів передачі даних.

5. Перелік завдань, які потрібно розробити

- Провести аналіз методів енергозбереження та комунікаційних протоколів в IoT.

- Обґрунтувати вибір апаратної платформи та програмних засобів.
- Розробити адаптивний алгоритм керування режимами роботи IoT-пристрою.
- Реалізувати програмне забезпечення для IoT-вузла та хмарного сервісу.
- Провести експериментальні дослідження ефективності розробленого рішення.

6. Орієнтовний перелік графічного (ілюстративного) матеріалу:

- Структурна схема системи моніторингу.
- Схема алгоритму адаптивного керування (FSM).
- Принципова схема вимірювального стенду.
- Архітектура хмарної частини системи на AWS.
- Презентація роботи.

7. Орієнтовний перелік публікацій:

- Збірник тез доповідей XVIII (2025) науково-практичної конференції магістрантів та аспірантів "Прикладна математика та комп'ютинг".
- Збірник доповідей International R&D Online Student Conference And Competition "Science And Technology Of The XXI Century
- Стаття у фаховому виданні «Центральноукраїнський науковий вісник. Технічні науки» (прийнято до друку, 2026, Вип. 13(44)).

8. Дата видачі завдання 1 листопада 2024р.

Календарний план

№ з/п	Назва етапів виконання магістерської дисертації	Термін виконання етапів магістерської дисертації	Примітка
1	Узгодження керівника кваліфікаційної роботи (МД) та його затвердження	15.09.24	
2	Визначення тематики (напрямку) дослідження магістерської дисертації	15.10.24	
3	Формулювання об'єкта і предмета дослідження	1.12.24	
4	Визначення структури магістерської дисертації	1.03.25	
5	Зміст та вступ до МД (робочі версія). Підготовка тез	20.03.25	

	доповіді для виступу на конференції		
6	Перший розділ МД з висновками (робочі версія). Підготовка тез доповіді для виступу на конференції	15.04.25	
7	Другий розділ МД з висновками (робочі версія)	15.05.25	
8	Остаточне формування тематики кваліфікаційної роботи	25.06.25	
9	Тема магістерської дисертації, заява на ім'я завідувача кафедри з точною назвою МД	10.09.25	
10	Зміст та вступ до МД (остаточний варіант)	24.09.25	
11	Реферат до МД (українською мовою) та перший розділ з висновками	8.10.25	
12	Титульний аркуш, завдання та другий розділ МД з висновками	22.10.25	
13	Залік з науково-дослідної практики	31.10.25	
14	Попередній захист магістерської дисертації	20.11.25	
15	Надання керівнику МД остаточного варіанту дисертації для перевірки на запозичення та збіг/ідентичність/схожість	5.12.25	
16	Здача всіх документів магістерської дисертації секретарю комісії (Бояріновій Ю.Є.)	18.12.25	
17	Захисти МД	25.12.25	

Студент

Ілля СТЕЦЮРЕНКО

Науковий керівник

Андрій ПЕТРАШЕНКО

РЕФЕРАТ

Актуальність теми: Стрімка інтеграція Інтернету речей (IoT) у сфери екологічного моніторингу та точного землеробства вимагає розгортання мереж автономних сенсорів у важкодоступних місцях. Критичною проблемою таких систем є обмежений енергетичний ресурс хімічних джерел живлення. Існуючі методи керування зі статичними інтервалами активності є неефективними в умовах динамічного середовища, що призводить до надлишкових витрат енергії та здорожчання експлуатації. Тому розробка адаптивних методів керування, здатних балансувати між деталізацією даних та енергозбереженням, є актуальним науково-прикладним завданням.

Мета роботи: Підвищення енергоефективності та подовження терміну автономної роботи ІТ-системи моніторингу кліматичних показників шляхом розробки, реалізації та дослідження модифікованого гібридного методу адаптивного керування режимами функціонування мікроконтролерів.

Об'єкт дослідження: Процес енергоспоживання в розподілених автономних ІТ-системах моніторингу параметрів навколишнього середовища.

Предмет дослідження: Методи, моделі та алгоритми адаптивного керування режимами роботи мікроконтролерів (ESP32) та комунікаційних інтерфейсів для мінімізації інтегральних енергетичних витрат.

Методи дослідження: У роботі використано методи системного аналізу (для вибору апаратної платформи та протоколів), теорії скінченних автоматів (для моделювання станів пристрою), об'єктно-орієнтованого програмування (для реалізації ПЗ), натурного експерименту (для вимірювання струмів споживання) та математичної статистики (для обробки результатів).

Наукова новизна: Удосконалено метод керування енергоспоживанням шляхом розробки гібридного адаптивного алгоритму, який поєднує локальну аналітику на мікроконтролері (для LoRa/BLE) та хмарне керування на базі AWS Lambda (для Wi-Fi), що дозволило знизити енергоспоживання на 72–87% порівняно з базовими методами. Вперше проведено комплексне порівняльне дослідження енергоефективності п'яти протоколів (MQTT, HTTP, CoAP, BLE, LoRaWAN) на

єдиній апаратній базі, за результатами якого доведено перевагу протоколу CoAP (12.47 мА) та технології LoRa (12.63 мА) для задач моніторингу.

Практична цінність: Розроблено універсальну програмно-апаратну платформу на базі ESP32 та BME680, реалізовано захищену архітектуру збору даних із використанням сервісів AWS. Впровадження розроблених алгоритмів дозволяє подовжити час автономної роботи пристрою.

Особистий внесок магістранта: Обґрунтовано вибір компонентної бази, розроблено архітектуру системи та схеми безпеки, програмно реалізовано адаптивні алгоритми для п'яти протоколів, створено вимірювальний стенд та проведено серію експериментів з аналізом результатів.

Апробація результатів дисертації:

- XVIII науково-практичній конференції магістрантів та аспірантів «Прикладна математика та комп'ютинг» (м. Київ, 2025 р.).
- International R&D Online Student Conference And Competition “Science And Technology Of The XXI Century” (м. Київ, 2025 р.).

Публікації:

- 1 стаття у науковому фаховому виданні України (категорія «Б») «Центральноукраїнський науковий вісник. Технічні науки»;
- 2 тез доповідей у збірниках матеріалів міжнародних та всеукраїнських конференцій.

Структура та обсяг роботи: Магістерська дисертація складається зі вступу, чотирьох розділів, висновків до кожного розділу, загальних висновків, списку використаних джерел (19 найменувань) та додатків. Загальний обсяг – 146 сторінок, з них 83 сторінки основного тексту, 2 рисунки, 3 таблиці, 5 додатків.

Ключові слова: IoT, ESP32, енергоефективність, адаптивний алгоритм, Deep Sleep, MQTT, CoAP, LoRaWAN, BLE, AWS IoT.

ABSTRACT

Relevance of the Topic: The rapid integration of the Internet of Things (IoT) into environmental monitoring and precision agriculture requires the deployment of autonomous sensor networks in remote locations. A critical issue for such systems is the limited energy resource of chemical power sources. Existing control methods with static activity intervals prove ineffective in dynamic environments, leading to excessive energy waste and increased operational costs. Therefore, developing adaptive control methods capable of balancing data granularity and energy conservation is an urgent scientific and applied task.

Objective of the Study: To increase energy efficiency and extend the autonomous operation life of the IT climate monitoring system by developing, implementing, and investigating a modified hybrid method for adaptive control of microcontroller operating modes.

Object of Study: The process of energy consumption in distributed autonomous IT systems for environmental parameter monitoring.

Subject of Study: Methods, models, and algorithms for adaptive control of microcontroller (ESP32) and communication interface operating modes to minimize integral energy costs.

Research Method: The study employs methods of system analysis (for selecting hardware and protocols), finite state machine theory (for modeling device states), object-oriented programming (for software implementation), full-scale experimentation (for measuring current consumption), and mathematical statistics (for data processing).

Scientific Novelty: The method of energy consumption control has been improved by developing a hybrid adaptive algorithm that combines local analytics on the microcontroller (for LoRa/BLE) and cloud-based control using AWS Lambda (for Wi-Fi), which reduced energy consumption by 72–87% compared to baseline methods. A comprehensive comparative study of the energy efficiency of five protocols (MQTT, HTTP, CoAP, BLE, LoRaWAN) on a single hardware platform was conducted for the first time, proving the advantage of the CoAP protocol (12.47 mA) and LoRa technology (12.63 mA) for monitoring tasks.

Practical Value: A universal hardware-software platform based on ESP32 and BME680 has been developed, and a secure data collection architecture using AWS services has been implemented. The implementation of the developed algorithms allows for extending the device's battery life.

Personal Contribution: The student justified the choice of components, designed the system architecture and security schemes, implemented adaptive algorithms for five protocols, created a measurement stand, and conducted a series of experiments with results analysis.

Approbation of the results of the dissertation:

- XVIII scientific and practical conference of master's and postgraduate students "Applied Mathematics and Computing" (Kyiv, 2025).
- International R&D Online Student Conference And Competition "Science And Technology Of The XXI Century" (Kyiv, 2025).

Publications:

- 1 article in the scientific professional publication of Ukraine (category "B") "Central Ukrainian Scientific Bulletin. Technical Sciences";
- 2 abstracts of reports in collections of materials of international and all-Ukrainian conferences.

Structure and scope of work: The master's dissertation consists of an introduction, four sections, conclusions to each section, general conclusions, a list of sources used (19 names) and appendices. Total volume – 146 pages, of which 83 pages of main text, 2 figures, 3 tables, 5 appendices.

Keywords: IoT, ESP32, Energy Efficiency, Adaptive Algorithm, Deep Sleep, MQTT, CoAP, LoRaWAN, BLE, AWS IoT.

ЗМІСТ

СПИСОК ТЕРМІНІВ, СКОРОЧЕНЬ ТА ПОЗНАЧЕНЬ	13
ВСТУП.....	16
1. АНАЛІЗ СТАНУ ПИТАННЯ ТА ПОСТАНОВКА ЗАДАЧІ	20
1.1 Роль та значення енергоефективності в сучасних IoT-системах моніторингу	20
1.2 Огляд апаратних платформ для IoT-систем з низьким енергоспоживанням. 22	
1.2.1 Платформи з інтегрованим Wi-Fi: Espressif Systems.....	22
1.2.2 Платформи з фокусом на ультранизькому енергоспоживанні	23
1.2.3 Порівняльний аналіз платформ	24
1.2.4 Обґрунтування вибору платформи для дослідження	25
1.3 Аналіз програмних методів та алгоритмів оптимізації енергоспоживання ...	25
1.3.1 Режими зниженого енергоспоживання (Power Saving Modes)	26
1.3.2 Алгоритмічні методи оптимізації.....	27
1.4. Порівняльний аналіз протоколів та архітектур передачі даних в аспекті енергоефективності.....	28
1.4.1 Протоколи на базі IP-мереж (Wi-Fi).....	28
1.4.2 Альтернативні архітектури зв'язку.....	29
1.4.3 Узагальнюючий аналіз	30
1.5 Виявлення невирішених аспектів проблеми та постановка задачі дослідження.....	32
1.5.1 Невирішені аспекти проблеми.....	32
1.5.2 Постановка задачі дослідження.....	33
Висновки до розділу	34
2. МЕТОДИ, ЗАСОБИ ТА ПІДХОДИ ДО ДОСЛІДЖЕННЯ	36

2.1	Методологічна основа дослідження.....	36
2.2	Обґрунтування вибору апаратних засобів дослідження	38
2.2.1	Мікроконтролерна платформа ESP32	38
2.2.2	Сенсорний модуль BME680	39
2.2.3	Апаратні засоби для реалізації BLE та LoRaWAN архітектур	40
2.3	Обґрунтування вибору програмних засобів та середовища розробки	41
2.3.1	Середовище розробки та фреймворк	41
2.3.2	Мова програмування	42
2.3.3	Ключові бібліотеки для прошивки	42
2.3.4	Хмарна інфраструктура та сервіси	43
2.4	Формування концептуальної моделі адаптивного алгоритму	43
2.4.1	Модель скінченного автомата (Finite State Machine)	44
2.4.2	Логіка прийняття рішень: функція адаптації.....	46
2.4.3	Адаптація моделі до різних комунікаційних архітектур	47
2.5	Методи оцінки енергоспоживання та точності даних.....	47
2.5.1	Методи оцінки енергоспоживання	48
2.5.2	Методи оцінки якості та актуальності даних	49
	Висновки до розділу	50
3.	РОЗРОБКА ЗАПРОПОНОВАНОГО РІШЕННЯ.....	52
3.1.	Загальна архітектура програмного забезпечення та апаратного стенду	53
3.1.1	Апаратна архітектура експериментального стенду.....	53
3.1.2	Загальна архітектура програмного забезпечення	54
3.2	Реалізація архітектури прямого підключення до хмари (Wi-Fi)	55
3.2.1	Програмна реалізація обміну даними через протокол MQTT	55
3.2.2	Програмна реалізація обміну даними через протокол HTTP	56

3.2.3 Програмна реалізація обміну даними через протокол CoAP	57
3.3 Реалізація архітектури з локальним шлюзом (Bluetooth Low Energy)	58
3.3.1 Реалізація архітектури «Сенсорний вузол – Шлюз»	58
3.3.2 Програмна реалізація сенсорного вузла в режимі BLE Advertising	59
3.3.3 Програмна реалізація функціонала BLE-шлюзу	60
3.4 Реалізація архітектури глобальної мережі (LoRaWAN)	60
3.4.1 Архітектура та компоненти системи LoRa	61
3.4.2 Налаштування хмарної інтеграції	62
3.4.3 Програмна реалізація кінцевого вузла	63
3.5 Розробка хмарної частини системи на AWS	64
3.5.1 Уніфікована функція AWS Lambda.....	64
3.5.2 Інтеграція потоків даних та маршрутизація	65
3.5.3 Структура баз даних.....	66
3.5.4 Налаштування інформаційних панелей у Grafana	67
3.6 Забезпечення інформаційної безпеки в розробленій системі	67
3.6.1 Захист каналів зв'язку та аутентифікація пристроїв.....	68
3.6.2 Політики безпеки та розмежування доступу в хмарному середовищі	69
3.6.3 Захист API та шлюзів	70
Висновки до розділу	71
4. ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ	73
4.1 Методика проведення експериментальних досліджень.....	73
4.1.1 Схема та апаратна конфігурація вимірювального стенду.....	73
4.1.2 Розрахункова модель автономності	74
4.1.3 Аналіз доцільності використання режиму Deep Sleep	75
4.2 Аналіз енергоефективності протоколів на базі Wi-Fi	76

4.2.1 Дослідження енергоспоживання при використанні протоколу HTTP	76
4.2.2 Дослідження енергоспоживання при використанні протоколу MQTT	77
4.2.3 Дослідження енергоспоживання при використанні протоколу CoAP	77
4.3 Порівняльний аналіз енергоспоживання фіксованого та адаптивних режимів	78
4.3.1 Ефективність впровадження режиму Deep Sleep	78
4.3.2 Ефективність локальної адаптації	79
4.4 Оцінка ефективності хмарного керування (v4) порівняно з локальними алгоритмами	80
4.4.1 Порівняння ефективності алгоритмів керування для протоколу HTTP...	80
4.4.2 Порівняння ефективності алгоритмів керування для протоколу MQTT .	81
4.4.3 Порівняння ефективності алгоритмів керування для протоколу CoAP ...	81
4.5 Аналіз енергоефективності альтернативних архітектур	82
4.5.1 Аналіз енергоефективності технології Bluetooth Low Energy	82
4.5.2 Аналіз енергоефективності технології LoRa	83
4.6 Узагальнений порівняльний аналіз та оцінка автономності	83
Висновки до розділу 4	85
ВИСНОВКИ	87
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	90
ДОДАТОК А	92
ДОДАТОК Б	95
ДОДАТОК В	96
ДОДАТОК Г	107
ДОДАТОК Д	119

СПИСОК ТЕРМІНІВ, СКОРОЧЕНЬ ТА ПОЗНАЧЕНЬ

API (Application Programming Interface) — прикладний інтерфейс програмування.

AWS (Amazon Web Services) — хмарна платформа, що надає обчислювальні потужності, доступ до баз даних та інші сервіси.

BLE (Bluetooth Low Energy) — технологія бездротового персонального зв'язку з низьким енергоспоживанням.

BME680 — інтегрований сенсор навколишнього середовища (температура, вологість, тиск, газ) від компанії Bosch Sensortec.

CoAP (Constrained Application Protocol) — спеціалізований протокол передачі даних для обмежених пристроїв, що працює поверх UDP.

Deep Sleep — режим глибокого сну мікроконтролера, при якому вимикається більшість периферії для економії енергії.

DevEUI (Device Extended Unique Identifier) — унікальний ідентифікатор кінцевого пристрою в мережі LoRaWAN.

ESP32 — серія недорогих мікроконтролерів з низьким енергоспоживанням та вбудованими Wi-Fi та Bluetooth.

GAP (Generic Access Profile) — профіль, що визначає загальні процедури виявлення та підключення пристроїв у BLE.

GPIO (General Purpose Input/Output) — інтерфейс вводу/виводу загального призначення.

HTTP (HyperText Transfer Protocol) — протокол передачі гіпертексту, що використовується для обміну даними в World Wide Web.

I2C (Inter-Integrated Circuit) — послідовна асиметрична шина для зв'язку між інтегральними схемами.

IoT (Internet of Things) — Інтернет речей, мережа фізичних об'єктів, оснащених вбудованими технологіями для взаємодії.

JSON (JavaScript Object Notation) — текстовий формат обміну даними, заснований на JavaScript.

LoRa (Long Range) — технологія модуляції радіосигналу для передачі даних на великі відстані з низьким енергоспоживанням.

LoRaWAN (Long Range Wide Area Network) — мережевий протокол, що працює поверх модуляції LoRa.

LPWAN (Low-Power Wide-Area Network) — енергоефективна мережа далекого радіусу дії.

MQTT (Message Queuing Telemetry Transport) — легкий мережевий протокол, що працює поверх TCP/IP, орієнтований на обмін повідомленнями між пристроями за принципом "видавець-підписник".

NTP (Network Time Protocol) — мережевий протокол для синхронізації внутрішнього годинника комп'ютера.

OTA (Over-The-Air) — технологія бездротового оновлення прошивки.

OTAA (Over-The-Air Activation) — метод активації пристрою в мережі LoRaWAN, при якому сесійні ключі генеруються динамічно.

P2P (Peer-to-Peer) — однорангова мережа, де пристрої взаємодіють безпосередньо один з одним.

REST (Representational State Transfer) — архітектурний стиль взаємодії компонентів розподіленого додатка в мережі.

RSSI (Received Signal Strength Indicator) — показник рівня потужності прийнятого сигналу.

RTC (Real-Time Clock) — годинник реального часу; в контексті ESP32 також позначає область пам'яті, що зберігається під час сну.

SPI (Serial Peripheral Interface) — послідовний периферійний інтерфейс для обміну даними між мікроконтролером та периферійними пристроями.

TLS (Transport Layer Security) — криптографічний протокол, що забезпечує захищений обмін даними.

TTN (The Things Network) — глобальна відкрита мережа Інтернету речей на базі LoRaWAN.

UDP (User Datagram Protocol) — протокол транспортного рівня без встановлення з'єднання.

ULP (Ultra-Low-Power co-processor) — співпроцесор в ESP32, призначений для виконання простих задач у режимі глибокого сну.

VOC (Volatile Organic Compounds) — леткі органічні сполуки.

ВСТУП

Сучасний етап розвитку інформаційного суспільства характеризується глобальною цифровізацією та стрімким поширенням концепції Інтернету речей (Internet of Things, IoT). Технології IoT стали фундаментом для побудови інфраструктури «розумних міст» (Smart City), систем точного землеробства (Precision Agriculture), моніторингу екологічного стану довкілля та промислової автоматизації (Industry 4.0). За даними аналітичних агентств, кількість підключених автономних пристроїв зростає експоненційно, що створює безпрецедентний попит на ефективні рішення для збору та передачі телеметричних даних.

Однією з ключових проблем масового впровадження IoT-систем є енергозабезпечення розподілених сенсорних вузлів. Переважна більшість таких пристроїв функціонує у важкодоступних локаціях без підключення до стаціонарної електромережі, покладаючись виключно на автономні хімічні джерела живлення. У цьому контексті виникає протиріччя між необхідністю забезпечити тривалий термін автономної роботи (від кількох місяців до років) та зростаючими вимогами до обчислювальної потужності, частоти передачі даних і безпеки з'єднань.

Існуючі методи керування енергоспоживанням, що базуються на статичних алгоритмах з фіксованими інтервалами активності, демонструють недостатню ефективність в умовах динамічної зміни параметрів навколишнього середовища. Вони призводять або до надлишкових витрат енергії в періоди стабільності контрольованих параметрів, або до втрати важливих даних при їх різких змінах. Крім того, часта заміна елементів живлення у масштабних сенсорних мережах призводить до критичного зростання експлуатаційних витрат (OPEX) та негативного впливу на довкілля, що суперечить принципам «Зеленого Інтернету речей» (Green IoT).

Таким чином, науково-технічна задача розробки адаптивних методів керування режимами роботи мікроконтролерів, які здатні динамічно підлаштовувати

стратегію енергоспоживання під поточний стан середовища та тип комунікаційного каналу, є надзвичайно актуальною. Її вирішення дозволить створити енергоефективні системи моніторингу нового покоління, що поєднують високу автономність з надійністю та інформативністю.

Зв'язок роботи з науковими програмами, планами, темами. Магістерська дисертація виконана на кафедрі системного програмування і спеціалізованих комп'ютерних систем Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського» у відповідності до пріоритетних напрямків розвитку науки і техніки України. Тематика дослідження узгоджується з науковим напрямом кафедри щодо розробки енергоефективних розподілених комп'ютерних систем та спеціалізованих засобів моніторингу.

Мета і задачі дослідження. Метою роботи є підвищення енергоефективності та подовження терміну автономної роботи ІТ-систем моніторингу кліматичних показників шляхом розробки та експериментального дослідження модифікованого гібридного методу адаптивного керування режимами функціонування мікроконтролерів.

Для досягнення поставленої мети необхідно вирішити такі задачі:

1. Провести системний аналіз сучасних апаратних платформ, методів енергозбереження та комунікаційних протоколів (MQTT, HTTP, CoAP, BLE, LoRaWAN) з метою виявлення факторів, що найбільше впливають на енергоспоживання автономних пристроїв.
2. Обґрунтувати вибір програмно-апаратного комплексу (мікроконтролер ESP32, сенсор BME680, хмарна платформа AWS), що забезпечує оптимальний баланс між продуктивністю та енергоефективністю.
3. Розробити математичну модель та алгоритм адаптивного керування інтервалами сну, що базується на аналізі динаміки змін параметрів середовища та передбачає можливість зовнішнього керування з боку хмарного сервісу.

4. Спроектувати та програмно реалізувати мультипротокольную архітектуру системи, що підтримує роботу через різні канали зв'язку та забезпечує уніфіковану обробку даних.
5. Створити універсальний вимірювальний стенд та провести комплексні експериментальні дослідження для кількісної оцінки ефективності розроблених рішень та порівняння енергоспоживання різних протоколів.
6. Об'єкт дослідження – процес енергоспоживання в розподілених автономних ІТ-системах моніторингу параметрів навколишнього середовища.
7. Предмет дослідження – методи, моделі та алгоритми адаптивного керування режимами роботи мікроконтролерів та бездротових інтерфейсів для мінімізації інтегральних енергетичних витрат.

Методи дослідження. У роботі використано комплексний методологічний підхід:

- методи системного аналізу — для класифікації існуючих рішень та обґрунтування вибору компонентної бази;
- методи теорії алгоритмів та скінченних автоматів — для формалізації логіки переходів між енергозберігаючими станами (Deep Sleep, Active Mode);
- методи об'єктно-орієнтованого програмування — для реалізації вбудованого програмного забезпечення та хмарних функцій (AWS Lambda);
- методи натурного експерименту — для вимірювання миттєвих та середніх значень струму споживання на розробленому апаратному стенді;
- методи математичної статистики — для обробки результатів вимірювань, оцінки похибок та підтвердження достовірності отриманих даних.

Наукова новизна одержаних результатів полягає у наступному:

1. Удосконалено метод керування енергоспоживанням сенсорних вузлів шляхом розробки гібридного адаптивного алгоритму, який, на відміну від відомих, поєднує локальний аналіз даних на рівні мікроконтролера (для

протоколів LoRa/BLE) з централізованим хмарним керуванням (для Wi-Fi протоколів), що дозволяє знизити енергоспоживання на 72–87% без втрати інформативності.

2. Дістало подальший розвиток застосування протоколу CoAP у зв'язці з проміжним шлюзом для передачі телеметрії в хмарні платформи, що дозволило експериментально довести його перевагу (на 20–25%) над традиційними протоколами MQTT та HTTP в умовах періодичного моніторингу.
3. Вперше отримано комплексні порівняльні характеристики енергоефективності п'яти комунікаційних технологій (Wi-Fi/HTTP, Wi-Fi/MQTT, Wi-Fi/CoAP, BLE, LoRaWAN) на єдиній апаратній базі ESP32, що дозволило сформулювати науково обґрунтовані рекомендації щодо проектування енергоефективних систем залежно від умов експлуатації.

Практичне значення одержаних результатів.

- Технічний аспект: Розроблено універсальну програмну бібліотеку та архітектуру, що дозволяє швидко розгортати енергоефективні вузли моніторингу на базі доступних компонентів. Створено методику та стенд для точного вимірювання енергоспоживання IoT-пристроїв.
- Економічний аспект: Впровадження розроблених алгоритмів дозволяє подовжити термін автономної роботи пристроїв від стандартного акумулятора (1000 мА·год) з кількох днів до 3–12 місяців, що суттєво знижує витрати на технічне обслуговування мережі.
- Соціальний аспект: Підвищення надійності та автономності систем екологічного моніторингу сприяє своєчасному виявленню небезпечних змін у навколишньому середовищі.

1. АНАЛІЗ СТАНУ ПИТАННЯ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Роль та значення енергоефективності в сучасних IoT-системах моніторингу

Технологічна парадигма Інтернету речей (Internet of Things, IoT), що передбачає об'єднання фізичних об'єктів, оснащених сенсорами, виконавчими механізмами та засобами комунікації, у єдину комп'ютерну мережу для збору та обміну даними, трансформувалася з концептуальної ідеї у глобальний ринок, що динамічно розвивається. За прогнозами провідних аналітичних агенцій, таких як Statista та IoT Analytics, кількість активних IoT-з'єднань у світі, що вже перевищила 15 мільярдів у 2023 році, зросте до понад 29 мільярдів до 2030 року. Цей експоненційний ріст зумовлений мініатюризацією компонентів, зниженням їх вартості та розповсюдженням бездротових технологій зв'язку. Пристрої IoT проникають у всі сфери людської діяльності: від побутових приладів у «розумних» будинках до складних промислових комплексів (Industrial IoT, IIoT), носимих медичних сенсорів та багатокомпонентної інфраструктури «розумних» міст.

Одним із найбільш соціально та екологічно значущих напрямків застосування IoT є моніторинг навколишнього середовища, де мініатюрні сенсорні вузли дозволяють вирішувати широкий спектр завдань, що раніше були економічно недоцільними. Зокрема, у сфері точного землеробства (Precision Agriculture) IoT-датчики забезпечують вимірювання фізико-хімічних параметрів ґрунту, що дозволяє оптимізувати використання ресурсів та підвищити врожайність на 30–40%. Не менш важливим є моніторинг якості повітря у мегаполісах та промислових зонах, де високощільні мережі відстежують концентрацію шкідливих газів і твердих частинок у реальному часі, надаючи дані для прийняття управлінських рішень. Окрім цього, автономні станції здійснюють контроль водних ресурсів, виявляючи хімічні та біологічні забруднення, а також забезпечують раннє виявлення лісових пожеж, що дозволяє значно скоротити час реагування на надзвичайні ситуації та мінімізувати збитки.

Особливістю переважної більшості таких систем моніторингу є їх розгортання у віддалених або важкодоступних місцях, де відсутня можливість підключення до централізованих електромереж. За таких умов джерелом живлення для сенсорних вузлів стають автономні елементи: первинні гальванічні елементи, вторинні акумулятори або системи збору енергії (energy harvesting). Проте кожен з цих підходів має суттєві обмеження: ємність батарей є скінченною, а процеси збору енергії — нестабільними. Таким чином, проблема обмеженості ресурсів живлення стає фундаментальним викликом, що стримує довготривалу та надійну експлуатацію розподілених IoT-мереж.

З огляду на це, оптимізація енергоспоживання перетворюється на критично важливу вимогу, що визначає життєздатність системи. Важливість енергоефективності обґрунтовується комплексом взаємопов'язаних факторів. По-перше, зниження середнього струму споживання безпосередньо впливає на подовження терміну автономної роботи. Наприклад, перехід від постійної активності до інтелектуальних режимів сну здатний збільшити час роботи вузла від акумулятора з кількох місяців до кількох років, що дозволяє суттєво збільшити інтервали між технічним обслуговуванням. По-друге, це сприяє зменшенню сукупної вартості володіння (Total Cost of Ownership, TCO). Оскільки значна частина експлуатаційних витрат припадає на логістику та заміну елементів живлення у важкодоступних місцях, подовження життєвого циклу батарей кардинально змінює економічну модель та рентабельність проєкту. По-третє, енергоефективність є запорукою підвищення надійності та безперервності даних (Data Continuity). Мінімізація ризиків передчасного розрядження батареї дозволяє уникнути втрати критично важливої інформації та утворення «сліпих зон», що є необхідною умовою для побудови точних прогнозних моделей та своєчасного реагування на зміни.

Отже, розробка та дослідження нових методів зниження енергоспоживання є пріоритетним напрямком розвитку технологій Інтернету речей. Саме інтелектуальні програмні підходи до керування енергоспоживанням, що виходять за рамки простої апаратної оптимізації та дозволяють пристрою адаптуватися до

умов експлуатації, відкривають шлях до створення по-справжньому автономних, довготривалих та економічно ефективних систем моніторингу.

1.2 Огляд апаратних платформ для IoT-систем з низьким енергоспоживанням

Вибір апаратної платформи є одним із фундаментальних етапів проєктування енергоефективного IoT-пристрою. Від характеристик мікроконтролера (MCU) залежать не лише обчислювальні можливості та набір доступних інтерфейсів, але й базовий рівень енергоспоживання, який надалі підлягає оптимізації програмними методами. Сучасний ринок пропонує широкий спектр рішень, тому ключовий компроміс при виборі полягає в пошуку балансу між продуктивністю, наявністю інтегрованих бездротових модулів та гнучкістю режимів енергозбереження. У рамках дослідження було проаналізовано три основні категорії платформ.

1.2.1 Платформи з інтегрованим Wi-Fi: Espressif Systems

Мікроконтролери від компанії Espressif Systems здобули значну популярність у спільноті розробників IoT завдяки вдалому поєднанню вартості, продуктивності та наявності інтегрованих радіомодулів. Першим знаковим рішенням став представлений у 2014 році мікроконтролер ESP8266, який зробив технологію Wi-Fi масово доступною для вбудованих систем. Оснащений 32-бітним процесором Tensilica L106 (80–160 МГц), він забезпечує достатню продуктивність для збору даних через інтерфейси I2C, SPI, UART та їх передачі мережею. Однак одноядерна архітектура цього чіпа створює обмеження, змушуючи ділити обчислювальні ресурси між підтримкою стека Wi-Fi та виконанням основного коду, що може призводити до затримок у критичних задачах. Крім того, режими

енергозбереження ESP8266 є менш ефективними порівняно з сучасними аналогами, а струм у режимі глибокого сну зазвичай перевищує 20 мкА.

Логічним розвитком лінійки став мікроконтролер ESP32, який де-факто став стандартом для сучасних IoT-проєктів завдяки усуненню недоліків попередника. Ключовою відмінністю ESP32 є використання двоядерної архітектури (ядра Tensilica Xtensa LX6 або RISC-V), що дозволяє виділити окреме ядро для керування радіомодулями Wi-Fi та Bluetooth, залишивши інше для прикладного коду. Це значно підвищує стабільність системи. Розширені комунікаційні можливості, зокрема підтримка Bluetooth Low Energy (BLE), дозволяють реалізовувати сценарії локального налаштування або створення шлюзів. Вирішальним фактором для даного дослідження є просунута система керування живленням ESP32. Вона забезпечує споживання в режимі глибокого сну на рівні 10–20 мкА, а наявність спеціалізованого ULP-співпроцесора (Ultra-Low-Power) дозволяє виконувати моніторинг сенсорів без пробудження основних енергоємних ядер.

1.2.2 Платформи з фокусом на ультранизькому енергоспоживанні

Альтернативну категорію формують мікроконтролери, розроблені з пріоритетом на максимальну енергоефективність, часто за рахунок відсутності вбудованих потужних радіоінтерфейсів. Яскравими представниками цього напрямку є сімейство STM32 від STMicroelectronics (зокрема серія STM32L). Побудовані на архітектурі ARM Cortex-M, ці пристрої демонструють найкращі у своєму класі показники енергоспоживання: в режимі зупинки (Stop mode) зі збереженням вмісту RAM струм може становити сотні наноампер, що є недосяжним для платформ з інтегрованим Wi-Fi. Проте для IoT-завдань суттєвим недоліком є необхідність використання зовнішніх комунікаційних модулів, що ускладнює схемотехніку, збільшує габарити пристрою та вносить додаткові енерговитрати на обмін даними між MCU та модемом.

Аналогічну нішу займають мікроконтролери сімейства nRF52 від Nordic Semiconductor. Вони є лідерами ринку рішень на базі Bluetooth Low Energy і забезпечують відмінну енергоефективність, порівнянну з STM32L. Ці чіпи ідеально підходять для ношеної електроніки та маячків, проте, як і у випадку з STM32, відсутність вбудованого Wi-Fi вимагає наявності проміжного шлюзу для передачі даних у глобальну мережу, що ускладнює інфраструктуру системи моніторингу.

1.2.3 Порівняльний аналіз платформ

Для систематизації характеристик розглянутих платформ ключові параметри зведено у таблиці 1.1.

Таблиця 1.1 Порівняння параметрів

Характеристика	ESP8266	ESP32	STM32L4xx	nRF52
Архітектура	1x Tensilica L106	2x Tensilica LX6	ARM Cortex-M4	ARM Cortex-M4
Wi-Fi	+	+	-	-
Bluetooth	-	Classic + BLE	є моделі з BLE	BLE
Струм в режимі Deep Sleep	~ 20-80 мкА	~ 10-20 мкА	~ 0.5-5 мкА	~ 1-5 мкА
Додаткові переваги	Низька вартість	ULP співпроцесор	Найвища енергоефективність	Відмінна підтримка BLE
Основний недолік	Обмежена продуктивність	Вище споживання в активному режимі	Потребує зовнішнього Wi-Fi	Потребує зовнішнього Wi-Fi

1.2.4 Обґрунтування вибору платформи для дослідження

На основі проведеного порівняльного аналізу для реалізації магістерської роботи було обрано платформу ESP32. Хоча рішення на базі STM32 та nRF52 демонструють нижчі показники власного енергоспоживання в режимах сну, відсутність інтегрованого Wi-Fi модуля робить їх менш придатними для побудови автономного вузла з прямим підключенням до хмари. Використання зовнішніх модулів зв'язку нівелює переваги енергоефективного ядра та невиправдано ускладнює апаратну частину.

Платформа ESP32 пропонує збалансоване рішення типу «система-на-кристалі» (SoC). Поєднання достатньої обчислювальної потужності, вбудованих інтерфейсів Wi-Fi та Bluetooth, а також гнучких режимів енергозбереження з наявністю ULP-співпроцесора створює оптимальну базу для дослідження. Це дозволяє зосередити увагу на розробці та тестуванні програмних методів оптимізації енергоспоживання, не обмежуючись апаратними складнощами інтеграції різномірних компонентів.

1.3 Аналіз програмних методів та алгоритмів оптимізації енергоспоживання

Якщо апаратна платформа визначає потенціал енергоефективності IoT-пристрою, то саме програмні методи та алгоритми дозволяють цей потенціал реалізувати. Оптимізація на рівні програмного забезпечення є найбільш гнучким та потужним інструментом для зниження енергоспоживання,

1.3.1 Режими зниженого енергоспоживання (Power Saving Modes)

Сучасні мікроконтролери, зокрема ESP32, оснащені складною системою керування живленням (Power Management Unit, PMU), яка дозволяє програмно відключати невикористовувані блоки. Базовим станом є активний режим (Active Mode), у якому всі компоненти, включаючи процесорні ядра та радіомодулі, повністю функціонують. Споживання струму в цьому стані є максимальним (80–260 мА для ESP32), тому головною метою програмної оптимізації є мінімізація часу перебування системи в активній фазі. Для зниження енерговитрат без повної зупинки обчислень застосовується режим сну модема (Modem-Sleep), який автоматично відключає радіомодуль між сеансами передачі даних, знижуючи струм до 20–40 мА.

Компромісним варіантом між економією та швидкістю реакції є легкий сон (Light-Sleep). У цьому режимі зупиняються тактові генератори процесорних ядер, проте оперативна пам'ять та основна периферія залишаються під напругою. Це дозволяє відновити роботу за мікросекунди та підтримувати з'єднання з точкою доступу Wi-Fi (через механізм DTIM beacon) при споживанні на рівні 0,8–2 мА. Проте основним інструментом для досягнення тривалої автономності є режим глибокого сну (Deep-Sleep). У цьому стані відключається переважна більшість компонентів, включаючи процесор та RAM, а активними залишаються лише контролер реального часу (RTC) та частина його пам'яті. Споживання падає до 10–20 мкА, що дозволяє пристрою працювати роками. Вихід із цього режиму фактично є перезавантаженням мікроконтролера, яке може бути ініційоване таймером або зовнішньою подією.

Унікальною особливістю архітектури ESP32, що розширює можливості глибокого сну, є наявність співпроцесора ультранизького споживання (ULP Co-processor). Цей простий 8-бітний вузол здатен виконувати програми та опитувати сенсори, поки основні ядра знеструмлені. Це дозволяє реалізувати асинхронну

реакцію на події: основний процесор прокидається лише тоді, коли ULP фіксує значні зміни показників, що дозволяє уникнути «сліпих» пробуджень за таймером.

1.3.2 Алгоритмічні методи оптимізації

Саме лише використання апаратних режимів сну є необхідною, але не достатньою умовою енергоефективності; вирішальну роль відіграє логіка керування циклами активності. Найпростішим підходом є циклічне увімкнення (Duty Cycling), що передбачає періодичне пробудження за фіксованим розкладом. Хоча цей метод ефективніший за постійну роботу, він має суттєвий недолік: ігнорування динаміки контрольованих процесів призводить до нераціональних витрат енергії в періоди стабільності параметрів. Вдосконаленням цього методу є агрегація даних (Data Aggregation), коли пристрій накопичує результати кількох вимірювань у пам'яті і відправляє їх єдиним пакетом. Це дозволяє скоротити кількість енергоємних процедур ініціалізації Wi-Fi та встановлення захищених з'єднань.

Найбільш перспективним підходом, що лежить в основі даного дослідження, є використання адаптивних алгоритмів. Їх суть полягає в динамічній зміні частоти вимірювань та передачі даних залежно від стану навколишнього середовища. Алгоритм працює за наступним принципом: пристрій вимірює поточні показники та порівнює їх із попередніми значеннями, що зберігаються в енергонезалежній пам'яті. Якщо зміни незначні (знаходяться в межах встановленого порогу гістерезису), система ідентифікує стан як стабільний і встановлює тривалий інтервал сну, пропускаючи сеанс передачі даних. У разі ж фіксації суттєвих змін, що можуть свідчити про важливі події, інтервал сну скорочується для забезпечення високої дискретизації моніторингу. Такий підхід забезпечує оптимальний баланс між деталізацією даних та енергозбереженням.

Логічним розвитком адаптивності є хмарно-керовані алгоритми (Cloud-driven algorithms), які передбачають перенесення центру прийняття рішень з мікроконтролера на сервер. У такій архітектурі пристрій отримує команду щодо тривалості наступного інтервалу сну безпосередньо від хмарного сервісу у відповідь на передані дані. Це дозволяє застосовувати складні математичні моделі, враховувати зовнішні фактори (прогноз погоди, час доби) та оновлювати логіку роботи без необхідності зміни прошивки пристрою. Саме поєднання вбудованих режимів Deep Sleep з адаптивними та хмарними стратегіями є ключовим напрямком для досягнення максимальної автономності систем моніторингу.

1.4. Порівняльний аналіз протоколів та архітектур передачі даних в аспекті енергоефективності

Вибір способу передачі даних є одним із вирішальних факторів, що впливають на загальне енергоспоживання автономного IoT-пристрою. Сеанс зв'язку, особливо з використанням енергоємних радіомодулів, як-от Wi-Fi, є однією з найбільш «дорогих» операцій у життєвому циклі пристрою.

1.4.1 Протоколи на базі IP-мереж (Wi-Fi)

Ця група протоколів використовує стандартний стек TCP/IP або UDP/IP і зазвичай працює в локальних мережах Wi-Fi, що забезпечує високу швидкість передачі даних та легку інтеграцію з існуючою інтернет-інфраструктурою. Найпоширенішим є HTTP/HTTPS (HyperText Transfer Protocol), який, попри свою універсальність, часто є надлишковим для IoT-пристроїв. Кожен запит вимагає встановлення нового TCP-з'єднання з триетапним рукошлякуванням, а громіздкі

текстові заголовки значно збільшують обсяг трафіку. Використання шифрування HTTPS додає ресурсоємний процес TLS-рукоштовування, що в сукупності призводить до тривалого часу активності радіомодуля та робить протокол неефективним для частої передачі невеликих обсягів телеметрії [5][6].

Спеціалізованою альтернативою є протокол MQTT (Message Queuing Telemetry Transport) [7], який працює поверх TCP, але оптимізований для мінімізації накладних витрат. Його енергоефективність забезпечується використанням легковагових бінарних заголовків (розміром від 2 байт) та моделлю «видавець-підписник», що спрощує логіку пристрою. Важливою перевагою є механізм Keep-Alive, який дозволяє підтримувати постійне з'єднання без необхідності повторних рукоштовувань, обмежуючись лише рідкісними пакетами підтвердження активності. Крім того, наявність різних рівнів якості обслуговування (QoS) дозволяє гнучко балансувати між надійністю доставки та витратами енергії.

Для пристроїв з обмеженими ресурсами також застосовується протокол CoAP (Constrained Application Protocol) [8]. Він реалізує REST-архітектуру подібно до HTTP, але працює поверх протоколу UDP, що є його ключовою перевагою. Відсутність вимог до підтримки постійного TCP-з'єднання робить сеанси зв'язку значно коротшими, а використання механізмів підтвердження доставки та протоколу безпеки DTLS забезпечує необхідну надійність при менших обчислювальних витратах.

1.4.2 Альтернативні архітектури зв'язку

Для сценаріїв, де використання Wi-Fi є неможливим через відсутність покриття або неприпустимо високе енергоспоживання, застосовуються принципово інші архітектури. Однією з них є LoRaWAN (Long Range Wide Area Network) [9][10] — протокол класу LPWAN, оптимізований для багаторічної

роботи від однієї батареї. Архітектура мережі будується за топологією «зірка», де кінцеві пристрої не мають IP-адрес і передають дані на шлюзи. Найбільш енергоефективні пристрої класу А активують радіомодуль лише на короткий проміжок часу для передачі малих пакетів даних на великі відстані (кілометри). Низька швидкість передачі у цьому випадку є компромісною платою за високу чутливість приймача та рекордну автономність.

Іншим ефективним підходом є використання Bluetooth Low Energy (BLE) [11][12] для побудови персональних мереж (PAN) з архітектурою «сенсорний вузол → шлюз». У такій схемі сенсор мінімізує споживання, відмовляючись від енергоємного Wi-Fi на користь BLE, який забезпечує швидке встановлення з'єднання та передачу даних на локальний шлюз (смартфон або хаб). Особливо ефективним є режим «Advertising», коли пристрій періодично транслює дані в ефір без встановлення з'єднання, що дозволяє досягти наднизького рівня енергоспоживання.

1.4.3 Узагальнюючий аналіз

Вибір оптимального протоколу чи архітектури залежить від конкретного сценарію застосування та обмежень проєкту. Порівняльна характеристика розглянутих технологій наведена у таблиці 1.2.

Таблиця 1.2 Характеристики протоколів

Протокол/Архітектура	Енергоспоживання	Пропускна здатність	Радіус дії	Складність інфраструктури	Типовий сценарій
HTTP/HTTPS	Високе	Висока	Wi-Fi (до 100 м)	Низька (стандартна)	Рідкісні запити,

Протокол/Архітектура	Енергоспоживання	Пропускна здатність	Радіус дії	Складність інфраструктури	Типовий сценарій
				Wi-Fi (мережа)	передача великих файлів
MQTT	Низьке	Висока	Wi-Fi (до 100 м)	Середня (потрібен MQTT-брокер)	Часта передача телеметрії в реальному часі
CoAP	Дуже низьке	Висока	Wi-Fi (до 100 м)	Низька	Енергоефективна альтернатива HTTP
BLE	Наднизьке	Середня	PAN (до 50 м)	Висока (потрібен шлюз)	Носимі пристрої, локальні сенсорні мережі
LoRaWAN	Наднизьке	Дуже низька	LPWAN (кілометри)	Висока (потрібен LoRaWAN-шлюз)	Віддалений моніторинг (агро, екологія)

Підсумовуючи проведений аналіз, можна стверджувати, що для системи моніторингу кліматичних показників у зоні покриття Wi-Fi найбільш перспективними є протоколи MQTT та CoAP. Протокол HTTP може розглядатися як базовий варіант для порівняльної оцінки ефективності оптимізації. Водночас архітектури на базі BLE та LoRaWAN є критично важливими альтернативами для

специфічних умов розгортання, де пряме підключення до IP-мережі є недоцільним. Розуміння сильних та слабких сторін кожної технології дозволяє обґрунтовано підходити до проєктування енергоефективної системи та визначення меж застосовності розроблюваного рішення.

1.5 Виявлення не вирішених аспектів проблеми та постановка задачі дослідження

Проведений у попередніх підрозділах комплексний аналіз стану питання дозволяє стверджувати, що проблема енергоефективності є центральною для розвитку автономних ІТ-систем моніторингу на базі Інтернету речей. Розгляд сучасних апаратних платформ (зокрема ESP32), програмних методів оптимізації та різноманітних комунікаційних архітектур засвідчив наявність широкого інструментарію для вирішення цієї проблеми. Однак, незважаючи на значну кількість публікацій, на перетині цих технологій залишається низка прогалин, які формують науково-практичний інтерес для подальшої роботи.

1.5.1 Невирішені аспекти проблеми

Перш за все, у науковій літературі спостерігається дефіцит комплексних порівняльних досліджень різних архітектур зв'язку для ідентичних прикладних задач. Більшість існуючих робіт фокусуються на оптимізації в межах однієї технології, наприклад, порівнюючи MQTT та HTTP через Wi-Fi або аналізуючи переваги LoRaWAN. Натомість бракує експериментальних даних, отриманих у контрольованих умовах, які б дозволили прямо зіставити ефективність принципово різних підходів — прямого підключення до хмари, використання локального

шлюзу (BLE) та глобального шлюзу (LoRaWAN) — при виконанні однакової задачі моніторингу з єдиним адаптивним алгоритмом.

Другим важливим аспектом є недостатня вивченість гібридних архітектур керування енергоспоживанням у мережах із різною затримкою. Концепція перенесення логіки прийняття рішень з пристрою у хмарне середовище (наприклад, на AWS Lambda) є відносно новою, і вплив комунікаційного середовища на її ефективність досліджено фрагментарно. Зокрема, відсутній аналіз компромісу між перевагами хмарних обчислень та обмеженнями, що накладаються затримками передачі даних у мережах Wi-Fi та LoRaWAN. Крім того, потребує систематизації питання сукупного впливу різних рівнів оптимізації. Існує потреба в експериментальному визначенні синергетичного ефекту від поєднання апаратних режимів сну, використання ULP-співпроцесора та адаптивних алгоритмів, а також у виявленні внеску кожного з цих рівнів у загальне зниження енергоспоживання.

1.5.2 Постановка задачі дослідження

На основі виявлених прогалин формулюється науково-технічна проблема, яка полягає у відсутності єдиного, експериментально підтвердженого підходу до вибору та оптимізації архітектури зв'язку для енергоефективних IoT-систем, який би інтегрував апаратні можливості, адаптивні алгоритми та гібридну логіку керування.

Виходячи з цього, метою даної магістерської роботи є розробка та експериментальне дослідження способу оптимізації енергоспоживання IT-системи моніторингу кліматичних показників шляхом створення адаптивного алгоритму збору та передачі даних, а також порівняльна оцінка його ефективності при реалізації з використанням різних архітектур та протоколів передачі даних (Wi-Fi, BLE, LoRaWAN).

Для досягнення поставленої мети передбачено виконання такої послідовності дій:

1. Проведення системного аналізу апаратних платформ та комунікаційних архітектур для класифікації факторів впливу на енергоспоживання та виявлення обмежень існуючих рішень.
2. Розробка архітектури багаторівневої системи моніторингу, яка інтегрує апаратні режими сну, адаптивну логіку та хмарні сервіси, забезпечуючи гнучкість зміни комунікаційної технології.
3. Програмна реалізація запропонованого адаптивного алгоритму для мікроконтролера ESP32 у кількох варіаціях: для прямого зв'язку з хмарою через Wi-Fi (протоколи MQTT, HTTP, CoAP), для локальної передачі через шлюз (BLE) та для віддаленого моніторингу через мережевий сервер (LoRaWAN).
4. Проєктування універсального вимірювального стенду та розробка методики експериментальних досліджень для об'єктивного вимірювання струму споживання, оцінки автономності та якості передачі даних.
5. Проведення натурних експериментів, аналіз отриманих даних для визначення найбільш енергоефективної архітектури в різних сценаріях та формування практичних рекомендацій щодо проєктування подібних систем.

Висновки до розділу

У першому розділі магістерської дисертації здійснено комплексний аналіз проблеми підвищення енергоефективності автономних ІТ-систем моніторингу, що функціонують на базі технологій Інтернету речей. На основі огляду науково-технічної літератури встановлено, що саме енергоефективність виступає критичним фактором, який визначає не лише тривалість автономної роботи, але й економічну доцільність та надійність експлуатації розподілених сенсорних мереж.

За результатами порівняльного аналізу апаратних платформ обґрунтовано вибір мікроконтролера ESP32 як оптимальної елементної бази для дослідження. Це рішення продиктоване збалансованим поєднанням високої обчислювальної потужності, наявністю інтегрованих радіоінтерфейсів Wi-Fi та BLE, а також гнучкістю доступних режимів енергозбереження, зокрема наявністю унікального ULP-співпроцесора для фонових моніторингу.

У ході дослідження систематизовано програмні методи оптимізації, серед яких пріоритетними визначено керування режимами глибокого сну та застосування адаптивних алгоритмів збору даних. Особливу увагу приділено аналізу комунікаційної складової: проведено співставлення традиційних протоколів IP-мереж (HTTP, MQTT, CoAP) та спеціалізованих енергоефективних технологій (BLE, LoRaWAN), що дозволило виявити їхні переваги та обмеження в контексті автономного живлення.

Підсумовуючи аналіз, виявлено суттєві прогалини в існуючих підходах, зокрема відсутність комплексних експериментальних порівнянь ефективності різних комунікаційних архітектур при вирішенні ідентичних прикладних задач. Також констатовано недостатню вивченість синергетичного ефекту від інтеграції апаратних, програмних та архітектурних методів оптимізації в єдину систему. Визначення цих невирішених аспектів дозволило сформулювати науково-технічну проблему, мету та задачі даного дослідження, спрямовані на створення та експериментальну верифікацію універсального способу мінімізації енергоспоживання систем моніторингу.

2. МЕТОДИ, ЗАСОБИ ТА ПІДХОДИ ДО ДОСЛІДЖЕННЯ

2.1 Методологічна основа дослідження

Для досягнення поставленої мети та вирішення визначених задач дане магістерське дослідження ґрунтується на комплексному експериментально-аналітичному підході. Ця наукова парадигма поєднує теоретичний аналіз існуючих рішень, системне проєктування, програмно-апаратну реалізацію прототипів та проведення серії контрольованих експериментів для отримання кількісних, емпірично підтверджених оцінок ефективності. Вибір такої методології є принциповим, оскільки вона дозволяє не лише розробити новий спосіб оптимізації, але й обґрунтовано довести його переваги та визначити межі застосовності шляхом прямого вимірювання ключових показників продуктивності, надійності та енергоспоживання. На відміну від суто теоретичних або симуляційних досліджень, експериментально-аналітичний метод враховує реальні фізичні процеси та непередбачувані фактори, що виникають при роботі апаратного забезпечення та взаємодії з мережевою інфраструктурою.

Процес дослідження структуровано у вигляді послідовних взаємопов'язаних етапів. Початковою фазою є теоретичний та системний аналіз, який формує науковий фундамент роботи. Він передбачає систематичний огляд науково-технічної літератури, стандартів та публікацій у провідних наукометричних базах (IEEE Xplore, Scopus, Web of Science) за ключовими напрямками енергоефективності IoT. Це дозволяє сформулювати уявлення про існуючі підходи та виявити невирішені проблеми. На цьому ж етапі здійснюється декомпозиція глобальної задачі оптимізації на три рівні: апаратний (аналіз механізмів Deep Sleep, ULP), програмний (розробка адаптивної логіки) та комунікаційний (аналіз впливу архітектури зв'язку).

Наступним кроком є системне проєктування та моделювання, де закладається концептуальна основа майбутньої системи. Цей етап включає розробку узагальненої модульної архітектури ІТ-системи моніторингу для трьох основних

конфігурацій: прямого підключення (Wi-Fi), локального шлюзу (BLE) та глобального шлюзу (LoRaWAN). Для візуалізації взаємодії компонентів застосовується уніфікована мова моделювання UML. Паралельно формується концептуальна модель адаптивного алгоритму, описана у вигляді скінченного автомата, ключовим елементом якої є функція розрахунку оптимального інтервалу сну на основі динаміки кліматичних показників.

Етап програмно-апаратної реалізації полягає у перетворенні теоретичних моделей на фізичний прототип. Для цього створюється універсальний вимірювальний стенд на базі мікроконтролера ESP32 та сенсора BME680, оснащений засобами прецизійного вимірювання струму. Програмне забезпечення розробляється ітеративно в середовищі PlatformIO, при цьому код структурується модульно для забезпечення швидкого перемикавання між протоколами (MQTT, HTTP, CoAP, BLE, LoRaWAN) та версіями алгоритмів, що є критично важливим для чистоти експерименту.

Центральним етапом роботи є експериментальне дослідження, спрямоване на отримання об'єктивних даних. Методика дослідження передбачає чітке визначення незалежних змінних (тип протоколу, версія алгоритму, умови середовища) та контрольованих параметрів. Проводиться серія систематичних вимірювань енергоспоживання для різних сценаріїв роботи — від стабільних умов до динамічних змін, що дозволяє оцінити адаптивність алгоритму. Багаторазове повторення експериментів забезпечує статистичну значущість результатів.

Завершальним етапом є аналіз результатів та формування висновків. Отримані експериментальні дані підлягають статистичній обробці та візуалізації за допомогою платформи Grafana. Методологія аналізу передбачає гібридний підхід до зберігання даних: Amazon DynamoDB використовується для аналізу історичних трендів, а Redis — для моніторингу в реальному часі. На основі розрахованих показників (середніх значень, стандартних відхилень) проводиться порівняльний аналіз ефективності, формуються наукові висновки та розробляються практичні рекомендації для проектування енергоефективних IoT-систем. Такий комплексний

підхід забезпечує високий ступінь достовірності результатів і надає роботі значну практичну цінність.

2.2 Обґрунтування вибору апаратних засобів дослідження

Вибір апаратної платформи є фундаментальним етапом у проектуванні будь-якої вбудованої системи, а для енергоефективних IoT-пристроїв він набуває критичного значення. Апаратні компоненти безпосередньо визначають базовий рівень енергоспоживання, обчислювальні можливості, доступні комунікаційні інтерфейси та, як наслідок, саму можливість реалізації складних адаптивних алгоритмів. Для даного дослідження апаратний комплекс обирався за критеріями гнучкості, мінімального енергоспоживання у режимах сну, наявності необхідних бездротових модулів та широкої підтримки з боку спільноти розробників. Нижче наведено детальне обґрунтування вибору кожного з ключових компонентів експериментального стенду.

2.2.1 Мікроконтролерна платформа ESP32

В якості центрального обчислювального вузла системи обрано мікроконтролер ESP32 (плата розробника ESP-WROOM-32). Цей вибір ґрунтується на унікальному поєднанні високої продуктивності, широких комунікаційних можливостей та, що найважливіше для даного дослідження, розширених механізмів керування енергоспоживанням [13].

Архітектура ESP32 базується на двоядерному 32-бітному процесорі Tensilica Xtensa LX6 з тактовою частотою до 240 МГц. Така обчислювальна потужність у поєднанні з 520 КБ статичної оперативної пам'яті (SRAM) забезпечує достатній

ресурс для обробки даних із сенсорів, реалізації складних алгоритмів та виконання криптографічних операцій (TLS/SSL) без суттєвих затримок. Крім того, наявність інтегрованих модулів Wi-Fi (802.11 b/g/n) та Bluetooth (v4.2 BR/EDR і BLE) дозволяє в межах одного апаратного стенду досліджувати та порівнювати ефективність трьох принципово різних комунікаційних архітектур.

Ключовою перевагою платформи є гнучка система керування живленням. У режимі повної функціональності (Active Mode) споживання струму становить 80–260 мА, проте для його зниження передбачено проміжні стани, такі як Modem-Sleep (20–30 мА) та Light-Sleep (0,8 мА). Центральним елементом дослідження є режим глибокого сну (Deep-Sleep), у якому відключаються процесорні ядра та більшість периферії, а споживання падає до 10–20 мкА. Унікальною особливістю ESP32 є наявність ULP-співпроцесора (Ultra Low Power), який здатен виконувати найпростіші програми моніторингу сенсорів навіть під час глибокого сну основного процесора, ініціюючи пробудження системи лише при перевищенні заданих порогових значень.

2.2.2 Сенсорний модуль BME680

Для збору кліматичних даних використано інтегрований модуль BME680 від компанії Bosch Sensortec. Цей вибір зумовлений його багатofункціональністю, оскільки на одному кристалі об'єднано чотири сенсори: температури (точність ± 1.0 °C), відносної вологості ($\pm 3\%$), барометричного тиску (± 1 hPa) та датчик летких органічних сполук (VOC) для оцінки якості повітря [14]. Висока інтеграція дозволяє спростити схемотехніку та знизити загальне енергоспоживання системи порівняно з використанням окремих датчиків.

Сенсор оптимізовано для роботи в автономних пристроях: у режимі очікування він споживає менше 1 мкА. Важливою характеристикою є можливість програмного керування параметрами вимірювань, такими як кратність

передискретизації (oversampling) та коефіцієнти ІІР-фільтрації. Це дозволяє гнучко адаптувати роботу модуля до конкретних умов, знаходячи оптимальний баланс між точністю даних та витратами енергії.

2.2.3 Апаратні засоби для реалізації BLE та LoRaWAN архітектур

Для повноцінного порівняльного дослідження базові можливості ESP32 було розширено додатковими апаратними засобами. Дослідження архітектури Bluetooth Low Energy (BLE) реалізовано за схемою з локальним шлюзом, що вимагає використання двох пристроїв. Перший — це сенсорний вузол на базі ESP32 та BME680, який працює автономно з вимкненим Wi-Fi, передаючи дані виключно через BLE (механізм Advertising). Другий пристрій виконує роль шлюзу (Gateway): він має постійне живлення, сканує ефір, приймає дані від сенсора та пересилає їх до хмарної інфраструктури AWS через Wi-Fi.

Для реалізації архітектури LoRaWAN до основного сенсорного вузла додано зовнішній трансивер на базі чіпа Semtech SX1276 (модуль HopeRF RFM95W), підключений через інтерфейс SPI [15]. Це забезпечує передачу даних на відстань до кількох кілометрів при низькому енергоспоживанні. Прийом даних здійснюється через LoRaWAN-шлюз, підключений до мережі The Things Network (TTN), що забезпечує подальшу інтеграцію з хмарною платформою.

Таким чином, сформований набір апаратних засобів створює гнучкий та репрезентативний вимірювальний стенд, що дозволяє провести комплексне порівняння різних підходів до побудови енергоефективних систем моніторингу.

2.3 Обґрунтування вибору програмних засобів та середовища розробки

Вибір програмного стеку технологій є не менш важливим етапом, ніж підбір апаратних компонентів, оскільки саме він визначає швидкість розробки, надійність, гнучкість системи та потенціал для подальшої оптимізації. Програмний інструментарій для даного дослідження формувався з урахуванням необхідності підтримки широкого спектра комунікаційних протоколів, забезпечення безпечної взаємодії з хмарною інфраструктурою та реалізації багаторівневих алгоритмів керування енергоспоживанням. Нижче наведено детальне обґрунтування вибору кожного елемента програмного комплексу.

2.3.1 Середовище розробки та фреймворк

В якості основного середовища розробки обрано PlatformIO IDE в інтеграції з редактором Visual Studio Code. На відміну від стандартного Arduino IDE, це рішення надає набір професійних інструментів, критично важливих для складного проєкту. Зокрема, PlatformIO забезпечує гнучке керування залежностями та ізоляцію проєктів через конфігураційні файли, що гарантує відтворюваність збірки. Інтеграція з VS Code надає потужні засоби статичного аналізу коду, автодоповнення та рефакторингу, а вбудована підтримка Git спрощує контроль версій. Крім того, наявність інструментів для апаратного налагодження (debugging) дозволяє здійснювати покрокове виконання коду та аналіз стану регістрів мікроконтролера.

Незважаючи на використання професійного IDE, базовим програмним фреймворком обрано Arduino Framework. Такий вибір зумовлений високим рівнем абстракції, що спрощує роботу з периферією ESP32, та наявністю розгалуженої екосистеми протестованих бібліотек для роботи з сенсорами та протоколами

(MQTT, LoRaWAN, BLE). Це дозволяє зосередити увагу на реалізації алгоритмічної логіки, мінімізуючи час на написання низькорівневих драйверів.

2.3.2 Мова програмування

Розробка вбудованого програмного забезпечення здійснюється мовою C++, яка є стандартом для екосистеми Arduino. Вибір C++ обґрунтований його високою продуктивністю та ефективністю використання обмежених ресурсів мікроконтролера. Підтримка об'єктно-орієнтованого підходу дозволяє створювати модульну архітектуру, інкапсулюючи логіку роботи з окремими сенсорами та протоколами у класи, що значно полегшує масштабування та підтримку коду.

2.3.3 Ключові бібліотеки для прошивки

Для реалізації функціоналу системи дібрано набір стабільних бібліотек. Взаємодія з сенсором BME680 забезпечується драйверами від Adafruit, які є галузевим стандартом. Для роботи з Wi-Fi протоколами використовуються бібліотека PubSubClient (для реалізації легковагого MQTT-клієнта) та HTTPClient (для захищених HTTPS-запитів). Підтримка технології Bluetooth Low Energy реалізована через нативні бібліотеки ESP32 BLE Arduino, а для роботи в мережах LoRaWAN застосовано стек MCCI LoRaWAN LMIC, що забезпечує відповідність регіональним стандартам. Серіалізація та парсинг даних здійснюються за допомогою бібліотеки ArduinoJson, яка оптимізована для роботи в умовах обмеженої оперативної пам'яті.

2.3.4 Хмарна інфраструктура та сервіси

Серверну частину системи побудовано на базі платформи Amazon Web Services (AWS), яка забезпечує високу надійність та масштабованість. Центральним вузлом прийому даних виступає сервіс AWS IoT Core [16], що гарантує безпечне підключення через сертифікати X.509 та маршрутизацію повідомлень. Бізнес-логіка адаптивних алгоритмів реалізована за допомогою безсерверних функцій AWS Lambda [17], які виконують аналіз даних та приймають рішення про зміну режимів роботи пристроїв.

Для зберігання даних застосовано гібридну модель. Історичні дані для довготривалого аналізу накопичуються у NoSQL базі даних Amazon DynamoDB, тоді як для забезпечення моніторингу в реальному часі використовується швидкісний кеш Amazon ElastiCache (Redis). Така архітектура дозволяє розділити потоки «гарячих» та «холодних» даних, оптимізуючи швидкодію системи. Для інтеграції пристроїв LoRaWAN задіяно мережевий сервер The Things Network (TTN), який пересилає розшифровані пакети до AWS. Візуалізація результатів здійснюється на платформі Grafana, яка об'єднує дані з обох типів сховищ для побудови комплексних інформаційних панелей.

2.4 Формування концептуальної моделі адаптивного алгоритму

Ключовим елементом даного дослідження є розробка та обґрунтування концептуальної моделі адаптивного алгоритму, спрямованого на динамічне керування енергоспоживанням IoT-пристрою. На відміну від традиційних підходів із фіксованим інтервалом збору та передачі даних, які є неефективними через надлишкові цикли активності у стабільних умовах, запропонована модель базується на принципі інтелектуальної адаптації. Суть цього принципу полягає в

тому, що частота активності пристрою (вимірювання та передача) безпосередньо залежить від динаміки змін кліматичних показників, що дозволяє досягти оптимального балансу між актуальністю даних та тривалістю автономної роботи.

2.4.1 Модель скінченного автомата (Finite State Machine)

Для формального опису поведінки IoT-пристрою та логіки його роботи найбільш доцільно використати модель скінченного автомата (Finite State Machine, FSM). Ця модель дозволяє чітко визначити всі можливі стани системи та умови переходів між ними, що робить алгоритм структурованим, передбачуваним та легким для реалізації.

Функціонування енергоефективного вузла моніторингу можна описати як цикли переходів між п'ятьма основними станами, логіку взаємодії яких наведено на рисунку 2.1.

Базовим є стан сну (STATE_SLEEP), у якому пристрій перебуває більшу частину часу для максимальної економії енергії, споживаючи мінімальний струм у режимі Deep Sleep. Вихід із цього стану ініціюється таймером або зовнішнім сигналом, після чого система переходить у фазу пробудження та вимірювання (STATE_WAKEUP & MEASUREMENT), де відбувається ініціалізація периферії та зчитування даних із сенсорів.

Отримані дані обробляються у стані аналізу (STATE_ANALYSIS), де відбувається порівняння поточних значень із попередніми для прийняття рішення про необхідність комунікації. Якщо виявлено значні зміни, система переходить у найбільш енергозатратний стан передачі (STATE_TRANSMISSION) для активації радіомодуля та відправки даних. В архітектурах із двостороннім зв'язком передбачено опціональний стан очікування команди (STATE_AWAIT_COMMAND), коли пристрій залишається активним для

отримання зворотного зв'язку від сервера (наприклад, нового інтервалу сну). Після завершення активних операцій цикл замикається поверненням у режим сну.

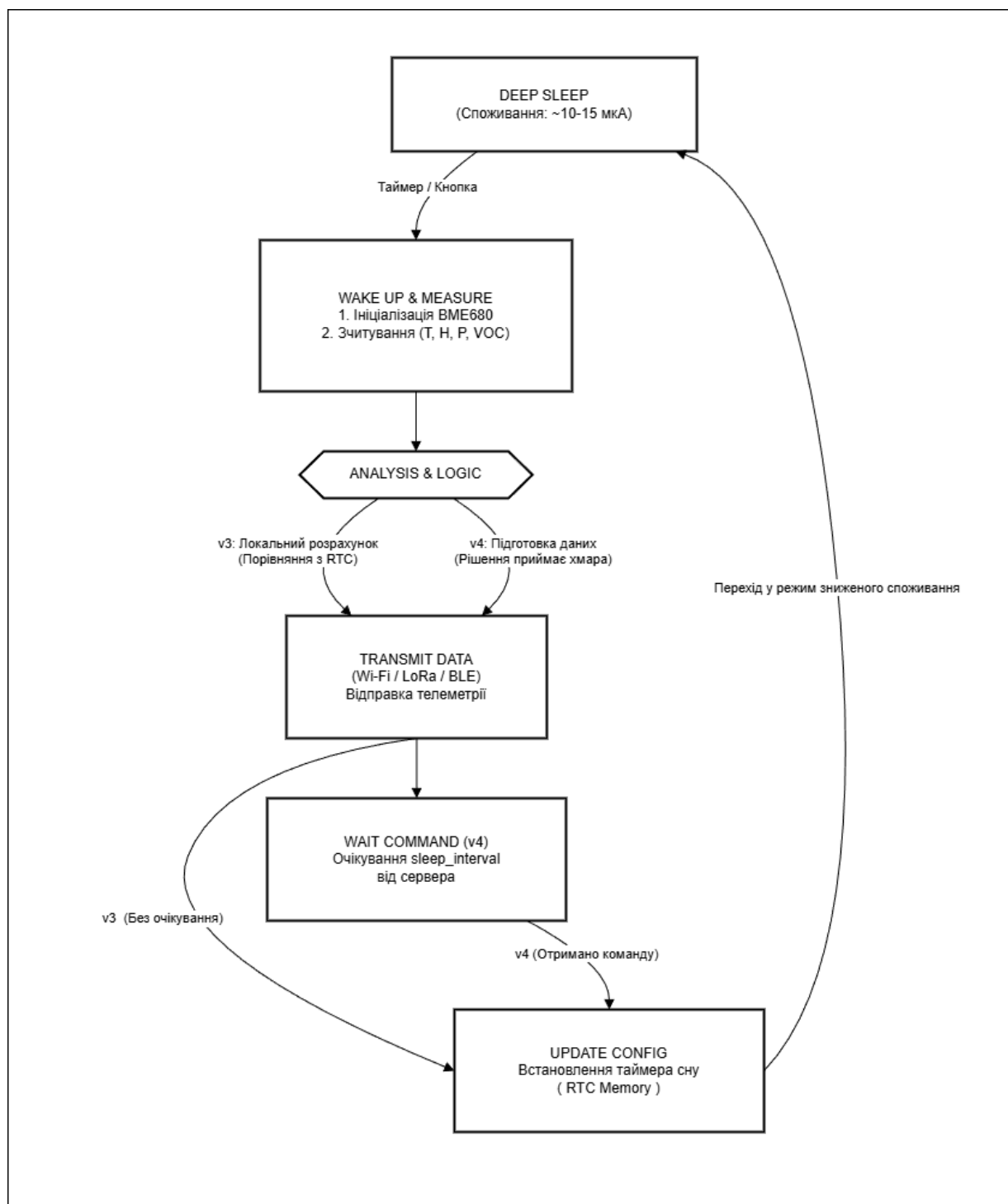


Рисунок 2.1 — Діаграма станів (FSM) адаптивного алгоритму роботи пристрою

2.4.2 Логіка прийняття рішень: функція адаптації

Центральним елементом моделі є функція, що визначає необхідність передачі даних та розраховує тривалість наступного циклу сну. Реалізація цієї функції можлива у двох парадигмах.

Локальна модель адаптації (Device-centric) передбачає інкапсуляцію всієї логіки прийняття рішень безпосередньо у прошивці мікроконтролера. Алгоритм базується на порівнянні поточних вимірювань із попередніми з використанням системи порогових значень. Якщо зміни не перевищують «порог стабільності», пристрій ідентифікує умови як незмінні, скасовує передачу даних та пропорційно збільшує інтервал сну до визначеного максимуму (наприклад, 10–15 хвилин), що є ключовим механізмом економії. При перевищенні «порогу значущості» дані передаються зі стандартною частотою, а у випадку досягнення «порогу аномалії» (різких змін) інтервал сну мінімізується для детального відстеження динаміки процесу.

Альтернативна хмарна модель адаптації (Cloud-centric) перетворює пристрій на виконавчий механізм типу «сенсор-актуатор», виносячи обчислювальну логіку на сервер (наприклад, AWS Lambda). У цьому випадку пристрій передає «сирі» дані у хмару, де серверна логіка, маючи доступ до повної історії вимірювань та необмежених обчислювальних ресурсів, виконує глибокий аналіз (включаючи виявлення трендів або використання машинного навчання). Результатом обробки є команда з оптимальним інтервалом сну, яку пристрій отримує та виконує. Такий підхід мінімізує навантаження на мікроконтролер та забезпечує гнучкість оновлення алгоритмів.

2.4.3 Адаптація моделі до різних комунікаційних архітектур

Запропонована концептуальна модель є універсальною, проте її практична реалізація адаптується до специфіки конкретних каналів зв'язку.

Для архітектури Wi-Fi (MQTT, HTTP, CoAP), де стан передачі є найбільш енергоємним через ініціалізацію IP-стеку, адаптивний алгоритм фокусується на мінімізації кількості сесій зв'язку. Тут ефективно реалізуються як локальна, так і хмарна моделі адаптації.

У випадку використання Bluetooth Low Energy (BLE) модель трансформується для схеми «сенсор-шлюз». У режимі Advertising адаптивний алгоритм динамічно змінює частоту розсилки рекламних пакетів, знижуючи її в стабільних умовах. Для режиму Connection логіка адаптації може бути делегована шлюзу, який керує частотою підключень до сенсорного вузла.

Для мереж LoRaWAN адаптація виконує подвійну функцію: заощадження енергії та дотримання жорстких обмежень на ефірний час (duty cycle). Відправка даних виключно при значущих змінах стає необхідною умовою функціонування. Хмарна модель тут є найбільш природною, оскільки проходження даних через мережевий сервер дозволяє легко інтегрувати аналітичні сервіси для керування параметрами пристрою.

Таким чином, сформована концептуальна модель на базі скінченного автомата та функції динамічної адаптації створює гнучку основу для розробки енергоефективної системи моніторингу.

2.5 Методи оцінки енергоспоживання та точності даних

Для об'єктивного та науково обґрунтованого порівняння ефективності розроблених архітектур та алгоритмів необхідно визначити чітку систему

вимірювання. Оцінка проводиться за двома ключовими, хоча й частково суперечливими, напрямками: енергоспоживання,

2.5.1 Методи оцінки енергоспоживання

Енергоспоживання є центральним об'єктом даного дослідження, тому його оцінка базується на прямих апаратних вимірюваннях споживаного струму на різних етапах робочого циклу. Такий підхід дозволяє отримати точні кількісні показники та уникнути похибок, властивих програмним симуляціям. Вимірювальний лабораторний стенд включає прецизійний цифровий мультиметр (або спеціалізований профайлер потужності типу Joulescope), підключений у розрив ланцюга живлення мікроконтролера, а також акумулятор відомої ємності (наприклад, Li-Po 1000 мА·год) як джерело живлення. Основним показником ефективності є середній струм споживання (I_{avg}) — інтегральна величина, що усереднює споживання за повний робочий цикл, включаючи фази активності та сну. Він розраховується за формулою:

$$I_{avg} = \frac{I_{active} * T_{active} + I_{sleep} * T_{sleep}}{T_{active} + T_{sleep}} \quad (1)$$

де I_{active} та T_{active} - струм і тривалість активної фази, а I_{sleep} та T_{sleep} - струм і тривалість фази сну.

Для порівняння різних режимів роботи в уніфікованих одиницях використовується метрика сумарного добового енергоспоживання (E_{day}), яка виражається у мВт·год/добу та визначається добутком середнього струму, напруги живлення (U) та тривалості доби (24 години):

$$E_{day} = I_{avg} * U * 24 \quad (2)$$

Ключовим практичним показником є розрахунковий час автономної роботи ($T_{autonomy}$), що демонструє тривалість функціонування пристрою від одного заряду. Він залежить від ємності акумулятора ($C_{battery}$) та коефіцієнта ефективності розряду (K_{eff}), який зазвичай приймається на рівні 0,8–0,9:

$$T_{autonomy} = \frac{C_{battery} * K_{eff}}{I_{avg}} \quad (3)$$

2.5.2 Методи оцінки якості та актуальності даних

Зниження енергоспоживання шляхом збільшення інтервалів сну неминуче призводить до компромісу з актуальністю даних, тому критично важливо оцінити, наскільки розроблені адаптивні алгоритми зберігають інформативність моніторингу. Для оцінки реактивності системи використовується метрика середньої затримки виявлення події ($T_{autonomy}$). Вона показує проміжок часу від моменту значної зміни кліматичного параметра (наприклад, стрибка температури) до моменту її фіксації сервером. Ризик пропуску короткочасних змін характеризується коефіцієнтом втрати значущих подій (SER_{loss_rate}), який розраховується як відношення кількості пропущених подій до їх загальної кількості за період спостереження (порівняно з еталонним пристроєм безперервного моніторингу):

$$SER_{loss_rate} = \frac{N_{missed_events}}{N_{total_events}} * 100\% \quad (4)$$

Додатково оцінюється надійність передачі даних, що є особливо важливим для протоколів без гарантованої доставки (наприклад, CoAP поверх UDP). Ця метрика

визначається як відсоток успішно доставлених пакетів від загальної кількості відправлених.

Висновки до розділу

У другому розділі магістерської дисертації сформовано теоретичний та методологічний фундамент для проведення експериментального дослідження. Обґрунтовано доцільність використання комплексного експериментально-аналітичного підходу, який дозволяє отримати верифіковані емпіричні дані ефективності на реальному обладнанні. Реалізація цього підходу базується на сформованій оптимальній апаратній платформі, ключовими елементами якої стали мікроконтролер ESP32, мультисенсор BME680 та додаткові комунікаційні модулі. Їх вибір продиктований функціональною гнучкістю та наявністю розширених механізмів керування енергоспоживанням, що є критично важливим для цілей роботи.

Програмна реалізація системи спирається на сучасний стек технологій (PlatformIO, C++, FreeRTOS) та хмарну інфраструктуру AWS (IoT Core, Lambda, DynamoDB, Redis), що забезпечує інструментарій для безпечної передачі, аналізу та візуалізації даних. Це створило середовище для впровадження розробленої універсальної концептуальної моделі адаптивного алгоритму. Модель побудована на базі скінченного автомата та передбачає гібридну логіку керування (локальну та хмарну), адаптовану до різних комунікаційних архітектур (Wi-Fi, BLE, LoRaWAN).

Для об'єктивної оцінки ефективності запропонованих рішень розроблено чітку систему метрик, що фокусується на показниках середнього струму споживання, розрахункового часу автономної роботи та якості (актуальності) даних. Визначена методика вимірювань дозволяє коректно порівнювати енергоефективність різних протоколів передачі даних та оцінювати реальний вплив оптимізації на автономність системи. Таким чином, у розділі повністю сформовано

методологічний апарат дослідження, створено необхідні передумови для переходу до етапу практичної розробки програмного забезпечення та проведення натурних експериментів.

3. РОЗРОБКА ЗАПРОПОНОВАНОГО РІШЕННЯ

Для забезпечення чистоти експерименту, верифікованості отриманих результатів та можливості проведення коректного порівняльного аналізу було спроектовано та виготовлено універсальний експериментальний стенд. Його ключовою особливістю є модульна архітектура, яка дозволяє тестувати широкий спектр комунікаційних протоколів та алгоритмічних підходів на єдиній апаратній базі, мінімізуючи вплив апаратних відмінностей на результати вимірювань енергоефективності. Загальну структуру взаємодії компонентів розробленої системи наведено на рисунку 3.1.

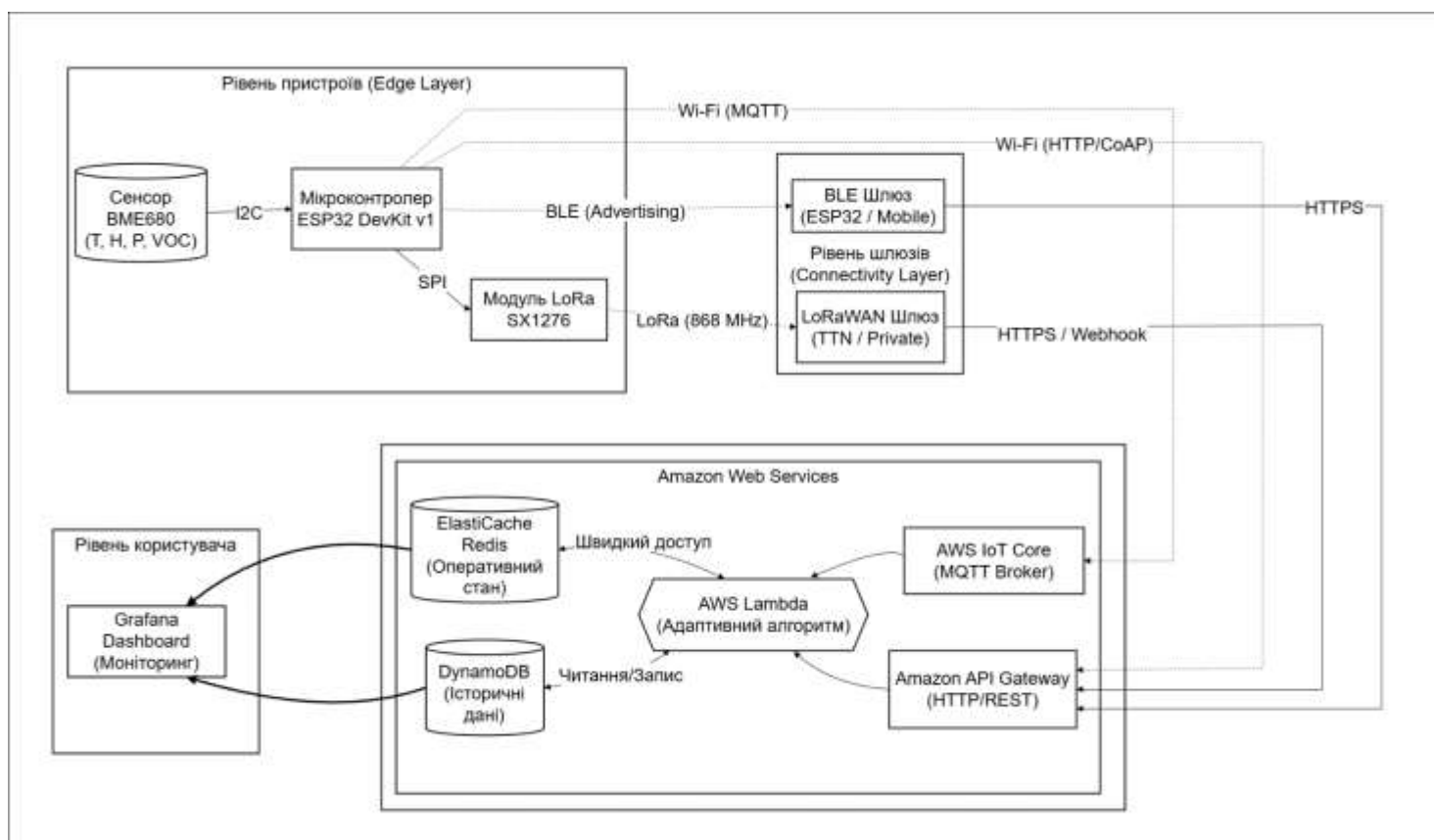


Рисунок 3.1 — Структурна схема архітектури розробленої системи моніторингу

3.1. Загальна архітектура програмного забезпечення та апаратного стенду

Для забезпечення чистоти експерименту, верифікованості отриманих результатів та можливості проведення коректного порівняльного аналізу було спроектовано та виготовлено універсальний експериментальний стенд. Його ключовою особливістю є модульна архітектура, яка дозволяє тестувати широкий спектр комунікаційних протоколів та алгоритмічних підходів на єдиній апаратній базі, мінімізуючи вплив апаратних відмінностей на результати вимірювань енергоефективності.

3.1.1 Апаратна архітектура експериментального стенду

Апаратна складова системи базується на компонентах, що відповідають вимогам низького енергоспоживання та функціональної гнучкості. Стенд реалізовано на безпайковій макетній платі, що дозволяє оперативно змінювати конфігурацію модулів.

В якості обчислювального ядра системи обрано плату розробника ESP32 DevKit v1 на базі мікроконтролера ESP-WROOM-32. Вибір цієї платформи обумовлений поєднанням критично важливих характеристик: двоядерна архітектура Tensilica Xtensa LX6 (240 МГц) дозволяє ефективно розподіляти обчислювальне навантаження, а інтеграція трансиверів Wi-Fi та Bluetooth на одному кристалі спрощує схемотехніку. Вирішальним фактором є підтримка режиму глибокого сну (Deep Sleep), у якому споживання знижується до 10–15 мкА, а також наявність ULP-співпроцесора для реалізації фонових моніторингу.

Сенсорна підсистема реалізована на базі інтегрованого цифрового датчика BME680 від Bosch Sensortec, який поєднує вимірювання температури, вологості, тиску та рівня летких органічних сполук (VOC). Модуль підключено до

мікроконтролера через інтерфейс I²C, використовуючи піни GPIO 21 (SDA) та GPIO 22 (SCL), та живиться напругою 3.3 В безпосередньо від стабілізатора плати.

Для реалізації архітектури глобальної мережі (LPWAN) систему доповнено зовнішнім трансивером на базі чіпа Semtech SX1276, що працює на частоті 868 МГц. Підключення модуля здійснено через інтерфейс SPI за оптимізованою схемою, яка уникає конфліктів із внутрішніми ресурсами ESP32: задіяно піни SCK (GPIO 18), MISO (GPIO 19), MOSI (GPIO 23) та NSS (GPIO 5). Для керування сигналами скидання та переривань використовуються піни GPIO 16 та GPIO 17 відповідно. Така конфігурація гарантує стабільну роботу бібліотеки LMIC та коректний вихід із режимів сну.

Елементи керування та діагностики включають апаратну кнопку на піні GPIO 4, яка слугує джерелом зовнішнього переривання для примусового пробудження пристрою. Живлення стенду забезпечується від літій-полімерного акумулятора або через USB-інтерфейс, у розрив якого підключено спеціалізований USB-тестер з функцією логування. Це дозволяє фіксувати профілі споживання струму з високою дискретизацією, виявляючи пікові навантаження під час роботи радіомодулів.

3.1.2 Загальна архітектура програмного забезпечення

Програмна складова системи розроблена мовою C++ у середовищі Arduino IDE з дотриманням принципів модульності та уніфікації. Для забезпечення ідентичних умов експерименту всі змінні налаштування винесено в окремий заголовний файл `credentials.h`. Цей файл централізовано зберігає параметри підключення до Wi-Fi, налаштування NTP-серверів, адреси MQTT-брокера та шлюзів, а також криптографічні ключі (сертифікати AWS, ключі LoRaWAN) і константи алгоритмів.

Для реалізації енергоефективних режимів роботи (версії v2, v3, v4) застосовано архітектурний патерн «Wake-and-Sleep». На відміну від класичної

структури Arduino-програм, де логіка виконується у нескінченному циклі `loop()`, у розробленому рішенні вся бізнес-логіка перенесена у функцію ініціалізації `setup()`. Алгоритм передбачає лінійну послідовність дій: після пробудження (за таймером або подією) мікроконтролер виконує ініціалізацію сенсорів, проводить вимірювання, обробляє дані та здійснює їх передачу. Після завершення цих операцій викликається функція переходу в режим `Deep Sleep`, при цьому пам'ять очищується (за винятком області `RTC`), а виконання програми зупиняється. Функція `loop()` залишається порожньою, що гарантує знаходження пристрою в режимі мінімального споживання до наступного циклу.

3.2 Реалізація архітектури прямого підключення до хмари (Wi-Fi)

Архітектура прямого підключення базується на використанні вбудованого в ESP32 модуля Wi-Fi для безпосередньої взаємодії з хмарними сервісами через стек протоколів TCP/IP або UDP/IP. Цей сценарій є найбільш поширеним для IoT-пристроїв, що експлуатуються в приміщеннях з розгорнутою мережевою інфраструктурою. У рамках дослідження реалізовано та проаналізовано роботу трьох протоколів прикладного рівня: MQTT, HTTP та CoAP.

3.2.1 Програмна реалізація обміну даними через протокол MQTT

В якості основного протоколу дослідження обрано MQTT (Message Queuing Telemetry Transport) завдяки його орієнтованості на роботу в мережах з низькою пропускнуою здатністю та можливими розривами з'єднання. Програмна реалізація клієнта здійснена за допомогою бібліотеки `PubSubClient`. Для забезпечення безпеки передачі даних з'єднання з брокером AWS IoT Core встановлюється через порт 8883

з використанням протоколу шифрування TLS 1.2. Автентифікація пристрою виконується за допомогою клієнтських сертифікатів X.509, що зберігаються у файлі конфігурації `credentials.h`, гарантуючи найвищий рівень захисту.

Процес розробки програмного забезпечення для MQTT пройшов чотири етапи еволюції. Базова версія (v1 Baseline) передбачала встановлення постійного TCP-з'єднання з брокером, яке утримувалося активним за допомогою пакетів PINGREQ. Передача даних здійснювалася з фіксованим інтервалом у 30 секунд. Цей режим забезпечував миттєву реакцію на команди, проте характеризувався найвищим енергоспоживанням через безперервну роботу радіомодуля. У наступній ітерації (v2 Fixed Deep Sleep) впроваджено стратегію примусового розриву з'єднання: після кожної публікації пристрій закривав TLS-сесію та переходив у режим глибокого сну. Це суттєво знизило середній струм, але додало накладні витрати на повторне встановлення захищеного з'єднання (TLS Handshake) при кожному пробудженні.

Версія v3 (Local Adaptive) реалізує адаптивний алгоритм безпосередньо на контролері. Використовуючи енергонезалежну пам'ять `RTC_DATA_ATTR`, пристрій зберігає історію вимірювань та порівнює поточні показники з попередніми. У разі стабільності параметрів інтервал сну подвоюється, а при виявленні змін — скидається до мінімуму. Фінальна версія v4 (Cloud Managed) переносить логіку керування у хмару (AWS Lambda). Пристрій публікує дані та підписується на топик команд, очікуючи асинхронне повідомлення з новим значенням інтервалу сну протягом 5-секундного вікна, після чого переходить у режим енергозбереження.

3.2.2 Програмна реалізація обміну даними через протокол HTTP

Протокол HTTP (Hypertext Transfer Protocol) використано у дослідженні як еталон для порівняння, оскільки через текстовий формат заголовків та відсутність

збереження сесії він є найбільш ресурсоємним. Реалізація базується на стандартній бібліотеці HTTPClient для ESP32. Передача телеметрії здійснюється методом POST у форматі JSON на ендпоінт сервісу AWS API Gateway.

Особливістю реалізації хмарного керування (v4) для HTTP є спрощена архітектура взаємодії порівняно з MQTT. Оскільки протокол працює за моделлю «запит-відповідь», відпадає потреба в підписці на окремі канали чи очікуванні асинхронних подій. Керуюча команда з новим інтервалом сну надходить безпосередньо у тілі HTTP-відповіді (Response Body) на запит телеметрії. Це дозволяє пристрою завершити сеанс зв'язку та перейти в сон одразу після отримання підтвердження від сервера, що потенційно зменшує час перебування в ефірі (Time-on-Air).

3.2.3 Програмна реалізація обміну даними через протокол CoAP

Протокол CoAP (Constrained Application Protocol) поєднує REST-архітектуру з ефективністю транспорту UDP, усуваючи накладні витрати на встановлення TCP-з'єднань. Оскільки сервіс AWS IoT Core не має нативної підтримки CoAP через UDP, було розроблено архітектуру з використанням проміжного шлюзу.

У цій схемі клієнт (ESP32) формує компактні повідомлення (типу NON або CON, метод PUT) з корисним навантаженням JSON і надсилає їх на проміжний шлюз. Роль шлюзу виконує віртуальний сервер AWS EC2 із розгорнутим Python-скриптом на базі бібліотеки aiocoap, який прослуховує порт 5683. Шлюз діє як проксі: приймає CoAP-пакети та пересилає дані до AWS Lambda через захищений канал HTTPS. Зворотний зв'язок у версії v4 реалізується шляхом трансляції відповіді від Lambda назад до пристрою у вигляді CoAP-повідомлення з кодом 2.04 Changed. Така реалізація дозволила дослідити ефективність UDP на «останній милі», зберігаючи при цьому можливості хмарної аналітики.

3.3 Реалізація архітектури з локальним шлюзом (Bluetooth Low Energy)

У сценаріях, де безпосереднє підключення сенсорних вузлів до мережі Інтернет через Wi-Fi є неможливим через відсутність покриття або недоцільним через жорсткі обмеження бюджету енергоспоживання, застосовується дворівнева архітектура «Сенсор — Шлюз». У рамках даного дослідження для реалізації каналу зв'язку «останньої милі» обрано технологію Bluetooth Low Energy (BLE). Вона є оптимальним вибором для передачі невеликих обсягів телеметричних даних на короткі відстані завдяки надзвичайно низькому піковому споживанню струму (менше 15 мА під час передачі) та швидкому виходу з режиму сну.

3.3.1 Реалізація архітектури «Сенсорний вузол – Шлюз»

Розроблена система базується на взаємодії двох функціонально різних типів пристроїв, ролі яких визначено відповідно до профілю GAP (Generic Access Profile).

Першим елементом є сенсорний вузол, реалізований на базі мікроконтролера ESP32 з підключеним датчиком BME680 та автономним живленням. Він функціонує в режимі радіомовника (Broadcaster), єдиним завданням якого є періодичне пробудження, вимірювання параметрів середовища та їх трансляція в ефір у вигляді рекламних пакетів (Advertising packets). Для максимальної економії енергії на цьому пристрої повністю відключено стек Wi-Fi та не реалізується стек TCP/IP, тобто вузол не має IP-адреси і не взаємодіє з Інтернетом напряду.

Другим елементом є локальний шлюз, побудований на другому мікроконтролері ESP32 зі стаціонарним живленням. Цей пристрій працює в гібридному режимі: з точки зору BLE він виступає як спостерігач (Observer), що постійно сканує радіоефір, а з точки зору зовнішньої мережі — як Wi-Fi клієнт із доступом до сервісів AWS. Шлюз виконує роль прозорого моста, перехоплюючи

пакети від сенсора, витягуючи корисне навантаження та пересилаючи його на сервер через захищене HTTPS-з'єднання. Такий розподіл функцій дозволяє делегувати найбільш енергозатратні операції (встановлення з'єднання, шифрування) стаціонарному пристрою, мінімізуючи навантаження на автономний сенсор.

3.3.2 Програмна реалізація сенсорного вузла в режимі BLE Advertising

Програмне забезпечення сенсорного вузла розроблено з використанням нативних бібліотек ESP32 BLE Arduino. Ключовим архітектурним рішенням стала відмова від встановлення повноцінного з'єднання (Connection) на користь режиму Advertising. Це обґрунтовано тим, що встановлення з'єднання вимагає процедури рукошукання та підтримки синхронізації, що витрачає додаткову енергію. Натомість режим оголошення дозволяє відправити пакет даних «в нікуди» (broadcast) за кілька мілісекунд і миттєво повернутися до сну.

Для ефективного використання обмеженого розміру рекламного пакету дані з сенсора BME680 кодуються у компактну бінарну структуру SensorData розміром 8 байт, що включає температуру, вологість, тиск та опір газу у цілочисельному форматі. Ця структура розміщується в полі Manufacturer Specific Data, а для ідентифікації пакетів системи використовується унікальний ідентифікатор виробника (0x0118).

Еволюція програмного забезпечення сенсора пройшла три етапи. Версія v1 (Базова) реалізує циклічне зчитування та трансляцію даних кожні 30 секунд без переходу в режим сну, що слугує базовою лінією для вимірювань. Версія v2 (Fixed Deep Sleep) впроваджує архітектуру «Wake-and-Sleep»: після трансляції даних протягом 3 секунд пристрій переходить у глибокий сон на фіксований час. Найбільш досконалою є версія v3 (Локальна адаптація), де пристрій використовує RTC-пам'ять для порівняння поточних показників із попередніми. У стабільних

умовах інтервал сну динамічно збільшується до 5 хвилин, а при виявленні змін — скорочується до 30 секунд. Реалізація хмарного керування (v4) для даної архітектури визнана недоцільною через значні затримки, пов'язані з очікуванням зворотної відповіді в режимі одностороннього зв'язку.

3.3.3 Програмна реалізація функціонала BLE-шлюзу

Програмне забезпечення шлюзу (`ble_gateway.ino`) забезпечує міст між радіосередовищем BLE та IP-мережею. Алгоритм роботи починається з ініціалізації Wi-Fi та синхронізації часу через NTP, що необхідно для додавання часових міток до даних сенсора, який не має власного годинника. Після цього запускається модуль BLE в режимі активного сканування.

У процесі роботи шлюз аналізує всі знайдені пристрої через `callback`-функцію. Якщо виявлено пакет із відповідним ідентифікатором виробника (`0x0118`), бінарне навантаження витягується, десеріалізується у плаваючі значення та упаковується в JSON-документ. Фінальним етапом є відправка даних на уніфіковану `Lambda`-функцію через HTTPS POST-запит. При цьому шлюз ігнорує тіло відповіді від сервера, оскільки зворотний канал до сенсора відсутній. Така реалізація забезпечує повну прозорість інтеграції BLE-пристроїв у загальну хмарну екосистему.

3.4 Реалізація архітектури глобальної мережі (LoRaWAN)

Останньою і найбільш складною з точки зору організації каналу зв'язку є архітектура глобальної мережі низького енергоспоживання (LPWAN). Вона призначена для сценаріїв, де сенсорні вузли розташовані на значній відстані від

інфраструктури (кілометри) і не мають можливості використовувати Wi-Fi або BLE. У даній роботі дослідження базується на технології LoRa (Long Range). Враховуючи відсутність стабільного покриття публічних операторів у місці проведення експериментів, було розроблено модель приватної мережі з топологією «зірка», яка функціонально та енергетично емулює роботу кінцевого пристрою в мережі LoRaWAN класу A.

3.4.1 Архітектура та компоненти системи LoRa

Розроблена система функціонує на трьох логічних рівнях, що забезпечують наскрізну передачу даних. Першим рівнем є кінцевий вузол (End Node) — автономний пристрій на базі мікроконтролера ESP32 та трансивера Semtech SX1276. Його основна функція полягає у зборі кліматичних даних, їх первинній обробці, кодуванні у компактний бінарний формат та передачі через радіоканал. Енергетичний профіль пристрою передбачає перебування в режимі глибокого сну більшу частину часу, з активацією радіопередавача лише на сотні мілісекунд. Важливою особливістю є відсутність стеку TCP/IP, що мінімізує накладні витрати на обробку даних.

Другий рівень представлений шлюзом (Gateway), реалізованим на базі аналогічного апаратного забезпечення (ESP32 + SX1276) зі стаціонарним живленням. Він виконує роль прозорого моста між радіоефіром та Інтернетом, постійно прослуховуючи задану частоту. При отриманні валідного пакету шлюз вимірює параметри якості сигналу (RSSI та SNR), додає їх до корисного навантаження і пересилає на третій рівень — хмарний сервер, реалізований через уніфіковану функцію AWS Lambda, де відбуваються дешифрування, зберігання та аналітика.

Для забезпечення балансу між дальністю зв'язку та енергоспоживанням радіоканал налаштовано на частоту 868.1 МГц (ISM діапазон) зі смугою

пропускання 125 кГц. Обрано коефіцієнт розширення спектру SF7, що забезпечує найвищу швидкість передачі даних (близько 5.5 кбіт/с) і найменший час знаходження в ефірі (Time-on-Air), що є критично важливим для енергозбереження.

3.4.2 Налаштування хмарної інтеграції

Програмне забезпечення кінцевого вузла спроектовано з акцентом на мінімізацію часу активності (Time-on-Air). Для цього застосовано оптимізацію формату даних: замість текстового JSON використано бінарне пакування у структуру SensorData розміром усього 8 байт. Значення температури, вологості, тиску та VOC перетворюються у цілочисельні типи, що дозволяє передати пакет на SF7 всього за 40–50 мс.

Алгоритм роботи базується на версії v3 (Локальна адаптація). Після пробудження та ініціалізації шини SPI пристрій зчитує дані з BME680 і порівнює їх із попередніми значеннями, збереженими в RTC-пам'яті. Якщо дані стабільні, інтервал наступного сну збільшується (наприклад, до 15 хвилин), а при виявленні динаміки — зменшується до базового рівня. Після прийняття рішення пакет відправляється в ефір у режимі асинхронної передачі, а радіомодуль і мікроконтролер негайно переходять у режим сну.

У ході дослідження було встановлено недоцільність реалізації повноцінного хмарного керування (v4) для LoRaWAN класу А. Згідно з протоколом, для отримання команди від сервера пристрій повинен відкривати вікна прийому через 1 та 2 секунди після передачі. Це змушує мікроконтролер залишатися активним щонайменше 2 секунди, що у 40 разів перевищує час самої передачі (~50 мс). Оскільки пріоритетом роботи є мінімізація енергоспоживання, було прийнято рішення відмовитися від зворотного каналу зв'язку на користь автономного локального алгоритму адаптації.

3.4.3 Програмна реалізація кінцевого вузла

Програмне забезпечення кінцевого вузла спроектовано з акцентом на мінімізацію часу активності (Time-on-Air). Для цього застосовано оптимізацію формату даних: замість текстового JSON використано бінарне пакування у структуру SensorData розміром усього 8 байт. Значення температури, вологості, тиску та VOC перетворюються у цілочисельні типи, що дозволяє передати пакет на SF7 всього за 40–50 мс.

Алгоритм роботи базується на версії v3 (Локальна адаптація). Після пробудження та ініціалізації шини SPI пристрій зчитує дані з BME680 і порівнює їх із попередніми значеннями, збереженими в RTC-пам'яті. Якщо дані стабільні, інтервал наступного сну збільшується (наприклад, до 15 хвилин), а при виявленні динаміки — зменшується до базового рівня. Після прийняття рішення пакет відправляється в ефір у режимі асинхронної передачі, а радіомодуль і мікроконтролер негайно переходять у режим сну.

У ході дослідження було встановлено недоцільність реалізації повноцінного хмарного керування (v4) для LoRaWAN класу A. Згідно з протоколом, для отримання команди від сервера пристрій повинен відкривати вікна прийому через 1 та 2 секунди після передачі. Це змушує мікроконтролер залишатися активним щонайменше 2 секунди, що у 40 разів перевищує час самої передачі (~50 мс). Оскільки пріоритетом роботи є мінімізація енергоспоживання, було прийнято рішення відмовитися від зворотного каналу зв'язку на користь автономного локального алгоритму адаптації.

3.5 Розробка хмарної частини системи на AWS

Хмарна інфраструктура виступає інтелектуальним ядром розробленої системи, забезпечуючи централізований збір, надійне зберігання, аналітичну обробку даних, а також реалізацію зворотного зв'язку для керування режимами роботи кінцевих пристроїв (у версіях v4). Архітектура побудована на базі безсерверних (serverless) компонентів Amazon Web Services (AWS), що забезпечує автоматичне масштабування, високу відмовостійкість та оплату лише за фактично використані ресурси обчислення та зберігання.

3.5.1 Уніфікована функція AWS Lambda

Ключовим елементом бізнес-логіки на стороні хмари є розроблена функція `lambda_function.py` мовою Python. На відміну від традиційного підходу, що передбачає створення окремих обробників для кожного протоколу, у даній роботі реалізовано архітектурний патерн «Single Entry Point» (Єдина точка входу). Це дозволяє одній функції уніфіковано обробляти потоки даних із гетерогенних джерел: HTTP-запити від Wi-Fi пристроїв, перенаправлення від MQTT-брокера, а також пакети від шлюзів CoAP, BLE та LoRaWAN. Функціональність Lambda-функції охоплює три логічні етапи. На першому етапі відбувається прийом та нормалізація даних: універсальний парсер обробляє вхідну подію, структура якої варіюється залежно від джерела, витягує корисне навантаження у форматі JSON та приводить його до єдиного внутрішнього стандарту. Зокрема, для забезпечення точності при подальшому зберіганні, значення температури та вологості конвертуються у тип `Decimal`. На другому етапі реалізується стратегія подвійного запису даних (Persistence Layer): повний набір історичних даних із часовими мітками зберігається в базі Amazon DynamoDB, тоді як останній актуальний стан

пристрою (snapshot) оновлюється в оперативному кеші Redis. Третій етап є критичним для енергоефективності, оскільки саме тут реалізується алгоритм хмарної адаптації (v4). Якщо вхідний запит містить маркер керування, функція виконує роль центру прийняття рішень. Вона звертається до Redis для миттєвого отримання попередніх показників та порівнює їх із поточними. На основі розрахунку інтегрального показника зміни середовища (Δ_{avg}) обирається оптимальний інтервал сну (T_{sleep}) за дискретною шкалою: при суттєвих змінах ($\Delta_{avg} > 2.0$) інтервал встановлюється на рівні 30 с; при помірних ($1.0 < \Delta_{avg} \leq 2.0$) він збільшується до 120 с; а в стабільних умовах ($\Delta_{avg} \leq 1.0$) досягає максимуму в 600 с. Додатково перевіряються умови форс-мажору (наприклад, різкі стрибки температури), що ініціює встановлення прапора `force_transmit`. Розраховані параметри упаковуються в JSON-відповідь для передачі пристрою.

3.5.2 Інтеграція потоків даних та маршрутизація

Організація взаємодії хмарної частини з кінцевими пристроями реалізована через декілька каналів надходження інформації, які зводяться до єдиної точки обробки. Для протоколу MQTT, який є нативним для екосистеми AWS, налаштовано механізм безпосередньої взаємодії через сервіс AWS IoT Core. Обробка вхідних повідомлень здійснюється за допомогою правил маршрутизації (Rules Engine). Створене SQL-подібне правило діє як фільтр, що підписується на топик телеметрії та при надходженні даних автоматично ініціює виконання уніфікованої Lambda-функції. Для реалізації зворотного зв'язку (у версії v4) функція використовує AWS SDK (boto3), публікуючи керуючі команди в окремий топик MQTT.

Поряд із цим система підтримує пряме надходження даних через протокол HTTP, використовуючи сервіс Amazon API Gateway як точку входу для REST-запитів. Це дозволяє пристроям з підтримкою Wi-Fi передавати телеметрію

методом POST без встановлення постійного з'єднання. Для інших досліджуваних технологій — CoAP, BLE та LoRaWAN — застосовано архітектуру з використанням проміжних шлюзів. У цих сценаріях шлюзи (програмні на EC2 або апаратні на ESP32) виконують роль трансляторів протоколів: вони приймають специфічні пакети даних, здійснюють їх попередню обробку та конвертують у стандартні запити HTTPS (або MQTT), які потім спрямовуються до хмарного ядра. Такий підхід дозволив уніфікувати вхідний інтерфейс для всіх типів пристроїв, незалежно від використовуваного ними фізичного каналу зв'язку.

3.5.3 Структура баз даних

Для забезпечення балансу між вартістю зберігання, швидкістю доступу та аналітичними можливостями спроектовано гібридну модель даних, що використовує два типи СУБД. В якості основного сховища історичних даних обрано Amazon DynamoDB — повністю керовану NoSQL базу даних. Таблиця ClimateData використовує часову мітку (timestamp) як ключ партиціонування, що дозволяє ефективно виконувати вибірку за часові інтервали для побудови графіків та ретроспективного аналізу.

Для задач, що вимагають миттєвого доступу до даних, використовується Amazon ElastiCache for Redis. Це високопродуктивне сховище в оперативній пам'яті зберігає дані у вигляді JSON-об'єкта за фіксованим ключем, який перезаписується при кожному оновленні. Така схема гарантує, що в кеші завжди знаходиться лише останній актуальний стан системи. Використання Redis є критично важливим для роботи алгоритму адаптації, який потребує швидкого порівняння поточних значень із попередніми без виконання "дорогих" запитів до основної бази даних, а також для живлення інформаційних панелей реального часу з мінімальною затримкою.

3.5.4 Налаштування інформаційних панелей у Grafana

Візуалізація зібраних даних та моніторинг стану системи реалізовані на платформі Grafana, яка підключена до хмарної інфраструктури через API Gateway. Налаштовано два типи інформаційних панелей, що обслуговують різні потреби користувачів. Панель моніторингу реального часу (Real-time) призначена для оперативного контролю та відображає поточний статус за допомогою віджетів типу "Gauge" та "Stat". Джерелом даних для неї слугує Redis, що забезпечує оновлення показників із затримкою менше 50 мс.

Натомість аналітична панель (Historical) орієнтована на дослідницькі задачі та аналіз динаміки змін. Вона містить часові ряди (Time series) з можливістю масштабування. Запити для цієї панелі обробляються тією ж Lambda-функцією, але в режимі вибірки з DynamoDB за вказаний період. Це дозволяє візуалізувати роботу адаптивних алгоритмів, зокрема, кореляцію між стабільністю кліматичних параметрів та частотою точок передачі даних на графіку.

3.6 Забезпечення інформаційної безпеки в розробленій системі

В умовах використання відкритих каналів зв'язку та розгортання компонентів системи в глобальній мережі Інтернет, питання захисту цілісності, конфіденційності та доступності даних набувають критичного значення. Для розробленої системи моніторингу реалізовано комплексну, ешелоновану модель безпеки (Defense-in-Depth), що охоплює рівень кінцевих пристроїв, транспортний рівень та рівень хмарної інфраструктури. Такий підхід гарантує, що компрометація одного компонента не призведе до злому всієї системи.

3.6.1 Захист каналів зв'язку та аутентифікація пристроїв

Основним вектором атак для IoT-систем є перехоплення або підміна даних під час їх передачі. Для протидії цим загрозам у системі впроваджено суворі криптографічні стандарти.

Захист протоколів TCP/IP (MQTT, HTTP). Для всіх з'єднань, що використовують стек TCP/IP (версії Wi-Fi), обов'язковим є використання протоколу транспортного рівня TLS (Transport Layer Security) версії 1.2. Це забезпечує наскрізне шифрування трафіку між мікроконтролером ESP32 та сервісами AWS, унеможливлюючи атаки типу «людина посередині» (Man-in-the-Middle, MITM). Для протоколу MQTT, який є основним каналом керування, реалізовано механізм взаємної аутентифікації (mutual TLS, mTLS). Цей процес передбачає двосторонню перевірку:

1. Аутентифікація сервера: Пристрій перевіряє справжність сервера AWS IoT Core за допомогою збереженого у прошивці кореневого сертифіката (AWS Root CA). Це захищає від підключення до підроблених серверів-зловмисників.
2. Аутентифікація клієнта: Сервер перевіряє справжність пристрою за допомогою унікального клієнтського сертифіката X.509 та закритого ключа (Private Key), які генеруються на етапі ініціалізації (provisioning).

Зберігання криптографічних ключів на ESP32 реалізовано з використанням захищеної файлової системи (SPIFFS) або енергонезалежної пам'яті (NVS), що ускладнює їх вилучення навіть при фізичному доступі до пристрою.

Захист мереж LPWAN (LoRaWAN). Безпека в сегменті LoRaWAN забезпечується на рівні стандарту протоколу, який використовує алгоритм шифрування AES-128. Реалізовано схему захисту з двома типами сесійних ключів:

1. Network Session Key (NwkSKey): Забезпечує перевірку цілісності пакету (MIC) та аутентифікацію вузла на рівні мережевого сервера.

2. Application Session Key (AppSKey): Використовується для наскрізного шифрування корисного навантаження (payload). Мережевий оператор (наприклад, TTN) бачить лише зашифровані дані, а їх розшифрування відбувається лише на кінцевому сервері застосунку (в даному випадку — у функції AWS Lambda). Для підключення пристроїв обрано метод активації OTAA (Over-The-Air Activation), який є більш безпечним порівняно з ABP (Activation By Personalization), оскільки сесійні ключі генеруються динамічно при кожному перезавантаженні пристрою, а не «зашиті» жорстко.

3.6.2 Політики безпеки та розмежування доступу в хмарному середовищі

На рівні хмарної платформи AWS безпека базується на моделі розділеної відповідальності та реалізується через сервіс керування ідентифікацією та доступом (IAM — Identity and Access Management). Застосовано принцип найменших привілеїв (Principle of Least Privilege), згідно з яким кожному компоненту надаються лише ті права, які критично необхідні для виконання його функцій.

IoT-політики (IoT Policies). Для кожного фізичного пристрою (або групи однотипних пристроїв) створено та прив'язано до сертифіката X.509 спеціальну політику доступу. Вона строго регламентує дозволені дії та ресурси:

- Action: `iot:Publish` дозволено виключно у топик телеметрії даного пристрою (`climate/monitoring/device_01/data`). Спроба публікації в чужий топик або системний топик буде автоматично заблокована брокером.
- Action: `iot:Subscribe` дозволено лише на особистий топик команд (`climate/monitoring/device_01/commands`). Така гранулярність гарантує, що навіть у випадку компрометації ключів одного сенсора зловмисник

не зможе вплинути на роботу інших вузлів мережі або перехопити їхні дані.

Ролі виконання Lambda (Execution Roles). Уніфікована Lambda-функція працює під спеціально створеною роллю IAM, яка має дозволи лише на:

- Запис (PutItem) у конкретну таблицю DynamoDB (ClimateData).
- Доступ до кластера ElastiCache (Redis).
- Публікацію повідомлень у топіки команд IoT Core. Будь-які інші дії (наприклад, створення нових користувачів, видалення баз даних, запуск обчислювальних інстансів EC2) заборонені на рівні політики ролі. Це мінімізує радіус ураження у випадку програмних помилок або ін'єкцій коду в Lambda-функцію.

3.6.3 Захист API та шлюзів

Для сегмента системи, що взаємодіє через протокол HTTP, точкою входу слугує сервіс Amazon API Gateway. Для його захисту реалізовано:

- Throttling (Обмеження частоти запитів): Встановлено ліміти на кількість запитів за секунду (RPS) для запобігання DDoS-атакам та виснаженню бюджету.
- Валідація вхідних даних: На рівні API Gateway налаштовано моделі даних (Models), які перевіряють структуру JSON-запиту ще до його передачі на обробку в Lambda. Це відфільтровує некоректні або шкідливі пакети даних.

Для проміжних шлюзів (CoAP Gateway, BLE Gateway) реалізовано механізм «білих списків» (Allowlisting). Шлюз обробляє та пересилає дані лише від тих сенсорів, чий ідентифікатори (MAC-адреси або DevEUI) попередньо зареєстровані в системі конфігурації. Пакети від невідомих джерел ігноруються без обробки, що захищає систему від засмічення стороннім трафіком.

Таким чином, розроблена архітектура безпеки забезпечує конфіденційність даних користувача, цілісність вимірювань та стійкість системи до найбільш поширених векторів кібератак, що дозволяє рекомендувати її для використання у відповідальних застосуваннях.

Висновки до розділу

У третьому розділі магістерської дисертації виконано повний цикл інженерного проектування та практичної реалізації програмно-апаратного комплексу для дослідження енергоефективності IoT-систем. Основним результатом розробки стало створення універсальної апаратної платформи на базі мікроконтролера ESP32 та сенсора BME680. Модульна конструкція стенду дозволяє проводити коректні порівняльні експерименти для різних технологій зв'язку в ідентичних фізичних умовах, мінімізуючи вплив апаратних похибок.

В рамках роботи реалізовано мультипротокольную архітектуру, що підтримує п'ять комунікаційних стандартів: MQTT, HTTP, CoAP, BLE та LoRaWAN. Це дозволило охопити весь спектр сучасних IoT-рішень — від персональних мереж до глобального покриття. Для вирішення проблеми сумісності протоколів, що не мають нативної підтримки у хмарних платформах або прямого доступу до Інтернету (CoAP, BLE, LoRa), було розроблено та впроваджено архітектуру з проміжними шлюзами на базі віртуальних серверів EC2 та додаткових контролерів ESP32.

Вагомим досягненням є програмна реалізація чотирирівневої моделі оптимізації через еволюційний ряд алгоритмів (v1–v4). Це дозволило ізолювати вплив кожного методу енергозбереження та створити гібридну систему керування, яка ефективно поєднує автономну локальну адаптацію для низькошвидкісних протоколів (BLE/LoRa) із централізованою хмарною адаптацією для Wi-Fi з'єднань. Інтеграція всіх компонентів в уніфіковану хмарну екосистему AWS

забезпечила єдиний простір для збору та аналізу даних, що робить розроблену систему повністю функціональною та готовою до проведення натурних випробувань у наступному етапі дослідження.

4. ЕКСПЕРИМЕНТАЛЬНІ ДОСЛІДЖЕННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ

Четвертий розділ присвячено практичній перевірці ефективності запропонованих архітектурних та алгоритмічних рішень. У ньому наведено методику проведення експерименту, результати вимірювань енергоспоживання для різних протоколів (MQTT, HTTP, CoAP, BLE, LoRaWAN) у різних режимах роботи (v1–v4), а також виконано порівняльний аналіз отриманих даних. Ключовим завданням розділу є кількісне підтвердження гіпотези про те, що адаптивне керування інтервалами активності дозволяє суттєво подовжити час автономної роботи IoT-пристроїв.

4.1 Методика проведення експериментальних досліджень

Для забезпечення об'єктивності, точності та відтворюваності отриманих результатів розроблено спеціалізовану методику вимірювань, адаптовану до умов лабораторного експерименту. Дослідження спрямоване на фіксацію реальних профілів енергоспоживання мікроконтролера ESP32 у динамічних режимах роботи, а також на оцінку впливу перехідних процесів на загальний енергетичний баланс системи.

4.1.1 Схема та апаратна конфігурація вимірювального стенду

Експериментальна установка була спроектована таким чином, щоб дозволити одночасно забезпечувати живлення пристрою, контролювати логіку

виконання алгоритмів у реальному часі та з високою точністю вимірювати споживаний струм.

Об'єктом дослідження виступає універсальний сенсорний вузол на базі плати ESP32 DevKit v1. До неї підключено інтегрований сенсор BME680 через інтерфейс I2C (піни GPIO 21 та GPIO 22), а в сценаріях тестування мережі дальнього радіусу дії — трансивер Semtech SX1276 через інтерфейс SPI (піни GPIO 18, 19, 23, 5).

Інтерфейс живлення та налагодження організовано через USB-підключення до персонального комп'ютера, що виконує подвійну функцію. По-перше, це забезпечує стабілізоване живлення з напругою 5 В, яка перетворюється лінійним стабілізатором (LDO) на платі у робочу напругу 3.3 В. По-друге, це дозволяє здійснювати моніторинг логіки через віртуальний COM-порт зі швидкістю 115200 бод. Отримання налагоджувальної інформації дає змогу точно ідентифікувати початок та кінець кожної фази роботи (прокидання, ініціалізація, передача даних, перехід у сон) та синхронізувати їх із показаннями вимірювального приладу.

Для фіксації споживання струму застосовано метод розриву електричного кола (Series Ammeter Method). Використано модифікований USB-кабель із розірваним плюсовим провідником, у який послідовно увімкнено щупи цифрового мультиметра класу UNI-T UT61E. Така схема гарантує, що весь струм, споживаний платою (включно з втратами на перетворювачі та індикаторах), проходить через вимірювальний прилад. Мультиметр забезпечує достатню точність для фіксації як пікових струмів радіопередавача (сотні міліампер), так і низьких струмів у режимах сну.

4.1.2 Розрахункова модель автономності

Оскільки фізичне тестування повного циклу розряду батареї вимагає значного часу і залежить від характеристик конкретного хімічного джерела, оцінка часу

автономної роботи проводилася розрахунковим методом на основі експериментальних даних про середній струм споживання. За базову модель джерела живлення прийнято типовий літій-полімерний (Li-Po) акумулятор із номінальною ємністю (C_{bat}) 1000 мА·год та коефіцієнтом ефективності (K_{eff}) 0.85. Цей коефіцієнт враховує неможливість повного розряду батареї через напругу відсічки, саморозряд та ККД стабілізатора напруги. Час автономної роботи (T_{work}) розраховується за формулою:

$$T_{work} = \frac{C_{bat} * K_{eff}}{I_{avg}} \quad (5)$$

де I_{avg} — середній інтегральний струм споживання за повний робочий цикл, отриманий в результаті вимірювань.

4.1.3 Аналіз доцільності використання режиму Deep Sleep

Окрему увагу в дослідженні приділено оцінці ефективності та меж застосування режиму глибокого сну (Deep Sleep). Аналіз логів роботи пристрою засвідчив, що вихід із цього режиму для мікроконтролера ESP32 технічно є не просто «прокиданням», а повним перезавантаженням системи.

Процес пробудження супроводжується низкою енергоємних етапів: завантаженням початкового коду (Bootloader execution) з перевіркою цілісності пам'яті, ініціалізацією ядра процесора та системних частот, калібруванням радіочастотного тракту (RF Calibration) зі сплесками струму до 150–200 мА, а також повторною ініціалізацією периферії та сенсорів. Ця послідовність дій створює так звані «накладні витрати на пробудження» (wake-up overhead), коли пристрій витрачає енергію та час (від 200 мс до 2 с) ще до початку виконання корисної роботи [19].

4.2 Аналіз енергоефективності протоколів на базі Wi-Fi

Перша серія експериментів була присвячена дослідженню найбільш поширених протоколів, що використовують стандартний IP-стек та Wi-Fi з'єднання для передачі даних у хмару: HTTP (REST API), MQTT та CoAP (через UDP). Метою дослідження було визначення впливу вибору протоколу на енергоспоживання в умовах виконання однакової корисної роботи — передачі ідентичного JSON-об'єкта з даними сенсора BME680. Порівняльний аналіз проводився для двох полярних сценаріїв: базової версії (v1) без оптимізації з постійною активністю радіомодуля та фінальної версії (v4), що використовує режим Deep Sleep та хмарне керування інтервалом сну.

4.2.1 Дослідження енергоспоживання при використанні протоколу HTTP

Попри те, що MQTT розроблений як легковаговий протокол для IoT, його використання в контексті режимів глибокого сну виявило специфічні особливості. У базовому режимі (v1) споживання склало 63,43 мА, що перевищило показники HTTP. Це пояснюється необхідністю постійного фонового обміну службовими пакетами PINGREQ / PINGRESP для підтримання активної сесії з брокером AWS IoT Core.

Впровадження алгоритму v4 дозволило знизити середній струм до 15,34 мА, досягнувши економії у 75,8%. Однак, незважаючи на загальну ефективність протоколу, час активної фази зріс до 10,43 с, що вдвічі більше за аналогічний показник HTTP. Це зумовлено асинхронною природою взаємодії: після публікації даних пристрій змушений залишатися активним протягом певного «вікна прослуховування», очікуючи на асинхронну публікацію команди з боку сервера у відповідний топік.

4.2.2 Дослідження енергоспоживання при використанні протоколу MQTT

Попри те, що MQTT розроблений як легковаговий протокол для IoT, його використання в контексті режимів глибокого сну виявило специфічні особливості. У базовому режимі (v1) споживання склало 63,43 мА, що перевищило показники HTTP. Це пояснюється необхідністю постійного фонових обміну службовими пакетами PINGREQ / PINGRESP для підтримання активної сесії з брокером AWS IoT Core.

Впровадження алгоритму v4 дозволило знизити середній струм до 15,34 мА, досягнувши економії у 75,8%. Однак, незважаючи на загальну ефективність протоколу, час активної фази зріс до 10,43 с, що вдвічі більше за аналогічний показник HTTP. Це зумовлено асинхронною природою взаємодії: після публікації даних пристрій змушений залишатися активним протягом певного «вікна прослуховування», очікуючи на асинхронну публікацію команди з боку сервера у відповідний топік.

4.2.3 Дослідження енергоспоживання при використанні протоколу CoAP

Протокол CoAP, що працює поверх транспортного протоколу UDP, підтвердив статус найбільш ефективного рішення для задач моніторингу. У базовому режимі (v1) його споживання склало 60,46 мА, що є співрозмірним з іншими протоколами в активному стані. Проте в оптимізованому режимі (v4) CoAP продемонстрував найкращий результат серед усіх Wi-Fi рішень — 12,47 мА, забезпечивши економію енергії на рівні 79,3%.

Ключовою перевагою CoAP стало скорочення часу активної фази до 4,49 с. Це стало можливим завдяки відсутності фази встановлення з'єднання (handshake), характерної для TCP/TLS. Використання UDP дозволяє реалізувати стратегію «fire

and forget» (відправив і забув) одразу після підключення до мережі, а отримання відповіді від проміжного шлюзу відбувається значно швидше, ніж при прямій взаємодії з хмарним ядром.

4.3 Порівняльний аналіз енергоспоживання фіксованого та адаптивних режимів

Метою даного етапу досліджень було визначення кількісної ефективності впроваджених програмних методів оптимізації. Порівняльний аналіз здійснювався шляхом співставлення інтегральних показників енергоспоживання базових неоптимізованих версій програмного забезпечення та вдосконалених алгоритмів.

4.3.1 Ефективність впровадження режиму Deep Sleep

Першим і визначальним кроком оптимізації стала відмова від активного очікування в циклі програми на користь переведення мікроконтролера в режим глибокого сну (Deep Sleep) між сеансами зв'язку. Порівняння базової версії (v1) та версії з фіксованим інтервалом передачі (v2) засвідчило кардинальне зниження енергоспоживання для всіх досліджуваних протоколів. Зокрема, для протоколів сімейства Wi-Fi середній струм зменшився у 2,4–3 рази: для HTTP — з 59,61 мА до 24,68 мА, для MQTT — з 63,43 мА до 22,08 мА, а для CoAP — з 60,46 мА до 19,89 мА. Найбільш показовий результат зафіксовано для технології LoRa, де споживання впало з 96,78 мА до 19,20 мА, що відповідає п'ятикратному зменшенню.

Експеримент підтвердив, що впровадження режиму Deep Sleep є фундаментальною умовою для створення автономних пристроїв. Навіть з урахуванням енергетичних витрат на перезавантаження мікроконтролера та

повторне встановлення з'єднання, економія енергії під час фази сну (де струм складає ~ 10 мкА проти $\sim 70\text{--}80$ мА в активному режимі) повністю перекриває накладні витрати. Особливо висока ефективність для LoRa пояснюється специфікою протоколу, який, на відміну від Wi-Fi, не вимагає тривалих процедур рукошестискання (handshake) при пробудженні.

4.3.2 Ефективність локальної адаптації

Наступним рівнем оптимізації стало впровадження алгоритму локальної адаптації (v3), який дозволяє пристрою самостійно аналізувати динаміку зміни кліматичних показників і збільшувати інтервал сну в періоди стабільності. Порівняння версії з фіксованим інтервалом (v2) та адаптивної версії (v3) в умовах імітації реальної експлуатації показало подальше зниження споживання. Для протоколів MQTT та HTTP додаткова економія склала 27,0% (зниження до 16,11 мА) та 30,4% (зниження до 17,17 мА) відповідно. Ще вищу ефективність адаптація продемонструвала для енергоефективних протоколів: споживання LoRa знизилося на 34,2% (до 12,63 мА), а BLE — на 35,9% (до 14,38 мА).

Аналіз отриманих даних свідчить, що впровадження адаптивного алгоритму дозволяє отримати суттєвий додатковий виграш в енергоефективності, який становить близько третини від споживання попередньої версії. Це досягається за рахунок «інтелектуального» пропуску сеансів зв'язку, коли передача даних не несе нової інформації. Найбільший ефект спостерігається саме для протоколів LoRa та BLE, де сам процес передачі є відносно енергоємним або час активності є критичним параметром. Таким чином, зниження частоти виходу в ефір прямо пропорційно зменшує інтегральне споживання системи.

4.4 Оцінка ефективності хмарного керування (v4) порівняно з локальними алгоритмами

Четверта серія експериментів мала на меті дослідити ефективність архітектурного підходу, за якого логіка прийняття рішень щодо режиму роботи пристрою виноситься у хмарне середовище. У цій моделі (версія v4) мікроконтролер ESP32 виконує роль виконавчого механізму: він передає «сирі» дані вимірювань на сервер і очікує у відповідь команду, що містить розрахований інтервал наступного сну (`sleep_interval`). Порівняння проводилося з версією v3, де аналогічний алгоритм адаптації виконувався безпосередньо на пристрої.

4.4.1 Порівняння ефективності алгоритмів керування для протоколу HTTP

Порівняння версій для протоколу HTTP показало незначну перевагу хмарного керування. У режимі локальної адаптації (v3), де пристрій витрачав ресурси на зчитування історичних даних з RTC-пам'яті та виконання математичних операцій порівняння, середній струм споживання становив 17,17 мА. Перехід на хмарне керування (v4) дозволив знизити цей показник до 16,59 мА, забезпечивши покращення енергоефективності на рівні 3,4%.

Такий результат пояснюється синхронною природою протоколу HTTP («запит-відповідь»). Отримання керуючої команди відбувається безпосередньо в тілі HTTP-відповіді на POST-запит, що не вимагає від пристрою додаткових дій, відкриття нових з'єднань чи тривалого очікування. Фактично, виграш в енергоспоживанні було отримано за рахунок розвантаження мікроконтролера від локальних обчислювальних операцій.

4.4.2 Порівняння ефективності алгоритмів керування для протоколу MQTT

Для протоколу MQTT результати виявилися більш цікавими та продемонстрували покращення енергоефективності на рівні 4,8%. Середній струм знизився з 16,11 мА у версії v3 до 15,34 мА у версії v4. При цьому було зафіксовано певний парадокс: у версії v4 час активної фази (T_{act}) зріс майже вдвічі (до 10,43 с) через необхідність очікування асинхронного повідомлення з топіка команд (climate/monitoring/commands).

Зниження середнього струму на фоні збільшення часу активності пояснюється різницею у профілях споживання. У періоди пасивного очікування (коли працює лише радіоприймач Wi-Fi) миттєве споживання струму є меншим, ніж під час інтенсивних обчислень або активної передачі даних. Крім того, потужні алгоритми на стороні AWS Lambda, маючи доступ до повної історії даних, здатні швидше приймати рішення про переведення пристрою у режим «довгого сну» (600 с), тоді як консервативний локальний алгоритм v3 збільшує інтервали поступово.

4.4.3 Порівняння ефективності алгоритмів керування для протоколу CoAP

Для протоколу CoAP різниця між показниками виявилася мінімальною. Середній струм у режимі локальної адаптації становив 12,61 мА, а при хмарному керуванні — 12,47 мА. Зафіксована різниця близько 1% знаходиться в межах похибки вимірювань. Це дозволяє зробити висновок, що для протоколу CoAP, який базується на швидкому транспорті UDP, накладні витрати на локальні обчислення (у v3) та на очікування відповіді від проміжного шлюзу (у v4) є співрозмірними. Обидва методи демонструють однаково високу ефективність, що підтверджує гнучкість CoAP для різних архітектурних рішень.

4.5 Аналіз енергоефективності альтернативних архітектур

Друга серія експериментів була спрямована на дослідження альтернативних комунікаційних технологій — Bluetooth Low Energy (BLE) та LoRa, які позиціонуються як більш енергоефективні рішення для IoT порівняно з Wi-Fi. Особливістю цих тестів було те, що для BLE та LoRa реалізація хмарного керування (v4) була визнана недоцільною на етапі проєктування через специфіку протоколів, тому порівняльний аналіз проводився між базовою версією (v1) та версією з локальною адаптацією (v3).

4.5.1 Аналіз енергоефективності технології Bluetooth Low Energy

Вимірювання ефективності BLE проводилися для архітектури «Broadcaster», де сенсорний вузол не встановлює з'єднання, а лише транслює рекламні пакети. У базовому режимі (v1) середній струм споживання склав 56,88 мА. Попри те, що пікове споживання радіотракту BLE є нижчим за Wi-Fi, постійна трансляція пакетів (Advertising) вимагає значних витрат енергії, які виявилися співмірними зі споживанням Wi-Fi модуля в режимі простою.

Впровадження алгоритму «Wake-and-Sleep» у версії v3 дозволило знизити середній струм до 14,38 мА, забезпечивши економію на рівні 74,7%. Аналіз часових характеристик показав, що тривалість активної фази склала 3,56 с. Лівову частку цього часу (3 секунди) займає примусова трансляція рекламних пакетів, необхідна для гарантованого прийому даних шлюзом, який сканує ефір із певним інтервалом. Це вказує на те, що подальша оптимізація обмежена специфікою протоколу сканування, оскільки скорочення часу трансляції може призвести до втрати пакетів.

4.5.2 Аналіз енергоефективності технології LoRa

Технологія LoRa продемонструвала найбільш контрастні результати в ході дослідження. У базовому режимі (v1) середній струм досяг 96,78 мА, що стало найвищим показником серед усіх протестованих протоколів. Це пояснюється фізикою процесу передачі LoRa, яка вимагає значної потужності для модуляції сигналу, здатного долати великі відстані. У режимі постійної активності це призводить до швидкого вичерпання ресурсу батареї.

Однак оптимізована версія (v3) кардинально змінила картину, показавши один із найкращих результатів у дослідженні — 12,63 мА. Економія енергії склала рекордні 86,9%. Ключовою перевагою адаптивного режиму LoRa стало скорочення часу активної фази до 2,79 с, що є найшвидшим показником серед усіх протоколів. Завдяки використанню стратегії «Fire-and-Forget» (відправив і забув) та відсутності процедур рукошукання чи очікування відповіді (на відміну від Wi-Fi), пристрій виконує корисну роботу миттєво і повертається в сон. Таким чином, високий піковий струм передавача повністю компенсується екстремально малим часом його дії.

4.6 Узагальнений порівняльний аналіз та оцінка автономності

Фінальним етапом експериментальних досліджень стало проведення узагальненого аналізу, метою якого було зведення результатів найбільш ефективних версій для кожного протоколу (v4 для Wi-Fi, v3 для BLE/LoRa) та розрахунків теоретичного часу автономної роботи. Для забезпечення порівнянності результатів розрахунки базувалися на уніфікованій моделі джерела живлення з номінальною ємністю (C_{bat}) 1000 мА·год та коефіцієнтом ефективності (K_{eff}) 0.85.

Узагальнені показники енергоефективності та прогнозована автономність наведені в таблиці 4.1.

Таблиця 4.1 Підсумкове порівняння ефективності протоколів

Протокол	Версія	Середній струм I_{avg} (мА)	Час активності T_{act} (с)	Автономність (годин)	Автономність (днів)*
HTTP	v4 (Cloud)	16.59	5.05	51.2	2.1
MQTT	v4 (Cloud)	15.34	10.43	55.4	2.3
BLE	v3 (Local)	14.38	3.56	59.1	2.5
LoRa	v3 (Local)	12.63	2.79	67.3	2.8
CoAP	v4 (Cloud)	12.47	4.49	68.2	2.8

Лідерами за показниками енергозбереження стали протоколи CoAP та LoRa, які забезпечують час автономної роботи на рівні 2,8 доби у режимі інтенсивного тестування. Протокол BLE зайняв проміжну позицію (2,5 доби), тоді як MQTT та HTTP, попри оптимізацію, продемонстрували найнижчі показники (2,1–2,3 доби) через вищі накладні витрати на підтримку з'єднання.

Важливо наголосити, що наведені абсолютні значення автономності (2–3 дні) отримані для сценарію «стрес-тестування», де передача даних відбувалася майже щохвилини (інтервали сну 30–60 с). Цей режим використовувався для прискорення набору статистичних даних. При екстраполяції результатів на реальний сценарій моніторингу клімату, де типовий інтервал вимірювань складає 10–15 хвилин, час автономної роботи зростає лінійно і обчислюватиметься місяцями, що підтверджує практичну цінність розробленої системи.

Висновки до розділу 4

У четвертому розділі магістерської дисертації проведено комплексне експериментальне дослідження ефективності запропонованих методів оптимізації енергоспоживання ІТ-системи моніторингу кліматичних показників. Натурні випробування, виконані на розробленому універсальному стенді в умовах, наближених до реальної експлуатації, дозволили отримати емпіричні дані, що повністю підтверджують висунуту робочу гіпотезу: поєднання апаратних режимів глибокого сну з адаптивними алгоритмами керування забезпечує багаторазове зниження енергоспоживання.

Детальний аналіз отриманих результатів дозволив кількісно оцінити внесок кожного етапу оптимізації. Перший етап — перехід від базового алгоритму з постійною активністю (v1) до алгоритму з фіксованим інтервалом сну (v2) — продемонстрував зниження середнього струму споживання у діапазоні від 2,5 до 5 разів залежно від обраного протоколу. Другий етап, що полягав у впровадженні розроблених адаптивних алгоритмів (версії v3 та v4), забезпечив додаткову економію енергії на рівні 27–36% відносно версії v2. Таким чином, сумарний ефект від запропонованої комплексної оптимізації склав 72–87%, що є вагомим науково-практичним результатом, який суттєво підвищує автономність пристроїв.

Вперше в рамках одного дослідження було проведено пряме порівняння енергоефективності п'яти різних комунікаційних протоколів (MQTT, HTTP, CoAP, BLE, LoRa) на ідентичній апаратній платформі. Беззаперечним лідером серед рішень на базі IP-мереж визначено протокол CoAP (у зв'язці з проміжним шлюзом), який показав найкращий результат середнього струму — 12,47 мА. Це експериментально доводить перевагу використання легкого транспорту UDP для задач періодичного моніторингу. Високу ефективність також продемонструвала технологія LoRa в режимі P2P із результатом 12,63 мА, що практично дорівнює показникам CoAP, але при цьому забезпечує значно більший радіус покриття. Цей факт спростовує поширене упередження про високу енергоємність LoRa: завдяки

стратегії «швидкої передачі» вона виявилася однією з найекономічніших технологій. Щодо протоколів MQTT та HTTP, то хоча вони й показали дещо вище споживання (15–16 мА), впровадження хмарного керування (v4) дозволило наблизити їхні показники до лідерів, зробивши їх цілком конкурентоспроможними для автономних систем.

Важливим результатом роботи стало підтвердження ефективності гібридного підходу до керування режимами роботи. Дослідження виявило, що хмарна адаптація (v4) є найбільш доцільною для «важких» протоколів (MQTT, HTTP), де час встановлення з'єднання є значним. У цьому випадку перенесення логіки на сервер AWS Lambda дозволяє компенсувати енергетичні витрати за рахунок більш інтелектуального планування інтервалів сну. Водночас для протоколів із низькою пропускну здатністю (BLE, LoRa), де критичними є затримки на очікування відповіді від хмари, оптимальною визнано модель локальної адаптації (v3).

Практична значимість отриманих результатів підтверджується розрахунками часу автономної роботи. Встановлено, що розроблена система здатна функціонувати від стандартного літій-полімерного акумулятора ємністю 1000 мА·год до 3 діб у режимі інтенсивного стрес-тестування (передача щохвилини). При екстраполяції на реальний режим моніторингу (з передачею даних кожні 10–60 хвилин) автономність зростає до 3–12 місяців. Це робить розроблене рішення конкурентоспроможним у порівнянні з промисловими аналогами, які часто вимагають більш дорогих компонентів.

На основі аналізу експериментальних даних сформульовано чіткі рекомендації для проєктування аналогічних систем: для локальних мереж із наявним Wi-Fi покриттям рекомендується використовувати архітектуру на базі CoAP v4; для моніторингу віддалених об'єктів (поля, лісові масиви) безальтернативним варіантом є LoRa v3; а для екосистеми «розумного будинку» доцільно застосовувати BLE v3 або MQTT v4 залежно від наявності відповідного шлюзу. Таким чином, експериментально доведено працездатність та високу ефективність запропонованого способу оптимізації, що дозволяє рекомендувати його для широкого впровадження.

ВИСНОВКИ

У магістерській дисертації вирішено актуальну науково-прикладну задачу підвищення енергоефективності автономних ІТ-систем моніторингу кліматичних показників. Рішення базується на розробці та експериментальній верифікації комплексного підходу, що інтегрує апаратні можливості сучасних мікроконтролерів, оптимізацію комунікаційних протоколів та впровадження інтелектуальних адаптивних алгоритмів керування режимами роботи.

Основні наукові та практичні результати роботи полягають у наступному:

1. Системний аналіз та обґрунтування вибору платформи. Проведений аналіз сучасного стану проблеми енергозбереження в IoT показав, що традиційні методи моніторингу з фіксованими інтервалами вимірювань є енергетично неефективними, особливо в умовах стабільного навколишнього середовища. Вони призводять до нераціонального використання ресурсу джерел живлення через надлишкову активність радіомодулів. На основі порівняльного аналізу архітектур обґрунтовано вибір апаратної платформи на базі мікроконтролера ESP32 та інтегрованого сенсора BME680. Ця комбінація визначена як оптимальна завдяки наявності гнучких режимів глибокого сну (Deep Sleep), підтримці співпроцесора ULP для фонових моніторингу та інтеграції необхідних бездротових інтерфейсів.
2. Розробка мультипротокової архітектури. Створено унікальну програмно-апаратну архітектуру, яка об'єднує п'ять ключових комунікаційних технологій IoT: MQTT, HTTP, CoAP, BLE та LoRaWAN. Для кожного протоколу реалізовано чотирирівневу модель оптимізації (від v1 до v4), що дозволило ізолювати та кількісно оцінити внесок кожного методу енергозбереження. Важливим інженерним досягненням стала розробка спеціалізованих проміжних шлюзів для протоколів CoAP та BLE, які забезпечили їх безшовну інтеграцію у хмарну інфраструктуру AWS,

подолавши обмеження прямого підключення та розширивши сферу застосування системи.

3. Наукова новизна: гібридний метод керування. Ключовим науковим результатом роботи є розробка гібридного методу адаптивного керування енергоспоживанням. Цей метод вперше поєднує локальну адаптацію на рівні мікроконтролера для енергоефективних протоколів з низькою пропускну здатністю (LoRa, BLE) та централізовану хмарну адаптацію на базі AWS Lambda для ресурсоємних IP-протоколів (Wi-Fi). Такий підхід дозволив використати обчислювальну потужність хмари для глибокого аналізу даних та планування інтервалів сну, розвантаживши ресурси кінцевих пристроїв і мінімізувавши час їх перебування в активному режимі.
4. Експериментальне підтвердження ефективності. Натурні випробування повністю підтвердили ефективність запропонованих рішень. Встановлено, що перехід на режим Deep Sleep забезпечує зниження струму споживання у 2,5–5 разів, а впровадження адаптивних алгоритмів додає ще 27–36% економії. Сумарно це дозволило знизити середнє енергоспоживання системи на 72–87% порівняно з базовими реалізаціями. Визначено лідерів енергоефективності: протокол CoAP (середній струм 12,47 мА) для локальних мереж та LoRa (12,63 мА) для глобального покриття. Експериментально спростовано міф про високу енергоємність LoRa, довівши її ефективність завдяки мінімальному часу знаходження в ефірі.
5. Практична цінність та впровадження. Практична значимість роботи полягає у створенні готового до тиражування рішення, яке дозволяє автономним вузлам моніторингу працювати від компактних джерел живлення ємністю 1000 мА·год протягом 3–12 місяців (залежно від сценарію), що значно перевищує показники існуючих аналогів. Низька собівартість апаратного вузла (близько 18 USD) у поєднанні з оптимізованою безсерверною архітектурою AWS робить систему конкурентоспроможною для масового впровадження у сферах точного землеробства, екологічного моніторингу та інфраструктури «розумних міст».

Перспективи подальшого розвитку роботи вбачаються в інтеграції методів машинного навчання (ML) на стороні хмари для прогнозування трендів зміни кліматичних показників, що дозволить переходити від реактивної до предиктивної адаптації інтервалів сну. Розроблені у дисертації технічні рішення, алгоритми та методики є завершеними і можуть бути безпосередньо використані при проектуванні енергоефективних IoT-систем наступного покоління.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications / A. Al-Fuqaha, M. Guizani, M. Mohammadi [et al.] // IEEE Communications Surveys & Tutorials. — 2015. — Vol. 17, no. 4. — P. 2347–2376.
2. IoT Standards and Protocols [Electronic resource] / Postscapes. — Mode of access: <https://www.postscapes.com/internet-of-things-protocols/>.
3. Shelby Z. The Constrained Application Protocol (CoAP). RFC 7252 [Electronic resource] / Z. Shelby, K. Hartke, C. Bormann // IETF. — 2014. — Mode of access: <https://tools.ietf.org/html/rfc7252>.
4. Augustin A. A Study of LoRa: Long Range & Low Power Networks for the Internet of Things / A. Augustin, J. Yi, T. Clausen, W. M. Townsley // Sensors. — 2016. — Vol. 16, no. 9. — P. 1466.
5. Naik N. Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP / N. Naik // 2017 IEEE International Systems Engineering Symposium (ISSE). — Vienna, 2017. — P. 1–7.
6. Karagiannis V. A Survey on Application Layer Protocols for the Internet of Things / V. Karagiannis, P. Chatzimisios, F. Vazquez-Gallego, J. Alonso-Zarate // Transaction on IoT and Cloud Computing. — 2015. — Vol. 3, no. 1. — P. 11–17.
7. MQTT Version 3.1.1. OASIS Standard [Electronic resource] / A. Banks, R. Gupta. — 2014. — Mode of access: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
8. Shelby Z. The Constrained Application Protocol (CoAP). RFC 7252 [Electronic resource] / Z. Shelby, K. Hartke, C. Bormann // IETF. — 2014. — Mode of access: <https://tools.ietf.org/html/rfc7252>.
9. LoRaWAN 1.0.3 Specification [Electronic resource] / LoRa Alliance. — 2018. — Mode of access: <https://resources.lora-alliance.org/technical-specifications/lorawan-specification-v1-0-3/>.
10. LoRaWAN Architecture [Electronic resource] / The Things Network. — Mode of access: <https://www.thethingsnetwork.org/docs/lorawan/architecture/>.

11. Gomez C. Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology / C. Gomez, J. Oller, J. Paradells // Sensors. — 2012. — Vol. 12, no. 9. — P. 11734–11753.
12. Bluetooth Core Specification v4.2 [Electronic resource] / Bluetooth SIG. — 2014. — Mode of access: <https://www.bluetooth.com/specifications/specs/>.
13. ESP32 Series Datasheet [Electronic resource] / Espressif Systems. — 2023. — Mode of access: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
14. BME680. Low power gas, pressure, temperature & humidity sensor [Electronic resource] / Bosch Sensortec. — 2022. — Mode of access: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>.
15. SX1276 Wireless & RF Transceiver [Electronic resource] / Semtech Corporation. — 2024. — Mode of access: <https://www.semtech.com/products/wireless-rf/lora-connect/sx1276>
16. AWS IoT Core Documentation [Electronic resource] / Amazon Web Services. — Mode of access: <https://docs.aws.amazon.com/iot/>.
17. AWS Lambda Developer Guide [Electronic resource] / Amazon Web Services. — Mode of access: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>.
18. FreeRTOS. Real-time operating system for microcontrollers [Electronic resource]. — Mode of access: <https://www.freertos.org>.
19. Piyare R. Ultra Low Power Wake-Up Radios: A Hardware and Networking Survey / R. Piyare, A. L. Murphy, C. Kiraly, P. Tosato // IEEE Communications Surveys & Tutorials. — 2017. — Vol. 19, no. 4. — P. 2117–2157.

ДОДАТОК А

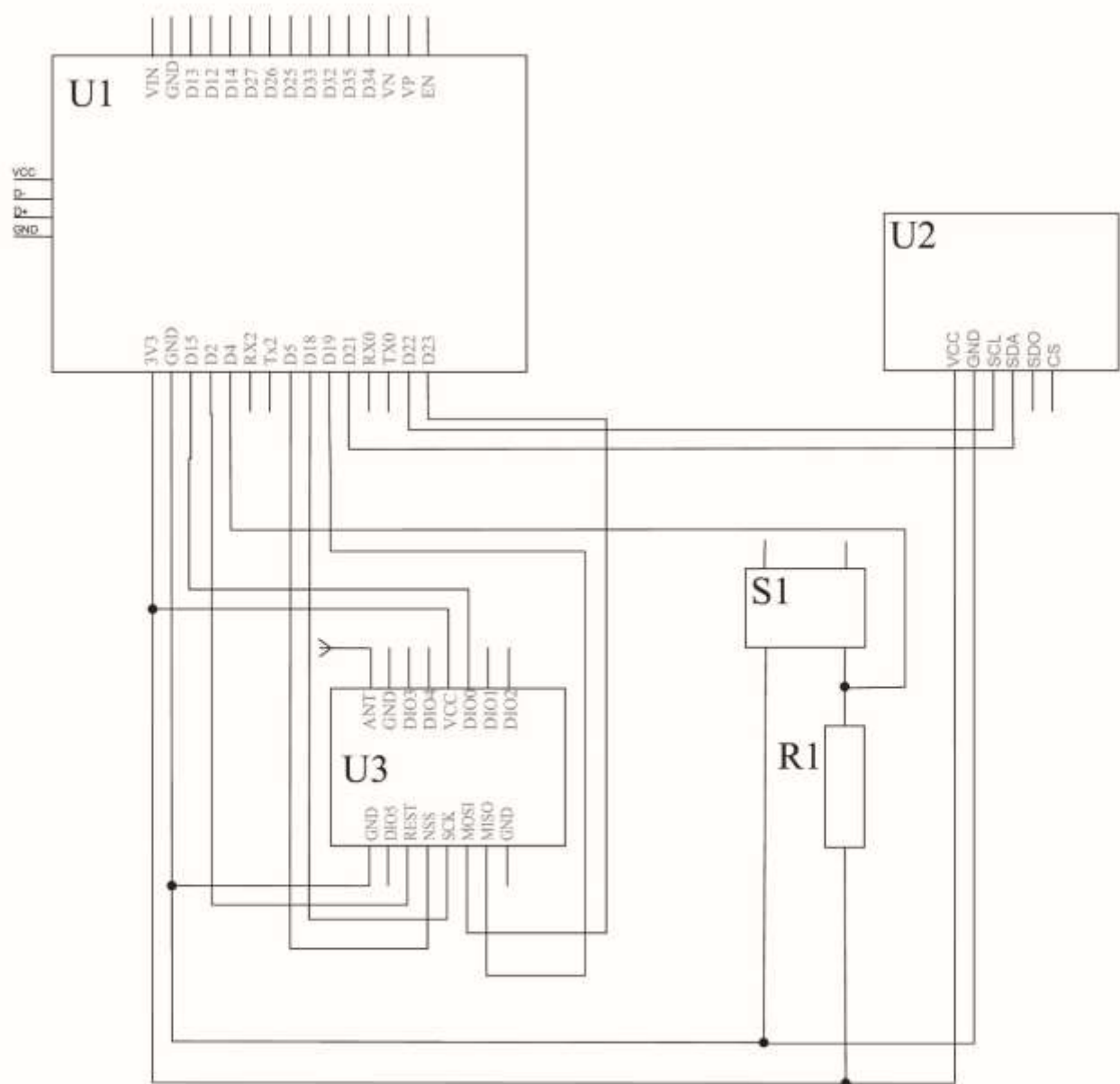
Спосіб оптимізації енергоживлення IoT-системи моніторингу кліматичних показників

Схема електрична принципова

ІАЛЦ.468364.001 ЕЗ

Аркушів 1

Київ - 2025

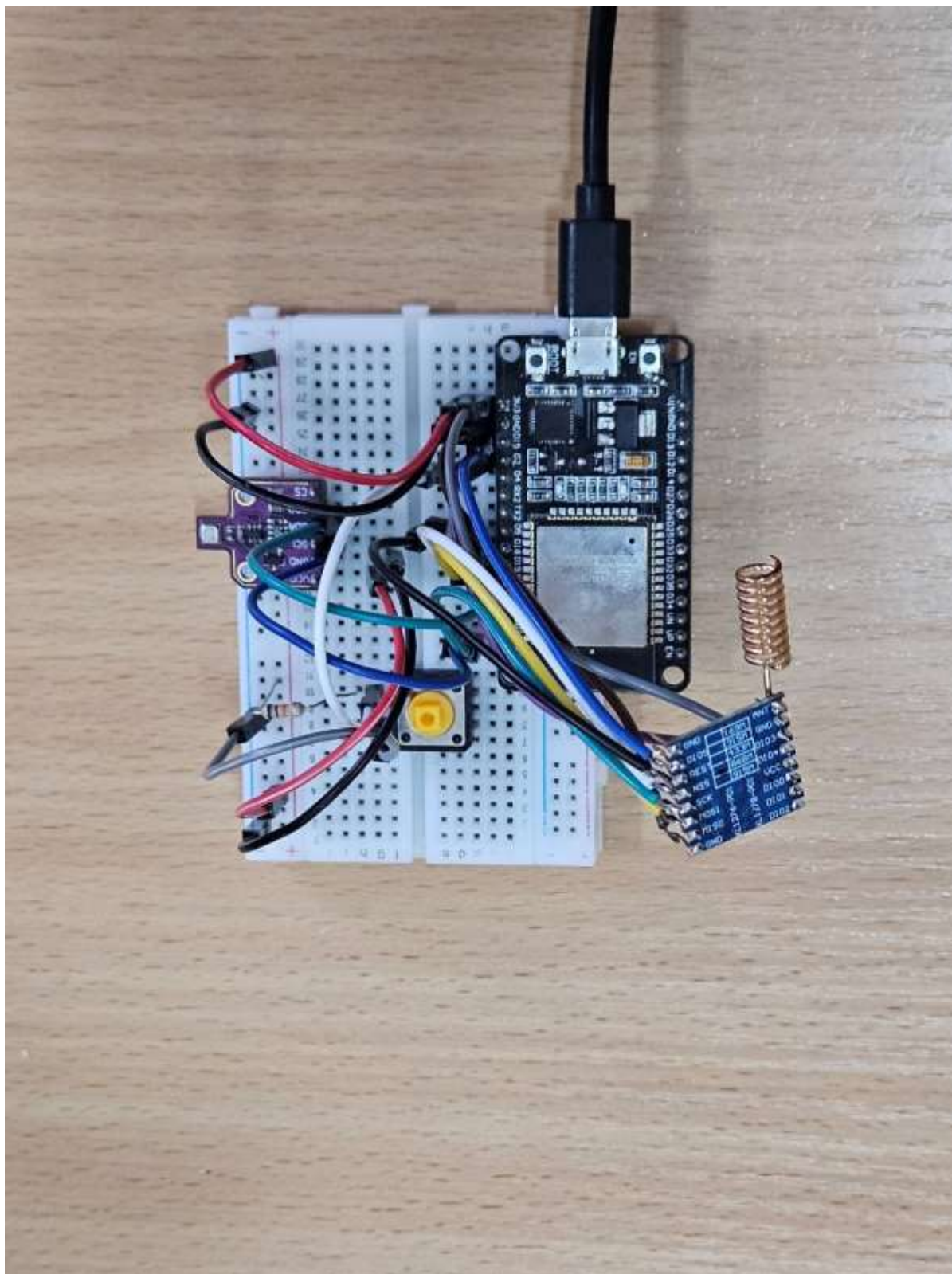


					ІАЛЦ.468364.001 ЕЗ										
Зм	Лист	№ докум.	Підп.	Дата											
Розроб.		Стецюренко І. С.			Спосіб оптимізації енергоживлення IoT-системи моніторингу кліматичних показників					Лім.		Лис		Листів	
Перев.		Петрашенко А. В.										1		1	
					Схема електрична принципова					НТУУ «КПІ ім. Ігоря Сікорського», ФПМ, КВ-41мп					
Н. контр.		Клятченко Я. М.													
Затв.		Романкевич В. О.													

Позначення		Найменування			Кільк.	Прим. 94		
		Мікросхеми (U)						
U1		Плата розробника ESP32 DevKit v1 (SoC ESP-WROOM-32)			1			
U2		Сенсорний модуль BME680 (Bosch Sensortec)			1	I2C		
U3		Радіомодуль LoRa SX1276 (868 МГц)			1	SPI		
		Комутаційні пристрої (S)						
S1		Кнопка тактова тактова KFC-012-7.3F			1			
		Резистори (R)						
R1		Резистор постійний 10 кОм, 0.25 Вт, 5%			1			
		Інші елементи						
—		Макетна плата (Breadboard)			1			
—		Джерело живлення (Power Bank / USB) 5В			1			

ДОДАТОК Б

Загальний вигляд плати



ДОДАТОК В

Лістинги програмного забезпечення

Лістинг 1 — Реалізація алгоритму хмарного керування (протокол MQTT)

Демонструє асинхронну взаємодію з AWS IoT Core, підписку на топик команд та динамічне оновлення інтервалу сну.

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <Adafruit_BME680.h>
#include "time.h"
#include "credentials.h"
#include "esp_sleep.h"

Adafruit_BME680 bme;
WiFiClientSecure espClient;
PubSubClient mqtt_client(espClient);

struct SensorReadings {
    float temperature;
    float humidity;
    float pressure;
    float voc;
    bool success;
};

void enterDeepSleep(int seconds) {
    Serial.printf("Перехід у Deep Sleep на %d секунд...\n", seconds);
    WiFi.disconnect(true);
    mqtt_client.disconnect();
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0);
    esp_sleep_enable_timer_wakeup(seconds * uS_TO_S_FACTOR);
    esp_deep_sleep_start();
}

void callback(char* topic, byte* payload, unsigned int length) {
    Serial.print(F("Отримано повідомлення: "));
    String message;
    for (unsigned int i = 0; i < length; i++) message += (char)payload[i];
    Serial.println(message);

    const char* command_topic = "climate/monitoring/commands";

    if (String(topic) == command_topic) {
        StaticJsonDocument<200> doc;
        DeserializationError error = deserializeJson(doc, message);

        if (!error) {
            if (doc.containsKey("sleep_interval")) {
                int new_interval = doc["sleep_interval"].as<int>();
                sleep_interval = constrain(new_interval, 30, 1200);
                Serial.printf("Хмара оновила інтервал сну: %d c\n", sleep_interval);
            }
            if (doc.containsKey("command") && strcmp(doc["command"], "force_transmit") == 0) {
                Serial.println("Отримано команду примусової передачі (для наступного разу).");
            }
        } else {
            Serial.println(F("Помилка JSON"));
        }
    }
}

void connectWiFi() {
    WiFi.mode(WIFI_STA);
```



```

WiFi.begin(ssid, password);
Serial.print("Підключення до Wi-Fi");
unsigned long start_time = millis();
while (WiFi.status() != WL_CONNECTED && millis() - start_time < wifi_timeout) {
    delay(500);
    Serial.print(".");
}
if (WiFi.status() != WL_CONNECTED) {
    Serial.println(F("\nПомилка Wi-Fi! Сон."));
    enterDeepSleep(sleep_interval);
}
Serial.println(F("\nWi-Fi підключено."));
}

void syncTime() {
    configTime(ntp_offset, 0, ntp_server);
    Serial.print("Синхронізація часу...");
    time_t now = time(nullptr);
    int retries = 0;

    while (now < 1735689600 && retries < 10) {
        delay(500);
        now = time(nullptr);
        Serial.print(".");
        retries++;
    }
    Serial.println();
}

void connectMQTT() {
    espClient.setCACert(aws_root_ca);
    espClient.setCertificate(aws_client_cert);
    espClient.setPrivateKey(aws_private_key);
    mqtt_client.setServer(mqtt_server, 8883);
    mqtt_client.setCallback(callback);

    Serial.print("Підключення до MQTT...");
    if (!mqtt_client.connect(mqtt_client_id)) {
        Serial.print(F(" Помилка! Код: "));
        Serial.println(mqtt_client.state());
        enterDeepSleep(sleep_interval);
    }
    Serial.println(F(" MQTT підключено."));

    mqtt_client.subscribe("climate/monitoring/commands");
}

SensorReadings readSensorData() {
    SensorReadings readings = {0, 0, 0, 0, false};
    if (bme.performReading()) {
        readings.temperature = bme.temperature;
        readings.humidity = bme.humidity;
        readings.pressure = bme.pressure / 100.0;
        readings.voc = bme.gas_resistance / 1000.0;
        readings.success = true;
    } else {
        Serial.println(F("Помилка зчитування BME680!"));
    }
    return readings;
}

void transmitData(SensorReadings data) {
    StaticJsonDocument<200> doc;
    doc["temperature"] = data.temperature;
    doc["humidity"] = data.humidity;
    doc["pressure"] = data.pressure;
    doc["voc"] = data.voc;
    doc["timestamp"] = time(nullptr);

    char buffer[256];
    serializeJson(doc, buffer);

    Serial.printf("Відправка MQTT: %s\n", buffer);

    if (mqtt_client.connected()) {
        mqtt_client.publish(mqtt_topic, buffer);
    }
}

```

```

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\n--- MQTT v4 ---");

  pinMode(BUTTON_PIN, INPUT_PULLUP);
  if(esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0) {
    Serial.println("Кнопка! Скидання інтервалу до 30с.");
    sleep_interval = 30;
  }

  if (!bme.begin()) {
    Serial.println(F("Помилка BME680!"));
    enterDeepSleep(60);
  }
  bme.setTemperatureOversampling(BME680_OS_8X);
  bme.setHumidityOversampling(BME680_OS_2X);
  bme.setPressureOversampling(BME680_OS_4X);
  bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
  bme.setGasHeater(320, 150);

  SensorReadings current = readSensorData();
  if (!current.success) enterDeepSleep(sleep_interval);

  connectWiFi();
  syncTime();
  connectMQTT();

  transmitData(current);

  Serial.println("Очікування команди від хмари...");
  unsigned long wait_start = millis();
  while (millis() - wait_start < 5000) {
    if (mqtt_client.connected()) {
      mqtt_client.loop();
    }
    delay(10);
  }

  enterDeepSleep(sleep_interval);
}

void loop() {}

```

Лістинг 2 — Реалізація алгоритму локальної адаптації (протокол HTTP)

Демонструє отримання керуючої команди безпосередньо у тілі HTTP-відповіді на POST-запит.

```

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <Adafruit_BME680.h>
#include <Wire.h>
#include "credentials.h"
#include "time.h"
#include "esp_sleep.h"

Adafruit_BME680 bme;
WiFiClientSecure espClient;
HTTPClient http;

struct SensorReadings {
  float temperature;
  float humidity;

```

```

float pressure;
float voc;
bool success;
};

void enterDeepSleep(int seconds) {
    Serial.printf("Перехід у Deep Sleep на %d секунд...\n", seconds);
    WiFi.disconnect(true);
    http.end();
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0);
    esp_sleep_enable_timer_wakeup(seconds * 1000000ULL);
    esp_deep_sleep_start();
}

void connectWiFi() {
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    Serial.print("Підключення до Wi-Fi");
    unsigned long start_time = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - start_time < wifi_timeout) {
        delay(500);
        Serial.print(".");
    }
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println(F("\nПомилка Wi-Fi! Сон.));
        enterDeepSleep(sleep_interval);
    }
    Serial.println(F("\nWi-Fi підключено.));
}

void syncTime() {
    configTime(ntp_offset, 0, ntp_server);
    Serial.print("Синхронізація часу...");
    time_t now = time(nullptr);
    int retries = 0;
    while (now < 1735689600 && retries < 10) {
        delay(500);
        now = time(nullptr);
        Serial.print(".");
        retries++;
    }
    Serial.println();
}

SensorReadings readSensorData() {
    SensorReadings readings = {0, 0, 0, 0, false};
    if (bme.performReading()) {
        readings.temperature = bme.temperature;
        readings.humidity = bme.humidity;
        readings.pressure = bme.pressure / 100.0;
        readings.voc = bme.gas_resistance / 1000.0;
        readings.success = true;
    }
    return readings;
}

void transmitDataAndReceiveCommand(SensorReadings data) {
    StaticJsonDocument<200> doc;
    doc["temperature"] = data.temperature;
    doc["humidity"] = data.humidity;
    doc["pressure"] = data.pressure;
    doc["voc"] = data.voc;
    doc["timestamp"] = time(nullptr);

    char buffer[256];
    serializeJson(doc, buffer);

    Serial.printf("Відправка HTTP: %s\n", buffer);

    espClient.setCACert(aws_root_ca);
    String url = String(http_endpoint) + "?mode=control";
    http.begin(espClient, url);
    http.addHeader("Content-Type", "application/json");

    int httpCode = http.POST(buffer);

    if (httpCode == 200) {
        String response = http.getString();
        Serial.println(F("Успішна відповідь від хмари:"));
    }
}

```

```

Serial.println(response);

StaticJsonDocument<200> respDoc;
DeserializationError error = deserializeJson(respDoc, response);
if (!error) {
    if (respDoc.containsKey("sleep_interval")) {
        int new_interval = respDoc["sleep_interval"].as<int>();
        sleep_interval = constrain(new_interval, 30, 1200);
        Serial.printf("Хмара оновила інтервал: %d c\n", sleep_interval);
    }
    else {
        Serial.println("Помилка парсингу JSON відповіді.");
    }
}
else {
    Serial.printf("Помилка HTTP! Код: %d\n", httpCode);
}
http.end();
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\n--- HTTP v4 ---");

    pinMode(BUTTON_PIN, INPUT_PULLUP);
    if(esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0) {
        Serial.println("Кнопка! Скидання інтервалу.");
        sleep_interval = 30;
    }

    if (!bme.begin()) { enterDeepSleep(60); }
    bme.setTemperatureOversampling(BME680_OS_8X);
    bme.setHumidityOversampling(BME680_OS_2X);
    bme.setPressureOversampling(BME680_OS_4X);
    bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
    bme.setGasHeater(320, 150);

    SensorReadings current = readSensorData();
    if (!current.success) enterDeepSleep(sleep_interval);

    connectWiFi();
    syncTime();

    transmitDataAndReceiveCommand(current);

    delay(100);
    enterDeepSleep(sleep_interval);
}

void loop() {}

```

Лістинг 3 — Реалізація протоколу CoAP v2 (Робота з UDP)

Демонструє ручне формування бінарного пакету CoAP та роботу через сокети UDP.

```

#include <WiFi.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <Adafruit_BME680.h>
#include "time.h"
#include "credentials.h"
#include "esp_sleep.h"

WiFiUDP udp;

```

```

Adafruit_BME680 bme;
byte packetBuffer[512];
uint16_t messageID = 0;
bool timeStatus = false;

struct SensorReadings {
    float temperature;
    float humidity;
    float pressure;
    float voc;
    bool success;
};

void enterDeepSleep(int seconds) {
    Serial.printf("Перехід у Deep Sleep на %d секунд...\n", seconds);
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0);
    esp_sleep_enable_timer_wakeup(seconds * 1000000ULL);
    esp_deep_sleep_start();
}

void connectWiFi() {
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    Serial.print("Підключення до Wi-Fi");
    unsigned long start_time = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - start_time < wifi_timeout) {
        delay(500);
        Serial.print(".");
    }
    if (WiFi.status() != WL_CONNECTED) {
        Serial.println(F("\nПомилка підключення Wi-Fi! Перехід у сон."));
        enterDeepSleep(MAIN_INTERVAL / 1000);
    }
    Serial.println(F("\nWi-Fi підключено."));
}

void syncTime() {
    configTime(ntp_offset, 0, ntp_server);
    Serial.print("Синхронізація часу...");
    time_t now = time(nullptr);
    while (now < 1735689600) {
        delay(100);
        now = time(nullptr);
        Serial.print(".");
    }
    Serial.println("\nЧас синхронізовано.");
    timeStatus = true;
}

void sendCoapPutRequest(const char* payload, size_t payloadLen) {
    int bufferIndex = 0;
    packetBuffer[bufferIndex++] = 0b01010000;
    packetBuffer[bufferIndex++] = 0x03;
    packetBuffer[bufferIndex++] = highByte(messageID);
    packetBuffer[bufferIndex++] = lowByte(messageID);
    messageID++;

    packetBuffer[bufferIndex++] = 0xB7; memcpy(&packetBuffer[bufferIndex], "climate", 7); bufferIndex
+= 7;
    packetBuffer[bufferIndex++] = 0x04; memcpy(&packetBuffer[bufferIndex], "data", 4); bufferIndex +=
4;
    packetBuffer[bufferIndex++] = 0xFF;

    memcpy(&packetBuffer[bufferIndex], payload, payloadLen);
    bufferIndex += payloadLen;

    if (udp.beginPacket(coap_server_ip, coap_port)) {
        udp.write(packetBuffer, bufferIndex);
        if (udp.endPacket()) {
            Serial.println("Запит CoAP успішно надіслано.");
        } else {
            Serial.println("Помилка відправки UDP пакета!");
        }
    } else {
        Serial.println("Помилка ініціалізації UDP з'єднання!");
    }
}

SensorReadings readSensorData() {

```

```

SensorReadings readings = {0, 0, 0, 0, false};
if (bme.performReading()) {
    readings.temperature = bme.temperature;
    readings.humidity = bme.humidity;
    readings.pressure = bme.pressure / 100.0;
    readings.voc = bme.gas_resistance / 1000.0;
    readings.success = true;
} else {
    Serial.println(F("Помилка зчитування BME680!"));
}
return readings;
}

void transmitData() {
    SensorReadings data = readSensorData();
    if (!data.success) return;

    StaticJsonDocument<256> doc;
    doc["temperature"] = data.temperature;
    doc["humidity"] = data.humidity;
    doc["pressure"] = data.pressure;
    doc["voc"] = data.voc;
    if (timeStatus) {
        doc["timestamp"] = time(nullptr);
    }

    char buffer[256];
    size_t len = serializeJson(doc, buffer, sizeof(buffer));

    Serial.printf("Відправка CoAP: %s\n", buffer);

    sendCoapPutRequest(buffer, len);
}

void listenForResponse() {
    Serial.println("Очікування відповіді від шлюзу...");
    unsigned long listenStart = millis();
    while (millis() - listenStart < 2000) {
        int packetSize = udp.parsePacket();
        if (packetSize) {
            Serial.println("\n--- Отримано відповідь від шлюзу ---");
            byte replyBuffer[packetSize];
            udp.read(replyBuffer, packetSize);
            return;
        }
        delay(10);
    }
    Serial.println("Час очікування відповіді вийшов.");
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("\n--- CoAP v2 ---");

    pinMode(BUTTON_PIN, INPUT_PULLUP);

    if (!bme.begin()) {
        Serial.println(F("Помилка ініціалізації BME680!"));
        enterDeepSleep(MAIN_INTERVAL / 1000);
    }
    bme.setTemperatureOversampling(BME680_OS_8X);
    bme.setHumidityOversampling(BME680_OS_2X);
    bme.setPressureOversampling(BME680_OS_4X);
    bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
    bme.setGasHeater(320, 150);

    connectWiFi();
    syncTime();
    transmitData();
    listenForResponse();

    enterDeepSleep(MAIN_INTERVAL / 1000);
}

void loop() {
}

```

Лістинг 4 — Програмна реалізація хмарного сервісу AWS Lambda

Демонструє роботу з радіомодулем SX1276, бінарне пакування даних та автономний алгоритм адаптації.

```
#include <SPI.h>
#include <LoRa.h>
#include <Wire.h>
#include <Adafruit_BME680.h>
#include <WiFi.h>
#include "esp_sleep.h"
#include "lorawan_config.h"

RTC_DATA_ATTR float previous_values[4] = {0, 0, 0, 0};
RTC_DATA_ATTR int sleep_interval = 60;
const float stability_thresholds[] = {0.5, 1.0, 1.0, 25.0};
#define uS_TO_S_FACTOR 1000000ULL

Adafruit_BME680 bme;

struct SensorData {
    int16_t temperature;
    uint16_t humidity;
    uint16_t pressure;
    uint16_t voc;
} __attribute__((packed));

void enterDeepSleep(int seconds) {
    Serial.printf("Перехід у Deep Sleep на %d секунд...\n", seconds);
    LoRa.sleep();
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0);
    esp_sleep_enable_timer_wakeup(seconds * uS_TO_S_FACTOR);
    esp_deep_sleep_start();
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    WiFi.mode(WIFI_OFF);
    btStop();
    Serial.println("\n--- LoRa v3 ---");

    pinMode(BUTTON_PIN, INPUT_PULLUP);
    if(esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0) {
        sleep_interval = 30;
    }

    if (!bme.begin()) { enterDeepSleep(60); }
    bme.setTemperatureOversampling(BME680_OS_8X);
    bme.setHumidityOversampling(BME680_OS_2X);
    bme.setPressureOversampling(BME680_OS_4X);
    bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
    bme.setGasHeater(320, 150);

    delay(500);
    float t = 0, h = 0, p = 0, v = 0;
    if (bme.performReading()) {
        t = bme.temperature;
        h = bme.humidity;
        p = bme.pressure / 100.0;
        v = bme.gas_resistance / 1000.0;
    } else {
        enterDeepSleep(sleep_interval);
    }

    bool is_stable = true;
    if (previous_values[0] != 0) {
        if (abs(t - previous_values[0]) > stability_thresholds[0]) is_stable = false;
        if (abs(h - previous_values[1]) > stability_thresholds[1]) is_stable = false;
        if (abs(p - previous_values[2]) > stability_thresholds[2]) is_stable = false;
    }
}
```

```

        if (abs(v - previous_values[3]) > stability_thresholds[3]) is_stable = false;
    } else { is_stable = false; }

    if (is_stable) {
        sleep_interval = min(sleep_interval * 2, 600);
        Serial.printf("Стабільно. Інтервал: %d\n", sleep_interval);
    } else {
        sleep_interval = max(sleep_interval / 2, 30);
        Serial.printf("Зміні. Інтервал: %d\n", sleep_interval);
    }

    previous_values[0] = t; previous_values[1] = h; previous_values[2] = p; previous_values[3] =
v;

    LoRa.setPins(LORA_CS_PIN, LORA_RST_PIN, LORA_DIO0_PIN);
    if (!LoRa.begin(LORA_FREQUENCY)) { enterDeepSleep(sleep_interval); }

    SensorData data;
    data.temperature = (int16_t)(t * 100);
    data.humidity = (uint16_t)(h * 100);
    data.pressure = (uint16_t)(p);
    data.voc = (uint16_t)(v);

    LoRa.beginPacket();
    LoRa.write((uint8_t*)&data, sizeof(data));
    LoRa.endPacket();
    Serial.println("Пакет LoRa відправлено.");
    enterDeepSleep(sleep_interval);
}
void loop() {}

```

Лістинг 5 — Програмна реалізація хмарного сервісу AWS Lambda

Реалізує єдину точку входу для даних, збереження в DynamoDB/Redis та логіку прийняття рішень для адаптивного керування.

```

import os
import json
import boto3
from decimal import Decimal
import time
import redis
from collections import defaultdict
import traceback

def convert_decimals(obj):
    if isinstance(obj, list):
        return [convert_decimals(i) for i in obj]
    elif isinstance(obj, dict):
        return {k: convert_decimals(v) for k, v in obj.items()}
    elif isinstance(obj, Decimal):
        return float(obj)
    else:
        return obj

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('ClimateData')
r = redis.Redis(
    host='redis-11332.c328.europe-west3-1.gce.redns.redis-cloud.com',
    port=11332,
    password=os.environ.get('REDIS_PASSWORD'),
    db=0,
    decode_responses=True
)
mqtt_client = boto3.client('iot-data', region_name='eu-central-1')

```



```

def lambda_handler(event, context):
    try:
        r.ping()
        print("Redis підключено")

        # Отримання вхідних даних
        if 'payload' in event:
            payload_str = event['payload']
        elif 'message' in event:
            payload_str = event['message']
        else:
            payload_str = json.dumps(event)

        payload = json.loads(payload_str)
        if isinstance(payload, list) and payload:
            payload = payload[0]

        current = {
            'temperature': float(payload.get('temperature', 0.0)),
            'humidity': float(payload.get('humidity', 0.0)),
            'pressure': float(payload.get('pressure', 0.0)),
            'voc': float(payload.get('voc', 0.0))
        }

        # Отримання попередніх даних із Redis
        prev_data = r.json().get('climate:latest', '$')
        if isinstance(prev_data, list) and prev_data:
            prev_data = prev_data[0] if prev_data else {}
        elif not prev_data:
            prev_data = {}

        prev = {
            'temperature': float(prev_data.get('temperature', current['temperature'])),
            'humidity': float(prev_data.get('humidity', current['humidity'])),
            'pressure': float(prev_data.get('pressure', current['pressure'])),
            'voc': float(prev_data.get('voc', current['voc']))
        }

        # Перевірка різниці температури для force_transmit
        temp_diff = abs(current['temperature'] - prev['temperature'])
        if temp_diff > 10.0: # Узгоджено з порогом ESP32
            mqtt_cmd = {"command": "force_transmit"}
            print("Sending force transmit:", json.dumps(mqtt_cmd))
            mqtt_client.publish(topic='climate/monitoring/commands', payload=json.dumps(mqtt_cmd))

        # Обчислення sleep_interval на основі всіх метрик
        diffs = [
            abs(current['temperature'] - prev['temperature']),
            abs(current['humidity'] - prev['humidity']),
            abs(current['pressure'] - prev['pressure']),
            abs(current['voc'] - prev['voc'])
        ]
        avg_diff = sum(diffs) / len(diffs) if diffs else 0.0

        sleep_interval = 600 # Базовий інтервал для ULP
        if avg_diff > 2.0: # Висока зміна
            sleep_interval = 30
        elif avg_diff > 1.0: # Помірна зміна
            sleep_interval = 120
        else: # Низька зміна
            sleep_interval = 600

        # Надсилання команди sleep_interval
        sleep_cmd = {"command": "sleep_interval", "sleep_interval": sleep_interval}
        print("Sending sleep command:", json.dumps(sleep_cmd))
        mqtt_client.publish(
            topic='climate/monitoring/commands',
            payload=json.dumps(sleep_cmd),
            qos=0
        )
        print("Sleep command published")

        # Збереження даних у DynamoDB і Redis
        now = int(time.time())
        item = {
            'timestamp': now,
            'temperature': Decimal(str(current['temperature'])),
            'humidity': Decimal(str(current['humidity'])),
            'pressure': Decimal(str(current['pressure'])),

```

```

        'voc': Decimal(str(current['voc']))
    }
    table.put_item(Item=item)
    clean_item = convert_decimals(item)
    r.json().set('climate:latest', '$', clean_item)
    # r.expire('climate:latest', 180)

# Агрегація даних із DynamoDBz
response = table.scan(
    FilterExpression='humidity <> :zero AND pressure <> :zero AND voc <> :zero',
    ExpressionAttributeValues={':zero': Decimal('0.0')}
)
if 'Items' not in response or not response['Items']:
    print("No items found in DynamoDB")
    return {
        'statusCode': 200,
        'body': json.dumps([]),
        'headers': {'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*'}
    }
grouped = defaultdict(list)
for item in response['Items']:
    ts_ms = int(item['timestamp']) * 1000
    for metric in ['temperature', 'humidity', 'pressure', 'voc']:
        value = float(item.get(metric, 0.0))
        if value is not None and (metric == 'temperature' or value != 0.0):
            grouped[metric].append([value, ts_ms])

# Сортювання за таймстемпом
for metric in grouped:
    grouped[metric].sort(key=lambda x: x[1])

result = [
    {"target": metric, "datapoints": datapoints}
    for metric, datapoints in grouped.items()
]
return {
    'statusCode': 200,
    'body': json.dumps(result),
    'headers': {'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*'}
}
except Exception as e:
    print("Exception:", e)
    traceback.print_exc()
    return {
        'statusCode': 500,
        'body': json.dumps({'error': str(e)})
    }

```

ДОДАТОК Г

Копії публікацій за темою дисертації



МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ЦЕНТРАЛЬНОУКРАЇНСЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

проспект Університетський, 8, м. Кропивницький, 25006, тел.: (0522) 55-92-34,
E-mail: rector@kntu.kr.ua; сайт: kntu.kr.ua; код з'єдн. з ЄДРПОУ 02070950

від 14 12 2025 р. № 12-09/09-1245 На № _____ від _____ 20__ р.

Щодо публікації в науковому
фаховому виданні

Довідка

видана І. С. Стецюренко, магістранту Національного технічного університету України "Київський політехнічний інститут імені Ігоря Сікорського", про те, що наукова стаття

І. С. Стецюренко А. В. Петрашенко
СПОСІБ ОПТИМІЗАЦІЇ ЕНЕРГОЖИВЛЕННЯ ІОТ-СИСТЕМИ МОНІТОРІНГУ
КЛІМАТИЧНИХ ПОКАЗНИКІВ

прийнята до друку редакційною колегією наукового збірника «Центральноукраїнський науковий вісник. Технічні науки» / «Central Ukrainian Scientific Bulletin. Technical Sciences» – Вип. 13(44), ч. 1, 2026 (ідентифікатор медіа R30-03350; ISSN 2664-262X(p); ISSN 2707-9449(o); DOI 10.32515/2664-262X; maiea.kntu.kr.ua; включений до Переліку наукових фахових видань України, в яких можуть публікуватись результати дисертаційних робіт на здобуття наукових ступенів доктора і кандидата наук (доктора філософії) в галузі технічних наук; категорія «Б»).

Довідка видана для пред'явлення за місцем вимоги.

Проректор з наукової роботи
та міжнародних зв'язків



Андрій ТИХИЙ

II International Final R&D online conference
“Advances in Science and Technology”



Technical University of Ukraine “Igor Sikorsky Kyiv Polytechnic Institute”
 Dnipro University of Technology
 National University “Yuri Kondratyuk Poltava Polytechnic”
 Taras Shevchenko National University of Kyiv
 Hellenic Mediterranean University, Greece
 Cyprus University of Technology, Cyprus
 Wiesbaden Business School, Germany



**II INTERNATIONAL FINAL R&D ONLINE CONFERENCE
 OF THE II INTERNATIONAL STUDENT RESEARCH PAPER
 COMPETITION
 “ADVANCES IN SCIENCE AND TECHNOLOGY”**

December 04, 2025

CONFERENCE PROCEEDINGS

Kyiv – 2025

CONTENT

SECTION: FUTURE TRENDS IN SCIENCE AND TECHNOLOGY

STRATEGIC FORESIGHT: NAVIGATING SIX MACRO TRENDS IN SCIENCE AND TECHNOLOGY (2025–2035)

Oleksandr Akimov, Kostiantyn Radchenko, Olena Shepeleva 3

TRANSDERMAL MEDICAL PATCHES FOR DELIVERING DRUGS FOR SLEEP DISORDERS

Anastasiia Bahalika, Valentyna Motronenko, Natalia Kompanets 6

QUADCOPTER STABILIZATION SYSTEMS

Valeriia Bekas, Olena Tachinina, Valentyna Lukianenko 9

METAMORPHIC TESTING: THE FUTURE OF RELIABLE AI

Oleksandr Belitskyi, Yakiv Yusyn, Olena Mukhanova 12

THE EVOLUTION OF INNOVATIONS IN THE NEXT DECADE: THE ROLE OF ARTIFICIAL INTELLIGENCE AND EMERGING TECHNOLOGIES

Yaroslav Bespechnyi, Rostyslav Pashov, Kateryna Tuliakova 15

AI-GENERATED CODE SECURITY

Mykola Bodnar, Andrii Rodionov, Maryna Degtiarenko 18

BIOMECHANICAL PRINCIPLES OF DESIGNING ADAPTIVE CYCLING FOR PEOPLE WITH UPPER LIMB AMPUTATIONS

Mariya Bulakayeva, Ganna Ovcharenko, Nataliia Kompanets 21

FUTURE TRENDS IN SCIENCE AND TECHNOLOGY: THE EVOLUTION OF COMPANION ROBOTS IN THE NEXT DECADE

Valeriia Burun, Anton Uhnienko, Viktor Zaitsev, Oksana Korbut 23

HOW THE SPREAD OF ARTIFICIAL INTELLIGENCE AFFECTS HUMAN ACTIVITY AND MOTIVATION

Kostiantyn Cherednyk, Kostiantyn Radchenko, Olena Shepeleva 26

THE EVOLUTION OF COMPUTATIONAL POWER: FROM MOORE'S LAW TO QUANTUM COMPUTING

Bohdan Cherniak, Anton Lavrinenko, Iryna Lytovchenko 28

THE ROLE OF BIOTECHNOLOGY IN HUMAN LONGEVITY AND HEALTH

Oleksandr Chukhmanenko, Natalia Poedinok, Nataliia Kompanets 30

THE RISE OF DIGITAL CURRENCIES: REDEFINING THE GLOBAL MARKET

Denys Derliuk, Kostiantyn Radchenko, Olena Shepeleva 33

THE INTEGRATION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING IN NEXT-GENERATION METEOROLOGICAL FORECASTING SYSTEMS

Part I. Future Trends in Science and Technology

<i>Viktoriia Duiko, Maria Hryhorak, Olena Kovalova</i>	35
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY: HOW ARTIFICIAL INTELLIGENCE WILL EVOLVE AND CHANGE THE SURROUNDING WORLD	
<i>Mykhailo Duka, Kostiantyn Radchenko, Olena Shepeleva</i>	37
AI-POWERED EMBEDDED SYSTEMS: SMART SENSORS FOR REAL-TIME MONITORING	
<i>Olga Fialkovska, Kostiantyn Radchenko, Olena Shepeleva</i>	39
SMART TECHNOLOGIES IN HEALTHCARE: INNOVATIONS AND IMPACT	
<i>Kateryna Grechishkina, Kostiantyn Radchenko, Olena Shepeleva</i>	42
INNOVATION IN CANCER TREATMENT USING AI	
<i>Maxym Haliuk, Kostiantyn Radchenko, Olena Mukhanova</i>	44
QUANTUM COMPUTING: THE IMPENDING THREAT TO MODERN CRYPTOGRAPHY AND THE SHIFT TO POST-QUANTUM STANDARDS	
<i>Kate Harmatiuk, Kostiantyn Radchenko, Olena Shepeleva</i>	47
PROACTIVE CYBERSECURITY FOR NEXT-GENERATION TECHNOLOGIES: BUILDING RESILIENT DIGITAL ECOSYSTEMS	
<i>Maksym Hinkul, Volodymyr Demchynskyi, Kateryna Havrylenko</i>	50
SPACE-BASED MANUFACTURING & ENERGY: OUT-OF-EARTH FACTORIES AND ORBIT POWER SYSTEMS	
<i>Victoria Honchar, Kostiantyn Radchenko, Olena Shepeleva</i>	52
DIGITAL DISRUPTION IN BRAND COMMUNICATION: TRACING THE JOURNEY FROM BILLBOARDS TO ALGORITHMS	
<i>Yuliia Hrokh, Tetiana Skorokhod, Inna Antonenko</i>	55
THE PAST AND FUTURE OF CHESS AI	
<i>Andrii Humanitskyi, Kostiantyn Radchenko, Olena Shepeleva</i>	57
HOW ARTIFICIAL INTELLIGENCE EVOLVE AND IMPACT HUMAN LIVES	
<i>Maksym Khomenko, Ivan Dychka, Olena Mukhanova</i>	60
ARTIFICIAL INTELLIGENCE IN LEGAL PRACTICE: OPPORTUNITIES AND RISKS	
<i>Valentina Knyazeva, Viktoriia Sydorenko, Inna Borkovska</i>	62
AI-DRIVEN IMPROVEMENTS FOR SOFTWARE-DEFINED NETWORKS: CURRENT STATE AND FUTURE PROSPECTS	
<i>Yana Koroliuk, Yuriy Kochura, Olha Grabar</i>	64
MODERN IMAGE COMPRESSION AND ITS POSSIBLE FUTURE	
<i>Oleksandr Korotych, Igor Tereikovskiy, Olena Mukhanova</i>	66

Part I. Future Trends in Science and Technology

METHOD FOR GENERATING SOURCE CODE DESCRIPTION USING AN ARTIFICIAL INTELLIGENCE MODEL	
<i>Albina Kostiuchenko, Andrii Petrashenko, Olena Mukhanova</i>	68
SORA 2 AS A FUTURE OF HUMANITY WITH AI	
<i>Bohdana Kovalenko, Oleksandra Yemelianenko, Myroslava Pshenychna, Inna Antonenko</i>	71
THE ROLE OF THE ECONOMY IN THE DEVELOPMENT OF MODERN SOCIETY	
<i>Yelizaveta Kovalenko, Maryna Petrenko</i>	73
THE ABSTRACTION REVOLUTION: BRIDGING CODE AND CIRCUITS IN THE ERA OF CUSTOM SILICON	
<i>Maksym Krutohuz, Kostiantyn Koliada, Olena Shepeleva</i>	75
MODERN TECHNIQUES, CHALLENGES & ECOSYSTEM STRATEGIES OF ABI STABILITY WITH PLUGIN ARCHITECTURES IN C++	
<i>Kostiantyn Kryvak, Konstiantyn Radchenko, Olena Shepeleva</i>	77
CANCEL CULTURE AND CONTEMPORARY ART: SOCIAL LIMITS OF CREATIVE FREEDOM	
<i>Liliia Lebid, Mykhailo Shapovalenko, Inna Antonenko</i>	80
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY	
<i>Ivan Levchuk, Kostiantyn Radchenko, Olena Shepeleva</i>	82
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY	
<i>Daniil Liashenko, Kostiantyn Radchenko, Olena Shepeleva</i>	84
3D PRINTING IN BIOMEDICAL ENGINEERING: PERSONALIZED BONE IMPLANTS FOR ENHANCED TISSUE REGENERATION	
<i>David Lipartiia, Hanna Ovcharenko, Nataliia Kompanets</i>	87
LEVERAGING AI-POWERED SMART SECURITY SYSTEMS FOR URBAN SAFETY AND INCIDENT PREDICTION	
<i>Yevhen Lukyanchuk, Olha Yefimova</i>	89
FUTURE TRENDS IN AIR QUALITY MONITORING: FROM REACTIVE MEASUREMENT TO PREDICTIVE INTELLIGENCE	
<i>Bohdan Lutsenko, Kostiantyn Radchenko, Olena Mukhanova</i>	91
CLASSIFYING LUNG DISEASES WITH ARTIFICIAL INTELLIGENCE	
<i>Denys Medvid, Serhii Lypovchenko, Serhii Kysliak, Nataliia Kompanets</i>	94
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY: ARTIFICIAL INTELLIGENCE IN MEDICINE	
<i>Vladyslav Maloivan, Kostiantyn Radchenko, Olena Shepeleva</i>	96

Part I. Future Trends in Science and Technology

THE FUTURE POTENTIAL OF QUANTUM COMPUTING: BETWEEN ENGINEERING CHALLENGES AND REVOLUTIONARY APPLICATIONS	
<i>Denys Manuilov, Kostiantyn Radchenko, Olena Shepeleva</i>	98
FUTURE: THE CONVERGENCE OF AI, BIOTECHNOLOGY AND HUMAN POTENTIAL	
<i>Anna Minenok, Diana Nahoha, Mykhailo Shapovalenko, Inna Antonenko</i>	100
ADAPTIVE COMMUNICATION PROTOCOLS FOR MILITARY NETWORKS BASED ON AI SIGNAL PREDICTION	
<i>Dmytro Miz, Anastasiia Rubak, Olena Betsko</i>	102
DNA-BASED COMPUTATION: BEYOND THE LIMITS OF TRADITIONAL ELECTRONICS	
<i>Sofiia Mykhailichenko, Kostiantyn Radchenko, Olena Mukhanova</i>	105
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY	
<i>Dmytro Nedoboiko, Kostiantyn Radchenko, Olena Shepeleva</i>	107
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY: INNOVATION OVER THE NEXT DECADE	
<i>Myroslava Nitchenko, Ivan Kostiuchenko, Iryna Litovchenko</i>	110
AI IN UKRAINE'S DEFENSE: BALANCING INNOVATION AND REGULATION	
<i>Tețiana Onoprienko, Rostyslav Pashov, Kateryna Tuliakova</i>	111
FROM ARTIFICIAL INTELLIGENCE TO GREEN ENERGY	
<i>Vlada Ovadenko, Kostiantyn Radchenko, Olena Shepeleva</i>	115
THE INNOVATIVE TECHNOLOGIES THAT AWAIT US OVER THE NEXT DECADE WILL BE ACTIVELY COMBINED WITH COOKING AND DESIGN	
<i>Sofiia Pavliuk, Mariia Vyskrebtsova, Myroslava Pshenychna, Inna Antonenko</i>	117
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY	
<i>Yehor Poliakov, Kostiantyn Radchenko, Olena Shepeleva</i>	118
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY: INNOVATIONS SHAPING THE NEXT DECADE	
<i>Yegor Popov, Kostiantyn Radchenko, Olena Shepeleva</i>	120
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY	
<i>Victoria Romanchenko, Kostiantyn Radchenko, Olena Shepeleva</i>	123
AN INTELLIGENT SYSTEM FOR RECOMMENDING PERSONALIZED EDUCATIONAL COURSES BASED ON STUDENTS' ACADEMIC PERFORMANCE AND INTERESTS	
<i>Yaroslav Rubis, Lesia Konoplianyk</i>	125

Part I. Future Trends in Science and Technology

ARTIFICIAL INTELLIGENCE IN HEALTHCARE: FROM DIAGNOSTICS TO DRUG DISCOVERY	
<i>Vadim Saiuk, Kostiantyn Radchenko, Olena Shepeleva</i>	127
CYBERSECURITY IN THE AGE OF ARTIFICIAL INTELLIGENCE: CHALLENGES AND SECURE-BY-DESIGN SOLUTIONS	
<i>Valeriia Sapozhko, Kostiantyn Radchenko, Olena Shepeleva</i>	129
THE FUTURE OF TECHNOLOGY	
<i>Tetiana Savchuk, Konstantin Radchenko, Olena Shepeleva</i>	132
USING ARTIFICIAL INTELLIGENCE INSTEAD OF LAWYER SERVICES IN THE FIELD OF JUSTICE	
<i>Kristina Shandrenko, Yevhen Lukyanchikov, Inna Borkovska</i>	133
FUTURE TRENDS IN SCIENCE AND TECHNOLOGY	
<i>Yevhen Shcherbatiuk, Kostiantyn Radchenko, Olena Shepeleva</i>	135
APPLICATION OF ASSOCIATION RULES AND SUPPORT VECTOR MACHINES FOR EARLY DETECTION OF FINANCIAL FRAUD	
<i>Oleksii Shtyrbulov, Lesia Konoplianyk</i>	137
BIOTECHNOLOGY AND PERSONALISED MEDICINE OF THE FUTURE	
<i>Kristina Shvydko, Kostiantyn Radchenko, Olena Shepeleva</i>	139
HYBRID KEYWORD EXTRACTION USING MODIFIED TF-IDF AND BERT-BASED SEMANTIC FILTERING	
<i>Illia Silin, Kostiantyn Radchenko, Olena Mukhanova</i>	141
SHAPING THE DIGITAL FRONTIER: THE ROLE OF AI, BLOCKCHAIN, AND CYBERSECURITY IN THE COMING DECADE	
<i>Arsen Skibchuk, Kostiantyn Radchenko, Olena Shepeleva</i>	144
THE LATEST ARTIFICIAL INTELLIGENCE BOOM AND WHAT TO EXPECT OF IT	
<i>Hlib Skopyk, Yakiv Yusyn, Olena Mukhanova</i>	146
BIOMETRIC CRYPTOGRAPHY: MERGING IDENTITY VERIFICATION WITH PRIVACY	
<i>Maksym Slobodzhian, Oksana Shkurat, Olena Mukhanova</i>	154
ECO-MATERIALS AND PLASTIC-FREE TECHNOLOGIES IN PACKAGING	
<i>Anastasia Sosnovska, Sophia Zamkova, Zoreslava Raiko, Olena Nazarenko, Inna Antonenko</i>	156
ADAPTIVE ENERGY OPTIMIZATION IN IOT: A COMPARATIVE ANALYSIS OF WI-FI, BLE, AND LORAWAN	
<i>Illia Stetsiurenko, Andrii Petrashenko, Olena Mukhanova</i>	159

Part I. Future Trends in Science and Technology

MODERN INNOVATIONS IN UKRAINIAN SCIENCE AND TECHNOLOGY*Anastasia Tsopa, Kostiantyn Radchenko, Olena Shepeleva* 161**MACHINE LEARNING***Taras Turchyk, Kostiantyn Radchenko, Olena Shepeleva* 162**FACTORIES AMONG THE STARS: THE GROWTH OF MAKING THINGS IN SPACE***Volodymyr Tymchenko, Kostiantyn Radchenko, Olena Shepeleva* 165**FUTURE TRENDS IN SCIENCE AND TECHNOLOGY***Sviatoslav Valchyshen, Kostiantyn Radchenko, Olena Shepeleva* 167**THE IMPACT OF QUANTUM COMPUTING ON AI: APPLICATIONS IN WORK AND SOCIETY***Davyd Vdovenko, Oleksandr Gaidai, Olena Volkova* 169**AI LITERACY IN SECONDARY EDUCATION: PREPARING STUDENTS FOR THE FUTURE OF WORK AND DIGITAL CITIZENSHIP***Dana Vitrovchak, Rostyslav Pashov, Kateryna Tuliakova* 171**EXPLORING THE FUTURE OF CLOUD COPUTING: TRENDS, INNOVATIONS AND CHALLENGES***Anna Volnytska, Kostiantyn Radchenko, Olena Shepeleva* 174**DEVELOPMENT OF FOILING METHODS AS A MEANS OF INCREASING THE ATTRACTIVENESS OF PRINTED PRODUCTS***Anna Yemets, Olha Andrushchenko, Olena Nazarenko, Inna Antonenko* 176**INNOVATIONS IN SCIENCE AND TECHNOLOGY OF THE NEXT DECADE***Oksana Yurchenko, Rostyslav Pashov, Kateryna Tulakova* 178**ARTIFICIAL INTELLIGENCE AND THE LEGAL EVALUATION OF DIGITAL EVIDENCE***Anastasiia Zabolotska, Yevhen Lukianchikov, Inna Borkovska* 180**THE FUTURE OF ARTIFICIAL INTELLIGENCE IN EDUCATION: PERSONALIZED LEARNING THROUGH TECHNOLOGY***Kyrylo Zdrovenko, Yakiv Yusyn, Olena Mukhanova* 182**AUTOMATION IN COMPUTER ENGINEERING***Eduard Zemlyanskyi, Kostiantyn Radchenko, Olena Shepeleva* 185**INTEGRATION OF ARTIFICIAL INTELLIGENCE INTO BIONIC PROSTHESES***Artem Zlenko, Kostiantyn Radchenko, Olena Shepeleva* 186

ADAPTIVE ENERGY OPTIMIZATION IN IOT: A COMPARATIVE ANALYSIS OF WI-FI, BLE, AND LORAWAN

Illia Stetsiurenko, Andrii Petrashenko, Olena Mukhanova

Faculty of Applied Mathematics, National Technical University of Ukraine

"Igor Sikorsky Kyiv Polytechnic Institute"

Keywords: Internet of Things (IoT), energy efficiency, ESP32, BME680, Deep Sleep, adaptive algorithm, MQTT, CoAP, Bluetooth Low Energy (BLE), LoRaWAN, AWS, cloud-centric control.

Introduction. Today, IoT has emerged as a mature idea that is a mainstream technological trend with projections of over 29 billion connections by the year 2030. A large portion of this comes from automatic monitoring systems in environmental studies, agriculture, as well as smart cities. In these applications, sensor nodes are often located in remote areas, leading to full dependence on batteries. This is a challenge in IoT that is primarily driven by the need to be as energy-efficient as possible since this affects the overall viability of the system. Today, data transmission in IoT systems is done through fixed time schedules. This is inefficient as the device continues to transmit data even when parameters are not changing, yet it is not able to capture fast changes that require quick acting systems that can dynamically adapt to environment variations.

Objectives. This article elaborates a holistic approach to optimizing the energy efficiency of an IoT-based climate monitoring system by proposing a new adaptive algorithm. It also compares the performance of the proposed algorithm under three communication network configurations. Based on a comprehensive analysis of hardware components (ESP32), software components (Deep Sleep), as well as network protocols (MQTT, HTTP, CoAP, Bluetooth Low Energy (BLE), LoRaWAN), we develop a holistic solution that is multilevel, cloud-based, and optimized in terms of IoT-based climate monitoring systems. Finally, the aim is to provide a new testbed solution to measure performance in a meaningful way as well as to provide recommendations in the field of sustainable IoT solutions.

Methods. This article proposes a prototyping-based experimental-analytical approach, particularly targeting hardware prototyping and experimental analysis. Specifically, the central point of the experimental framework is embodied by the ESP32-WROOM-32 board due to the presence of a high-power CPU core, Wi-Fi, Bluetooth Low Energy (BLE), as well as high-end DeepSleep modes (Espressif Systems, 2024). Climate data (temperature, humidity, air pressure, and aldehydes (VOC)) is measured by the use of Bosch BME680 (Bosch Sensortec, 2023). Also, to sufficiently cover experimental activities towards LoRaWAN-based architectures, a Semtech SX1276 is used. Lastly, the cloud integration framework is done in Amazon Web Services (AWS). Specifically, Amazon Lambda, as a functional computing service, Amazon DynamoDB as primary data warehousing, as well as Amazon

ElastiCache (Redis), as a high-speed database, is employed. More specifically, the central innovation block of this article, called the adaptive algorithm, is designed as a Finite State Machine (FSM), as proposed in this article, specifically under two paradigms: a “local, device-centric” one, where data analysis is conducted at the ESP32 board level by skipping data transmission if data is deemed stable, as well as a “Cloud-centric” one, where data analysis is conducted in AWS Lambda, commanding the board to sleep accordingly.

Results. The primary result is a framework that uses evolution-based optimization techniques to facilitate a stepwise analysis of optimized gains in energy savings. This starts from a “Baseline,” fixed “Always-On” approach (v1), a “Deep Sleep” optimized solution with fixed time periods (v2), and concludes with adaptive “Local Adaptation” (v3) and “Cloud Adaptation” (v4). Despite not being presently totally experimentally validated, theoretical predictions of gains appear promising. Simply progressing from v1 to v2 is predicted to achieve over a 98% reduction in current draw on average. Compared to v2, adaptive strategies (v3 & v4) will continue to promote a further reduction of between 40% to 70% in power usage as a result of eliminating “junk” data transmissions. At a hardware level, CoAP v4 & MQTT v4 will represent the most power-efficient communication protocols over Wi-Fi. As absolute power-saving champions in this “winner-takes-all” equation, respectively, will be the efficiency of BLE v3 & LoRaWAN v3 (Raza, Kulkarni, & Sooriyabandara, 2017). These will require (on average) incredibly small current values in the microampere (μA) range, theoretically extending battery life from weeks to years. “Local Adaptation” at v3 is expected to represent the most power-efficient transaction-based solution in this strategy, most appropriate therefore for ultra-low-power applications.

Conclusion. This article describes the structure of a new hybrid-multi-layer approach in the field of energy optimization of standalone IoT systems. It is important that the presented solution describes a new way of making a direct mathematical comparison between three rather disparate communication solutions: direct Wi-Fi communication, local BLE-gateway, and globally operated LoRaWAN. What is more important, this will be done in one monitoring task. This is rather valuable in practice since this piece of research is set to develop a concrete engineering framework, as well as a set of recommendations that will be validated. Moreover, this will result in a confirmation that by effectively adjusting a device “wake-and-sleep” schedule according to data stability, a balance between data validity and a long-term stand-alone functioning of the device is achievable. This approach will directly result in a sustainable as well as cost-efficient monitoring system in precision agriculture as well as environment science.

References:

Bosch Sensortec. (2023). *BST-BME680-DS001: BME680 Low power gas, pressure, temperature & humidity sensor*. <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>

- Espressif Systems. (2024). *ESP32 Series Datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Raza, U., Kulkarni, P., & Sooriyabandara, M. (2017). Low Power Wide Area Networks: An Overview. *IEEE Communications Surveys & Tutorials*, 19(2), 855–873. <https://doi.org/10.1109/COMST.2017.2652320>

MODERN INNOVATIONS IN UKRAINIAN SCIENCE AND TECHNOLOGY

Anastasia Tsopa, Kostiantyn Radchenko, Olena Shepeleva

Faculty of Applied Mathematics, National Technical University of Ukraine

"Igor Sikorsky Kyiv Polytechnic Institute"

Keywords: Ukrainian Innovations, Science and Technology, Artificial Intelligence (AI), Biotechnology, Sustainable Technology, Technological Trends, Engineering.

Introduction. In recent years, Ukraine has demonstrated significant progress in the field of science and technology, introducing innovative ideas that are already shaping the future of global development. Despite difficult social and economic conditions, Ukrainian researchers and engineers continue to create technologies that combine sustainability, artificial intelligence, and creative problem-solving. These innovations not only address present-day challenges but also show how science will evolve in the coming decade.

Objectives. The objective of this paper is to review and analyze recent key technological innovations from Ukraine across various sectors. It aims to highlight Ukraine's contribution to global scientific trends, particularly in sustainable technology, biotechnology, and artificial intelligence, and to demonstrate the nation's problem-solving capabilities and potential for future technological development.

Methods. This study is based on a descriptive analysis of current technological developments and successful case studies from Ukraine. The methodology involves a review of recent scientific publications, industry reports, and reputable journalistic sources to identify and evaluate significant innovations and their impact on both the national and global stages.

Results. One of the most remarkable trends is the development of eco-friendly technologies. Ukrainian start-up Re-leaf Paper, founded by scientist Valentyn Frechka, introduced a method of producing paper from fallen leaves. This innovation presents a vision of a future where waste materials are transformed into valuable products (Dydiv & Dydiv, 2023). The idea has gained global attention and inspired similar sustainable projects in other countries, contributing to the trend of circular economy and responsible resource management. Another direction that defines the future of technology is biotechnology and human-AI integration.

Ukrainian company Esper Bionics created an intelligent bionic prosthesis capable of learning from the user's muscle signals. This technology demonstrates how

ПРИКЛАДНА МАТЕМАТИКА ТА КОМП'ЮТИНГ ПМК' 2025

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ
MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE
NATIONAL TECHNICAL UNIVERSITY OF UKRAINE
"IGOR SIKORSKY KYIV POLYTECHNIC INSTITUTE"
APPLIED MATHEMATICS FACULTY

ПРИКЛАДНА МАТЕМАТИКА ТА КОМП'ЮТИНГ ПМК' 2025

Вісімнадцята наукова конференція магістрантів та аспірантів
Київ, 19 – 21 листопада 2025 р.

APPLIED MATHEMATICS AND COMPUTING AMC' 2025

Eighteenth Scientific Conference of Master and Postgraduate Students
Kyiv, November 19-21, 2025

ЗБІРНИК ТЕЗ ДОПОВІДЕЙ PROCEEDINGS

Рекомендовано Вченою радою
факультету прикладної математики



Київ, ПТФ «ПРОСВІТА» 2025

УДК 004.77:620.9

Магістрант Стецюренко І.С., к.т.н, доцент Петрашенко А. В.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

СПОСІБ ОПТИМІЗАЦІЇ ЕНЕРГОЖИВЛЕННЯ ІОТ-СИСТЕМИ МОНІТОРІНГУ КЛІМАТИЧНИХ ПОКАЗНИКІВ

Abstract

I.S. Stetsiurenko, student; A.V. Petrashenko, assoc. prof., PhD

A Method for Power Supply Optimization of an IoT Climate Monitoring System

This paper describes an adaptive, multi-architecture approach for energy optimization in IoT climate monitoring systems. The method integrates hardware sleep modes (Deep Sleep) with a hybrid adaptive algorithm (local and cloud-centric) implemented on an ESP32 platform. It proposes a framework for quantitatively comparing the energy efficiency of Wi-Fi (MQTT, CoAP), BLE, and LoRaWAN architectures for the same monitoring task. This approach provides a clear methodology for selecting the optimal communication strategy to maximize autonomous operation and data actuality in battery-powered IoT devices.

Вступ

Інтернет речей (IoT) більше не є запропонованою ідеєю чи парадигмою оскільки він є глобальним, і, прогнозується, що до 2030 року він матиме понад 29 мільярдів активних підключень. Це головним чином пов'язано з впровадженням автономних систем моніторингу в таких галузях, як навколишнє середовище, точне землеробство та розумні міста. У більшості таких застосувань життєво важливо, щоб сенсорні вузли були розташовані на віддалених місцях, де вони повністю залежать від живлення батареї. Що призводить до проблеми, пов'язаної із загальною енергоефективністю системи.

Сучасний метод передачі даних IoT здійснюється зі статичними або фіксованими інтервалами. Це неефективно, оскільки передбачає втрату енергії якщо параметри залишаються постійними. Водночас він не здатний впоратися і зі швидкими змінами, що підкреслює потребу в інтелектуальних системах, здатних адаптуватися до реальних сценаріїв.

Постановка завдання

Предметом цієї наукової роботи є дослідження ефективності використання енергії та створення оптимізованого методу її використання

в інформаційно-технологічній системі моніторингу кліматичних показників. Цього необхідно досягти шляхом розробки адаптивного алгоритму та порівняння його ефективності залежно від архітектури зв'язку. Для досягнення мети необхідно виконати такі завдання: Виконати системний аналіз апаратних, програмних та комунікаційних елементів, що впливають на енергетику. Розробити архітектуру багаторівневої енергоефективної системи, що поєднує апаратні режими енергозбереження з логікою та хмарними сервісами. Розробити прототипи програмного забезпечення системи для трьох архітектур: прямого зв'язку Wi-Fi (MQTT/HTTP/CoAP), для локального шлюзу (BLE) та для зв'язку на великій відстані (LoRaWAN). Необхідно розробити методологію та позицію для експериментального порівняння показників витрат енергії та релевантності даних. Вивчити очікувані дані та розробити відповідні рекомендації щодо найкращої архітектури, яку слід прийняти залежно від ситуації.

Архітектура апаратно-програмного комплексу

Як мікроконтролер для проектування експериментального стенду було обрано ESP32-WROOM-32 завдяки його потужним процесорним ядрам та підтримці рішень Wi-Fi/BLE, а також співпроцесору ULP та підтримці режиму глибокого сну [1]. Для збору кліматичних параметрів, таких як температура, вологість, тиск та леткі органічні сполуки, було використано Bosch BME680 [2]. Для перевірки альтернативних варіантів проектування архітектури LoRaWAN було використано SX1276.

Архітектура хмарного бекенду: Хмарний бекенд розроблено на платформі Amazon Web Services (AWS). Він використовує AWS IoT Core як брокер MQTT, AWS Lambda (Python) для обробки даних та виконання хмарної логіки, DynamoDB для зберігання даних часових рядів у хмарі протягом тривалішого періоду часу та ElastiCache (Redis) як високошвидкісний кеш для обробки стану в реальному часі та керування алгоритмом v4. Прошивка розроблена на C++ за допомогою Arduino IDE.

Концептуальна модель адаптивного алгоритму

Запропонований метод спирається на адаптивний алгоритм, змодельований за допомогою моделі кінцевого автомата (FSM). На відміну від статичного методу, динамічний метод автоматично обробляє стани машини ("Вимірювання", "Аналіз", "Передача", "Сон"). Найважливіша частина розроблена у двох парадигмах: Локальна адаптація: Логіка прийняття рішень повністю покладена на ESP32. Після пробудження він

порівнює поточні дані з минулими даними, що зберігаються в RTC_DATA_ATTR (пам'ять RTC). Як тільки він виявляє, що варіації не є критичними (нижче порогу стабільності), він негайно пропускає енергоємний процес зв'язку (Wi-Fi) та фіксує довший час сну. Для критичних варіацій він скорочує час сну з передачею даних. Хмарна адаптація. Це простий датчик, який надсилає дані. Дані аналізуються функцією AWS Lambda з повним знанням (в Redis/DynamoDB) історії даних та здатною виконувати складніші моделі. Після визначення оптимального часу сну хмара передає керуюче повідомлення датчику (наприклад, через обернену тему MQTT), щоб пристрій перейшов у режим сну в час, вказаний сервером.

Методика оцінки та теоретичні розрахунки

Для кількісної оцінки ефекту від кожного методу оптимізації програмне забезпечення було розроблено в чотирьох еволюційних версіях (на прикладі Wi-Fi):

- v1: Базова реалізація без оптимізації. Пристрій постійно активний.
- v2: Впровадження режиму Deep Sleep з фіксованим інтервалом пробудження.
- v3: Реалізація локального адаптивного алгоритму (пристрій сам приймає рішення про передачу).
- v4: Реалізація хмарно-керованого алгоритму (пристрій отримує команду про час сну від AWS Lambda).

Для об'єктивного порівняння ефективності архітектур вводиться ключова метрика — середній струм споживання ($I_{\text{сер}}$). Він розраховується за повний робочий цикл (активність + сон) за формулою:

$$I_{\text{сер}} = \frac{I_{\text{акт}} * t_{\text{акт}} + I_{\text{сну}} * t_{\text{сну}}}{t_{\text{акт}} + t_{\text{сну}}}$$

де $I_{\text{акт}}$ та $t_{\text{акт}}$ - струм і тривалість активної фази, а $I_{\text{сну}}$ та $t_{\text{сну}}$ - струм і тривалість фази сну.

Теоретичний розрахунок (v1 проти v2), що базується на даних документації ESP32 ($I_{\text{акт}} \approx 190 \text{ мА}$, $I_{\text{сну}} \approx 0.02 \text{ мА}$) [1] та проектних параметрах ($t_{\text{акт}} \approx 3 \text{ с}$, $t_{\text{сну}} \approx 30 \text{ с}$), прогнозує зниження середнього струму з 190 мА (v1) до ~17.29 мА (v2), що становить економію ~91%.

Для підтвердження цих розрахунків було проведено попередні експериментальні вимірювання середнього струму споживання для трьох протоколів, протестованих в архітектурі Wi-Fi (див. Табл. 1).

Аналіз експериментальних даних дозволяє зробити кілька ключових висновків:

1. Ефективність Deep Sleep (v2): Перехід від базового режиму v1 (50-68 мА) до режиму v2 (Deep Sleep) дає колосальне зниження енергоспоживання на 73-84%. Експериментальні значення (10-14 мА) виявилися навіть кращими за теоретичний розрахунок (~17 мА), що пов'язано з меншим реальним часом $T_{\text{акт}}$ (1.4-1.5 с) порівняно з теоретичним (3 с).

Таблиця 1. Експериментальне порівняння енергоспоживання протоколів у мережі Wi-Fi

Протокол	Режим	$T_{\text{акт}}$ (с)	$I_{\text{сер}}$ (мА)	Зниження відн. v1
HTTP	v1	1,4	50,89	0%
	v2	1,47	13,73	~73%
	v3 (30с)	1,5	13,33	~74%
	v4 (30с)	3,37	16,35	~67,9%
MQTT	v1	4,3	55,79	0%
	v2	1,43	14,55	~74%
	v3 (30с)	4,2	16,14	~71%
	v4 (30с)	7,14	20	~64%
CoAP	v1	3,13	67,44	0%
	v2	1,53	10,78	~84%
	v3 (30с)	1,47	10,46	~84,5%
	v4 (30с)	1,36	10,43	~84,6%

2. Вплив адаптивних алгоритмів (v3/v4): Результати виявилися неоднозначними. Для HTTP та MQTT, режими v3 та v4 показали гірші результати, ніж простий v2. Причиною є значне зростання $T_{\text{акт}}$ (активного часу), наприклад, до 7,14 с для MQTT v4, що, ймовірно, пов'язано з часом очікування відповіді від хмари та складністю локальних обчислень. Цей додатковий активний час "з'їв" усю економію від довшого сну.

3. Найкращий Wi-Fi протокол: CoAP продемонстрував найкращу ефективність у всіх режимах, досягнувши найнижчого споживання (~10.4 мА). Важливо, що для CoAP активний час $T_{\text{акт}}$ залишився стабільно низьким (1.3-1.5 с) навіть у режимах v3/v4, що вказує на його високу ефективність для IoT-задач [3].

Прогнози для BLE та LoRaWAN

Версії v1-v3 були також розроблені для систем BLE та LoRaWAN (де модель v4 є недоцільною). Очікується, що ці архітектури покажуть значно кращі результати. Навіть найкращий експериментальний показник CoAP

(~10.4 мА) є на порядки вищим за теоретичне споживання BLE/LoRaWAN, оскільки в них не використовується енергозатратний Wi-Fi. Прогнозується, що $I_{сер}$ для BLE та LoRaWAN буде в діапазоні десятків мікроампер (мкА), що збільшить термін автономної роботи до кількох років.

Висновки

У роботі пропонується та пояснюється гібридний багаторівневий метод оптимізації енергоспоживання автономними системами моніторингу Інтернету речей.

Наукова частина зосереджена на розробці гібридної моделі керування з використанням як локальної логіки, так і хмарної логіки. Що ще важливіше, вона забезпечує основу для безпосереднього порівняння трьох абсолютно різних архітектур зв'язку щодо виконання одного й того ж завдання моніторингу.

Цінність для практиків полягає в розробці методології та рекомендацій, перевірених за допомогою експериментів, для розробників Інтернету речей. Результати гарантуватимуть, що за допомогою інтелектуальних циклів пробудження та сну можна досягти балансу між актуальністю даних та автономною функціональністю. Запропонований метод можна негайно застосувати для розробки стійких систем моніторингу для точного землеробства або екології.

Література

1. Espressif Systems. (2024). ESP32 Series Datasheet. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
2. Bosch Sensortec. (2023). BST-BME680-DS001: BME680 Low power gas, pressure, temperature & humidity sensor. <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>
3. Jim, L. T., Sandre, F. S., & Al-Anbuky, A. (2019). Performance Evaluation of CoAP and MQTT Protocols over Unreliable Networks. *Proceedings*, 31(1), 49. <https://doi.org/10.3390/proceedings2019031049>

ЗМІСТ

1.	Андрусенко О.М., Хавронюк Б.А. КОНЦЕПТУАЛЬНА МОДЕЛЬ ТА СИСТЕМА ГЕОПОЗИЦІЮВАННЯ НА ОСНОВІ АНСАМБЛЕВИХ НЕЙРОННИХ МЕРЕЖ З УРАХУВАННЯМ ЧАСОВОЇ ПОСЛІДОВНОСТІ КАДРІВ	4
2.	Сирота С. В., Байда О. Є. МАТЕМАТИЧНЕ ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ОПТИМІЗАЦІЇ ІНВЕСТИЦІЙНОГО ПОРТФЕЛЯ	9
3.	Buslaiev V.O. EVALUATING REDDIT AS A PROXY FOR PUBLIC INTEREST IN VIDEO GAMES	14
4.	Жук І. С., Громова В.В., Жмурко К.Р. КОНЦЕПТУАЛЬНА МОДЕЛЬ ТА КОМП'ЮТЕРНЕ МОДЕЛЮВАННЯ ДІАЛЕКТІВ УКРАЇНСЬКОЇ МОВИ	19
5.	Жук І. С., Громова В.В., Ковальчук В. В. КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ ВИЯВЛЕННЯ АНОМАЛІЙ У ЗОБРАЖЕННЯХ СКЛЯНИХ ВИРОБІВ З ВИКОРИСТАННЯМ ГЛИБИННОГО НАВЧАННЯ	24
6.	Жук І. С., Громова В.В., Романецький М. С. МЕТОД ДЛЯ ВИДІЛЕННЯ ГОЛОСУ ЦІЛЬОВОГО МОВЦЯ ЗА АУДІОЗАПИСАМИ	30
7.	Казновський А. Т., Сирота С. В. ІНТЕГРАЦІЯ МЕТОДІВ КОМП'ЮТЕРНОГО ЗОРУ З ІНЕРЦІЙНИМИ СИСТЕМАМИ НАВІГАЦІЇ ДЛЯ ПІДВИЩЕННЯ АВТОНОМНОСТІ БПЛА	35
8.	Катеринич Д. В., Ориняк І. В. ЗАЛЕЖНИЙ ВІД ЧАСУ ЕЛЕМЕНТ ДЛЯ ДИНАМІЧНОЇ ЗАДАЧІ ОСЬОВОЇ ВЗАЄМОДІЇ ТРУБИ І ВОДИ	40
9.	Третинник В.В., Ковріжкін О.С. КОНЦЕПТУАЛЬНА МОДЕЛЬ ГІБРИДНОЇ СИСТЕМИ ДЛЯ ДЕТЕКЦІЇ ТА КЛАСИФІКАЦІЇ КОНТУРІВ НА ОСНОВІ ТРАДИЦІЙНИХ МЕТОДІВ ТА ГЛИБОКОГО НАВЧАННЯ	45
10.	Козік С. В., Ориняк І. В., Тавров Д. Ю. КОРОТАЦІЙНІ БАЛКОВИХ СПЛАЙНІВ, ЯК ІНСТРУМЕНТ ПРОЄКТУВАННЯ ДОРІГ, ЩО УТВОРЕНІ ДІЛЯНКАМИ КЛОТОІД	50

11.	Кривуляк І.І., Маркович Б.М., Корендій В.М. ВИКОРИСТАННЯ ОПТИЧНИХ СИСТЕМ ДЛЯ МУЛЬТИМОДАЛЬНОГО НАВЧАННЯ СИСТЕМ КЕРУВАННЯ БІОНІЧНИМИ ПРОТЕЗАМИ НА ОСНОВІ EMG ТА LSTM-МОДЕЛЕЙ	55
12.	Кришук С.А., Піць В.В. МАТЕМАТИЧНА МОДЕЛЬ БІОТЕХНОЛОГІЧНОГО ВИРОБНИЦТВА ПРОБІОТИЧНОГО ПРЕПАРАТУ	59
13.	Ліскін В.О., Пономарьов О.А. ПІДВИЩЕННЯ ЯКОСТІ РЕНТГЕНІВСЬКИХ ЗНІМКІВ НА ОСНОВІ НЕЙРОННИХ МЕРЕЖ	64
14.	Маслянюк П. П, Шевцов М. В. КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ КОМП'ЮТЕРНОГО МОДЕЛЮВАННЯ КЛЮЧОВИХ ПОКАЗНИКІВ ЕФЕКТИВНОСТІ ІННОВАЦІЙНИХ КОМПАНІЙ	69
15.	Сахаров С. Ю. АДАПТАЦІЯ СІАМСЬКИХ МЕРЕЖ ДО ФОТОГРАММЕТРИЧНИХ ХМАР ТОЧОК ДЛЯ ВИЯВЛЕННЯ ЗМІН В 3D СЕРЕДОВИЩІ	76
16.	Сінельник Г. С., Орняк І. В. РЕАЛІЗАЦІЯ СПЕКТРАЛЬНО КРУТИЛЬНО-ЗГИНАЛЬНОГО ЕЛЕМЕНТУ В МЕТОДІ УЗГОДЖЕНИХ ПЕРЕРІЗІВ ДЛЯ АНАЛІЗУ КОЛИВАНЬ ТОНКОСТІННИХ ПЛАСТИН	81
17.	Соловійов С.О., Пархоменко М.С., Рябоконь З.А. МАТЕМАТИЧНА МОДЕЛЬ ПРОГНОЗУВАННЯ ПОШИРЕННЯ ТА ЕФЕКТИВНОСТІ ЛІКУВАННЯ ВІЛ- ІНФЕКЦІЙ	86
18.	Третинник В.В., Гаврилюк Н.Я. КОНЦЕПТУАЛЬНА МОДЕЛЬ ЗАБЕЗПЕЧЕННЯ КОНФІДЕНЦІЙНОСТІ МЕДИЧНИХ ДАНИХ З ВИКОРИСТАННЯМ ПРОТОКОЛІВ НУЛЬОВОГО РОЗГОЛОШЕННЯ	91
19.	Третинник В. В., Рижкова Д. О. КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ ДІАГНОСТИКИ ХВОРОБ СИСТЕМИ КРОВООБІГУ НА ОСНОВІ МЕТОДІВ МАШИННОГО НАВЧАННЯ	96
20.	Третинник В. В., Харчук О.О. КОНЦЕПТУАЛЬНА МОДЕЛЬ РЕКОМЕНДАЦІЙНОЇ СИСТЕМИ ДЛЯ ПРОДАВЦІВ НА МАРКЕТПЛЕЙСІ	101

21.	Третинник В. В., Явтуховський В.С. ПОРІВНЯННЯ МЕТОДІВ ПОЯСНЮВАЛЬНОГО ШТУЧНОГО ІНТЕЛЕКТУ З ТОЧКИ ЗОРУ ЗМАГАЛЬНИХ АТАК НА ЗОБРАЖЕННЯ	106
22.	Маркович Б. М., Афтаназів І.С., Корендій В.М., Фостяк В.Т. ПРОГНОЗУВАННЯ ТРАЄКТОРІЇ ВОРОЖОГО БПЛА НА ОСНОВІ ЕКСТРАПОЛЯЦІЇ ПРОСТОРОВИХ ВИМІРЮВАНЬ	111
23.	Чертов О. Р., Кубрак В. В. ПІДВИЩЕННЯ РЕЛЕВАНТНОСТІ АСОЦІАТИВНИХ ПРАВИЛ У МОДЕЛІ РИНКОВОГО КОШИКА ЗАСОБАМИ КЛАСТЕРНОГО АНАЛІЗУ	116
24.	Боярінова Ю.Є., Віннікова У.В. МЕТОДИ ПЕРЕДАЧІ ДАНИХ У ПРИСТРОЯХ МОНІТОРИНГУ НАВКОЛИШНЬОГО СЕРЕДОВИЩА	121
25.	Боярінова Ю.Є., Однораз А.О. ПОРІВНЯЛЬНИЙ АНАЛІЗ ІНСТРУМЕНТІВ СТВОРЕННЯ ТА ЗБІРКИ ЮНІТ-ТЕСТІВ КОДУ НАПИСАНОГО МОВОЮ C/C++	127
26.	Юрович І.В.; Зайцев В.Г. СПОСІБ ВИЗНАЧЕННЯ ЧАСУ ПОЯВИ І КІНЦЕВОГО ТЕРМІНУ ВИКОНАННЯ ЗАДАЧ У КОМП'ЮТЕРНИХ СИСТЕМАХ РЕАЛЬНОГО ЧАСУ	132
27.	Пасічник М.Ю.; Зайцев В.Г. ПРОГНОЗУВАННЯ НАВАНТАЖЕННЯ ДЛЯ ДИСПЕТЧЕРИЗАЦІЇ ЗАВДАНЬ У СИСТЕМАХ РЕАЛЬНОГО ЧАСУ	137
28.	Болгов І. М.; Клятченко Я. М. СПЕЦІАЛІЗОВАНІ КОМП'ЮТЕРНІ ЗАСОБИ ДЛЯ ЗБОРУ ТА АНАЛІЗУ ЕКСПЛУАТАЦІЙНИХ ХАРАКТЕРИСТИК КОЛІСНОЇ ТЕХНІКИ	141
29.	Клятченко Я.М., Кальницький О.В. АНАЛІЗ ПАРАМЕТРІВ СТЕРЕОЗОРУ ТА ЇХ ВПЛИВУ НА ТОЧНІСТЬ І РОБОЧУ ДИСТАНЦІЮ	146
30.	Клятченко Я. М., Катюха Б.М МЕТОДИ, СТРУКТУРИ ТА АЛГОРИТМИ ПІДВИЩЕННЯ ЕНЕРГОЕФЕКТИВНОСТІ БЕЗДРОТОВИХ ІОТ МЕРЕЖ	151
31.	Клятченко Я. М., Кубай О.Ф. СПАЙКОВІ НЕЙРОННІ МЕРЕЖІ ДЛЯ ЕФЕКТИВНОЇ РЕАЛІЗАЦІЇ АЛГОРИТМІВ МАШИННОГО НАВЧАННЯ НА ПЛІС	156

32.	Klyatchenko Ya., Samoilenko D. RAY-TRACED SHADOW ATLAS FOR MULTIPLE DYNAMIC LIGHT SOURCES INTEGRATED WITH RASTERIZATION RENDERING	161
33.	Холодар А. А., Коляда К. В. АРХІТЕКТУРА ГЕОПРОСТОРОВОЇ ІНФОРМАЦІЙНОЇ СИСТЕМИ РЕЄСТРУВАННЯ ПРОЦЕСІВ РОЗМІНУВАННЯ	165
34.	Черичка М. А., Коляда К. В. ВИКОРИСТАННЯ ДИНАМІЧНИХ ТАБЛИЦЬ СПЕЦИФІКАЦІЙ ДЛЯ РЕКОНСТРУКЦІЇ БЛОКІВ ДАНИХ	170
35.	Боярінова Ю.Є., Липовецький Д.Є. СПОСІБ КЕРУВАННЯ БПЛА З ВИКОРИСТАННЯМ ГЕНЕТИЧНИХ АЛГОРИТМІВ	173
36.	Мартінова О.П., Костюков С. В. СИСТЕМА ТЕСТУВАННЯ ЕФЕКТИВНОСТІ АЛГОРИТМІВ БАЛАНСУВАННЯ НАВАНТАЖЕННЯ В SDN МЕРЕЖАХ	179
37.	Мартінова О.П., Литвин М.І. ІНФОРМАЦІЙНА СИСТЕМА МОНІТОРИНГУ ТА ПРОГНОЗУВАННЯ ТЕМПЕРАТУРИ СЕРЕДОВИЩА	184
38.	Мартінова О. П., Рябенко Б. Ю. МОДИФІКАЦІЯ АЛГОРИТМУ ДЕЙКСТРИ ДЛЯ ПІДВИЩЕННЯ ЕФЕКТИВНОСТІ БАГАТОШЛЯХОВОЇ ДИНАМІЧНОЇ МАРШРУТИЗАЦІЇ В КОМП'ЮТЕРНИХ МЕРЕЖАХ	190
39.	Калитенко М.П., Марченко О.І. АНАЛІЗ МЕТОДІВ І ТЕХНОЛОГІЙ ОБРОБКИ ЗОБРАЖЕНЬ	196
40.	Марченко О.І., Семенов М.С. АНАЛІЗ КЛЮЧІВ ШИФРУВАННЯ ЗГЕНЕРОВАНИХ НА ОСНОВІ НОРМАЛЬНОГО РОЗПОДІЛУ	202
41.	Жовтанюк М. В., Молчанов О. А. СПОСІБ ПОБУДОВИ РОЗПОДІЛЕНОГО ФІЛЬТРА БЛУМА З КОНСИСТЕНТНИМ ХЕШУВАННЯМ	207
42.	Шкільнюк В. О., Молчанов О. А. КОМП'ЮТЕРНА СИСТЕМА АСИНХРОННОГО ФІЛЬТРУВАННЯ МЕРЕЖЕВИХ ПАКЕТІВ	212
43.	Морозов К.В., Лунгов О.В. МЕТОД ВІДСТЕЖЕННЯ ПОГЛЯДУ КОРИСТУВАЧА ДЛЯ ВЗАЄМОДІЇ З КОМП'ЮТЕРОМ	216

44.	Морозов К.В, Пшеничний С.В. МЕТОД ІНТЕЛЕКТУАЛЬНОГО РОЗПОДІЛУ НАВАНТАЖЕННЯ У ВЕБ-СЕРВІСАХ НА ОСНОВІ ПРОГНОЗУВАННЯ ТРАФІКУ	221
45.	Павловський В.І., Вінницький В. О. СПОСОБИ ТА ЗАСОБИ ПІДВИЩЕННЯ ЕФЕКТИВНОСТІ ЗАХИСТУ ВІД ФІШІНГ-АТАК У СОЦІАЛЬНИХ МЕРЕЖАХ	227
46.	Павловський В.І., Кравченко Л.О. ПОРТАТИВНА КОМП'ЮТЕРНА СИСТЕМА КОНТРОЛЮ ЯКОСТІ ПОВІТРЯ В ТИМЧАСОВО ОБЛАДНАНИХ ПРИМІЩЕННЯХ З ВИСОКИМИ ВИМОГАМИ ДО СТАБІЛЬНОСТІ МІКРОКЛІМАТУ	232
47.	Петрашенко А. В., Костюченко А. В. МЕТОД ГЕНЕРАЦІЇ ОПИСУ ПРОГРАМНОГО КОДУ З ВИКОРИСТАННЯМ МОДЕЛІ ШТУЧНОГО ІНТЕЛЕКТУ	237
48.	Петрашенко А. В., Котлярський А. О. ПІДХОДИ ДО ЗАБЕЗПЕЧЕННЯ НАДІЙНОСТІ В ХМАРНИХ ПЛАТФОРМАХ AWS, AZURE І GOOGLE CLOUD	241
49.	Стецюренко І.С., Петрашенко А. В. СПОСІБ ОПТИМІЗАЦІЇ ЕНЕРГОЖИВЛЕННЯ ІОТ-СИСТЕМИ МОНІТОРІНГУ КЛІМАТИЧНИХ ПОКАЗНИКІВ	246
50.	Безруков І.О., Потапова К.Р., Бойко Н. В. АНАЛІЗ ЧУТЛИВОСТІ DRL-ПОЛІТИК ДО СТОХАСТИЧНИХ ЗБУРЕНЬ СЕРЕДОВИЩА	251
51.	Шевченко І.І., Потапова К.Р. МОДИФІКОВАНИЙ ДИФУЗІЙНИЙ АЛГОРИТМ ПІДВИЩЕННЯ ТОЧНОСТІ СИСТЕМИ РОЗПІЗНАВАННЯ НЕЧІТКОГО МОВЛЕННЯ	256
52.	Романкевич В.О., Єрмоленко І.А., Поліщук О. П. МЕТОД ПОБУДОВИ GL-МОДЕЛЕЙ ДЛЯ CONSECUTIVE-K- WITHIN-M-OUT-OF-N СИСТЕМ	263
53.	Романкевич В.О., Степаненко П.О. ІНФОРМАЦІЙНО-АНАЛІТИЧНА СИСТЕМА ДЛЯ ЗБОРУ ТА АНАЛІЗУ МЕДИЧНИХ ДАНИХ ПАЦІЄНТІВ ІЗ ВИКОРИСТАННЯМ ШТРИХОВОГО КОДУВАННЯ	270
54.	Романкевич О.М., Бужацький В.М. АНАЛІЗ ВПЛИВУ ПАРАМЕТРІВ КОНФІГУРАЦІЇ DUPLEX- SPONGE ГЕНЕРАТОРА З ПЕРЕСТАНОВКОЮ ASCON НА ЙОГО КРИПТОГРАФІЧНУ СТІЙКІСТЬ	275

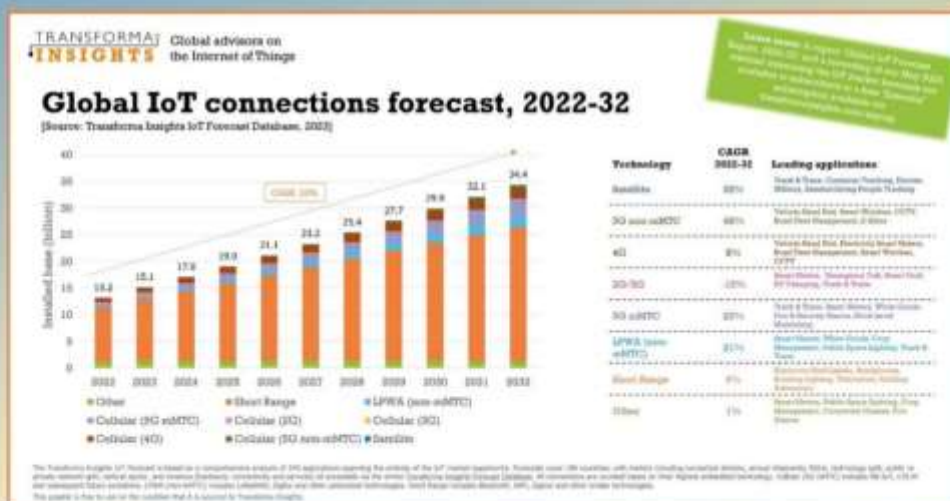
55.	Романкевич О.М., Орел І.В. СИСТЕМА МОНІТОРИНГУ ТА УПРАВЛІННЯ ЕНЕРГОСПОЖИВАННЯМ НА ОСНОВІ ТЕХНОЛОГІЙ ІНТЕРНЕТУ РЕЧЕЙ	280
56.	Сергієнко П.А., Філіпенко Д.О. СПОСІБ РОЗПІЗНАВАННЯ ПОВЕДІНКОВИХ ПАТЕРНІВ НА ОСНОВІ ПОПЕРЕДНЬО ВІДОМИХ ОБРАЗІВ	285
57.	Тарасенко В.П., Тарасенко-Клятченко О.В., Льчук О.О. ПОСТКВАНТОВІ ПІДХОДИ ДО ЗАХИСТУ ТУНЕЛЬНИХ ПРОТОКОЛІВ МЕРЕЖЕВОГО РІВНЯ	290
58.	Тарасенко В.П., Тарасенко-Клятченко О.В., Устименко І.В. ПОБУДОВА МАСШТАБОВАНОЇ ТА ВІДМОВОСТІЙКОЇ СЕРВЕРНОЇ ЧАСТИНИ СИСТЕМИ ЗБОРУ ЗВІТІВ	295
59.	Горба Д.О., Терейковський І. А. ПОСТАНОВКА ЗАДАЧІ РОЗРОБКИ НЕЙРОМЕРЕЖЕВИХ ЗАСОБІВ ДЕТЕКТУВАННЯ ОБ'ЄКТІВ У ВІДЕОПОТОЦІ	299
60.	Терейковський І. А., Коротич О. С. ОПІМІЗАЦІЯ НЕЙРОМЕРЕЖЕВОЇ МОДЕЛІ ВИЗНАЧИННЯ ОСНОВНОЇ ЧАСТОТИ ГОЛОСУ ЛЮДИНИ	305
61.	Терейковський І.А., Роговий Д.С. ПОРІВНЯЛЬНИЙ АНАЛІЗ МЕТОДІВ ПОКРАЩЕННЯ РОЗДІЛЬНОЇ ЗДАТНОСТІ СУПУТНИКОВИХ ЗНІМКІВ	311
62.	Тесленко О. К., Оніщук А. О. СИСТЕМА АВТОМАТИЗАЦІЇ РОЗГОТАННЯ ІШІ-ДОДАТКІВ У ХМАРІ	316
63.	Доліч А.І., Щербина О.А. МУЛЬТИАГЕНТНИЙ МЕТОД СТРУКТУРНО-ОБІЗНАНОГО ВИЯВЛЕННЯ ЛОГІЧНИХ СУПЕРЕЧНОСТЕЙ У НАУКОВИХ ТЕКСТАХ	322
64.	Наливайчук М.В., Шведов Д.Є. ДОСЛІДЖЕННЯ ЕФЕКТИВНОСТІ BLAZOR У РОЗРОБЦІ ІНТЕРАКТИВНИХ ВЕБ-ДОДАТКІВ	326
65.	Наливайчук М.В., Єгоров М. А. ЗАСОБИ АДАПТИВНОГО ШУМОПОГЛИНЕННЯ АУДІОСИГНАЛІВ НА БАЗІ ЗГОРТКОВОЇ НЕЙРОННОЇ МЕРЕЖІ	330
66.	Радченко К.О., Статечний С.В., Крайносвіт А.А. АДАПТИВНИЙ МЕТОД ПОКРАЩЕННЯ НАДІЙНОСТІ ТА ЕНЕРГОЕФЕКТИВНОСТІ РАДІОПРОТОКОЛУ ДЛЯ LOW POWER IOT СИСТЕМ	334

67.	Радченко К.О., Черненко А.О., Крайносвіт А.А. СПОСІБ РОЗПІЗНАВАННЯ ЗОБРАЖЕНЬ ЗА ДОПОМОГОЮ ОПТИМІЗОВАНОГО АЛГОРИТМУ ВИБОРУ М-КРАТНИХ ВЕКТОРІВ	340
68.	Беліцький О.С., Юсин Я.О. МЕТОД АДАПТАЦІЇ АРХІТЕКТУРНОГО ШАБЛОНУ METAMORPHIC TESTING-AS-A-SERVICE ДЛЯ МОВИ ПРОГРАМУВАННЯ JAVA	345
69.	Бурчак П.В., Олещенко Л.М. METHODS OF INTELLIGENT WEB APPLICATION PERFORMANCE ANALYSIS BASED ON ADAPTIVE MACHINE LEARNING MODELS	350
70.	Люшенко Л.А., Васильковський К.В. МЕТОД ПРОГРАМНОГО ПРОГНОЗУВАННЯ МАКРОЕКОНОМІЧНИХ ПОКАЗНИКІВ НА ОСНОВІ КОРЕЛЯЦІЙНОГО АНАЛІЗУ	354
71.	Онай М.В., Гулько Д. Т. КОМБІНОВАНИЙ АДАПТИВНИЙ ρ -МЕТОД ПОЛЛАРДА ДЛЯ ВИРІШЕННЯ ЗАДАЧІ ДИСКРЕТНОГО ЛОГАРИФМУВАННЯ	359
72.	Нікольський С.С., Гуцуляк Н.А. ОРГАНІЗАЦІЯ ПАРАЛЕЛЬНОГО ОБЧИСЛЕННЯ ЕКСПОНЕНТИ НА ПОЛЯХ ГАЛУА ДЛЯ КРИПТОГРАФІЧНИХ ЗАСТОСУВАНЬ	365
73.	Онай М.В., Дзенік Д.М. МЕТОД ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ГІБРИДНОГО ШИФРУВАННЯ ДАНИХ З ВИКОРИСТАННЯМ БІОМЕТРИЧНОГО КЛЮЧА	372
74.	Коваленко М. В., Юрчишин В. Я. МЕТОД АНАЛІЗУ ТЕКСТОВИХ ОГолошень для ПРОГНОЗУВАННЯ ПОПУЛЯРНИХ ВАКАНСІЙ	377
75.	Конюшняк О.В., Юсин Я.О. КОМБІНОВАНИЙ МЕТОД МУТАЦІЙНОГО ТЕСТУВАННЯ ІЗ ЗАСТОСУВАННЯМ ГЕНЕРАТИВНОГО ШТУЧНОГО ІНТЕЛЕКТУ	381
76.	Дичка І. А., Кучмій О. Е., Дрозденко Л. В. ВИЗНАЧЕННЯ ТВІРНИХ МНОГОЧЛЕНІВ ДВІЙКОВИХ КОРЕКТУВАЛЬНИХ КОДІВ БЧХ	385

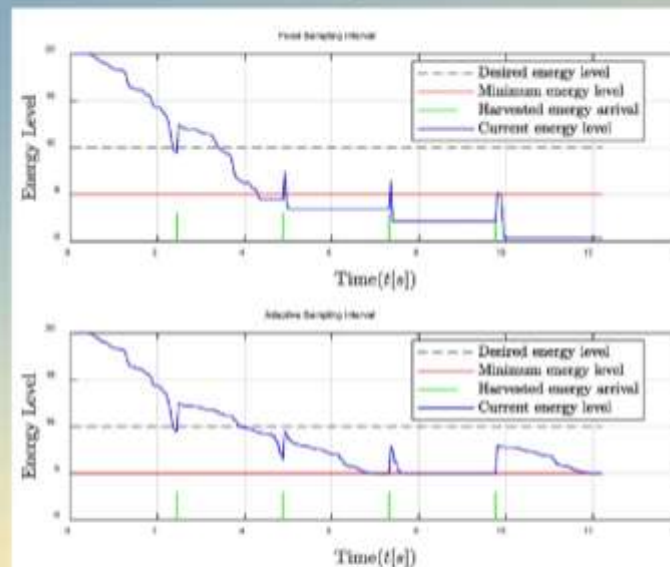
77.	Boiarinova Yu.E., Lu Lei THE DESIGN OF NETWORK SECURITY LOG ANALYSIS SYSTEM BASED ON ELK	391
78.	Нешадим О.М., Седухіна А.Д. СПОСІБ ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ СИМУЛЯЦІЇ РІДИН В РЕАЛЬНОМУ ЧАСІ	396
79.	Novak D.S., Sukpanya Chaleunsuk AN EMPIRICAL STUDY OF PYTHON FOR SOFT REAL-TIME IOT STREAM PROCESSING: INVESTIGATING LATENCY, MQTT QOS/TLS, AND BATCHING AT THE EDGE	401
80.	Олещенко Л.М., Антоненко І.В. МЕТОД ГЛИБОКОГО НАВЧАННЯ ДЛЯ АНАЛІЗУ ТОНАЛЬНОСТІ ТА УЗГОДЖЕНОСТІ ВІДГУКІВ КЛІЄНТІВ ЕЛЕКТРОННОЇ КОМЕРЦІЇ	407
81.	Oleshchenko L.M., Ziborov D.Y. ADAPTIVE DATA COMPRESSION METHOD AND SOFTWARE FOR OPTIMIZING DATA TRANSMISSION	412
82.	Oleshchenko L.M., Tan Huibin SOFTWARE TESTING METHOD USING MACHINE LEARNING ALGORITHMS	416
83.	Oleshchenko L.M., Borodenko S.O. METHOD AND SOFTWARE FOR DISTRIBUTED ANALYSIS AND FORECASTING OF ENTERPRISE FINANCIAL INDICATORS	421
84.	Onai Mykola, Tiangang Chang MODIFIED METHOD MULTIPLICATION OF POINTS OF ELLIPTICAL CURVE OVER $GF(2^m)$ USING FAST FOURIER TRANSFORM	425
85.	Згуровський Я.Ю., Онаї М.В. МЕТОДОЛОГІЯ ТЕСТУВАННЯ ПОСТКВАНТОВИХ АЛГОРИТМІВ: ПОРІВНЯЛЬНИЙ АНАЛІЗ РКС ТА PQС ТА ПРОЄКТУВАННЯ ФРЕЙМВОРКУ	431
86.	Пивовар О.О., Нешадим О.М. МЕТОД ПРОГНОЗУВАННЯ ЧАСОВИХ РЯДІВ З НЕПОВНИМИ ДАНИМИ З ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ	437
87.	Kostiantyn Radchenko, Wei Shenlai ADAPTIVE ALGORITHMS FOR PERSONALIZED DISTANCE LEARNING MANAGEMENT SYSTEMS	442

88.	Саяпіна І.О., Шевченко Г.О. МЕТОД АВТОМАТИЗАЦІЇ УПРАВЛІННЯ ДАНИМИ В ГЕТЕРОГЕННИХ БАЗАХ	447
89.	Балащенко О.В., Сулема Є.С. ВИКОРИСТАННЯ НЕЧІТКОЇ ЛОГІКИ ДЛЯ АНАЛІЗУ ДАНИХ СЕНСОРІВ ЗА ДОПОМОГОЮ МОВИ ПРОГРАМУВАННЯ ASAMPL	451
90.	Хіцко Я.В., Калашников-Травін В.В. МОДИФІКОВАНИЙ МЕТОД ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ КОНТЕНТО-ЗАЛЕЖНОЇ ФРАГМЕНТАЦІЇ ДАНИХ	455
91.	Oksana S. Shkurat, Peng Jiahao DIGITAL QUALITY ENHANCEMENT METHOD FOR LOW- LIGHT IMAGES	460
92.	Шкурат О.С., Пецеля А. А. НЕЙРОМЕРЕЖЕВИЙ СПОСІБ ПОШУКУ ЗОБРАЖЕНЬ ЗА ОЗНАКАМИ	464
93.	Oksana S. Shkurat, Maksym V. Slobodzian SOFTWARE METHOD FOR IMAGE SEGMENTATION UNDER VARIABLE LIGHTING CONDITIONS	468
94.	Дичка І.А.; Потапова К.Р.; Мелюх В.В. ФОРМАЛІЗАЦІЯ БАГАТОАСПЕКТНОГО КОНТЕКСТУ ДЛЯ АДАПТИВНОГО СЕМАНТИЧНОГО ПОШУКУ	472
95.	Заболотня Т.М., Дичка А.І., Довженко Р.О. СПОСІБ ОЦІНЮВАННЯ ПОТЕНЦІЙНОЇ ПОПУЛЯРНОСТІ ТЕКСТОВИХ ПУБЛІКАЦІЙ В СОЦІАЛЬНИХ МЕРЕЖАХ	476
96.	Северін А. І., Мостовий С.С. МЕТОД ІНТЕЛЕКТУАЛЬНОГО АНАЛІЗУ ТЕКСТУ НА ВИЯВЛЕННЯ ПРОПАГАНДИ	480
97.	Саяпіна І.О., Павлошин М.Ю. МЕТОД ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ДЛЯ ОПТИМІЗАЦІЇ ПРОДУКТИВНОСТІ БАЗ ДАНИХ	484
98.	Oksana S. Shkurat, Ivan V. Fedorchuk SPATIO-TEMPORAL FORECASTING OF DIGITAL IMAGES USING CONDITIONAL DIFFUSION MODEL	489
99.	Дичка А. І., Хоменко М. В. МЕТОД ПРОЄКТУВАННЯ МОНОЛІТНИХ ЗАСТОСУНКІВ РЕАЛЬНОГО ЧАСУ З ВИКОРИСТАННЯМ АКТОРНОЇ МОДЕЛІ ТА ПРИНЦИПІВ ЧИСТОЇ АРХІТЕКТУРИ	494

100.	Хішко Я.В., Черній Г.А. СПОСІБ ТА ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ АВТОМАТИЗАЦІЇ РУТИННИХ ЗАВДАНЬ ПРОГРАМІСТА	499
101.	Shkurat O. S., Nikolenko H. Ya. SOFTWARE METHOD FOR MEDICAL IMAGE RECOGNITION OF HUMAN INJURIES	504
102.	Oksana S. Shkurat, Artem V. Petselia SOFTWARE BASED METHOD FOR SEGMENTATION OF FLOOR AND CEILING REGIONS IN INDOOR IMAGES	508
103.	Шкурат О.С., Сметана М.О. АЛГОРИТМ АДАПТИВНОГО ПОКРАЩЕННЯ ЯКОСТІ ЗОБРАЖЕНЬ AIEA	512
104.	Oksana Tarasenko-Kliatchenko, Volodymyr Tarasenko, Wei Dong MOBILE APPLICATION FOR ADAPTIVE LEARNING USING ARTIFICIAL INTELLIGENCE	516
105.	Атаманюк О.В., Сулема Є.С. АНАЛІЗ МЕТОДІВ ВИРІШЕННЯ КОНФЛІКТІВ У МУЛЬТИАГЕНТНІЙ МОДЕЛІ ЦИФРОВОГО ДВІЙНИКА СКЛАДНОЇ СИСТЕМИ	521



Аналіз проблеми



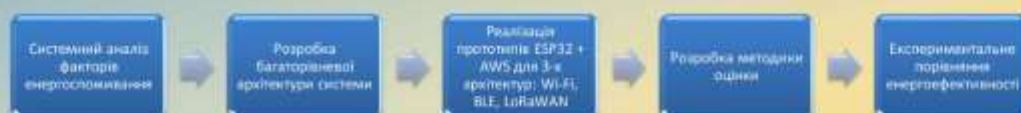
Мета та об'єкт дослідження

Мета: Розробка гібридного адаптивного методу керування для підвищення автономності.

Об'єкт: Процес енергоспоживання.

Предмет: Адаптивні алгоритми керування режимами роботи.

Задачі дослідження



Обґрунтування вибору апаратної платформи



ESP32 DevKit v1

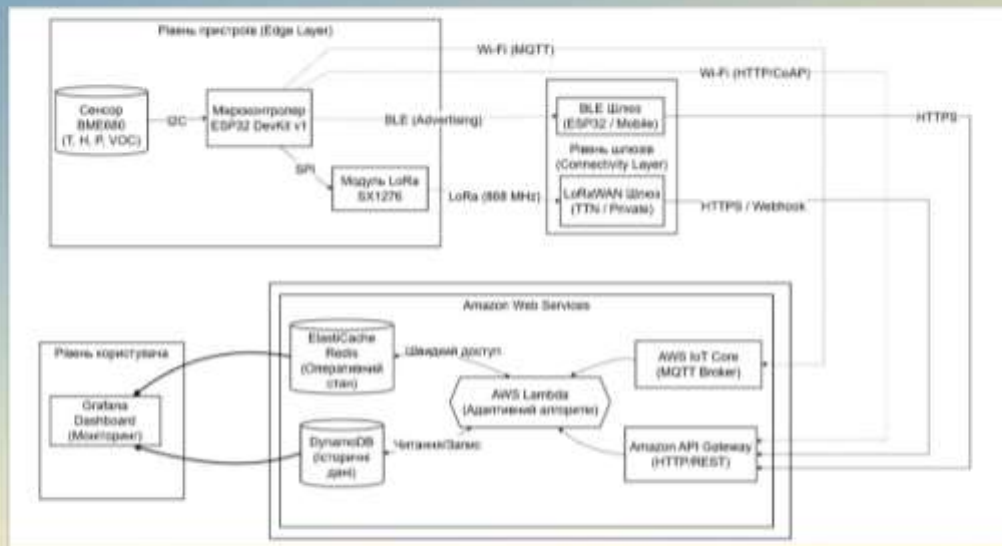


BME 680



SX1276

Загальна архітектура системи



Математична модель енергоспоживання

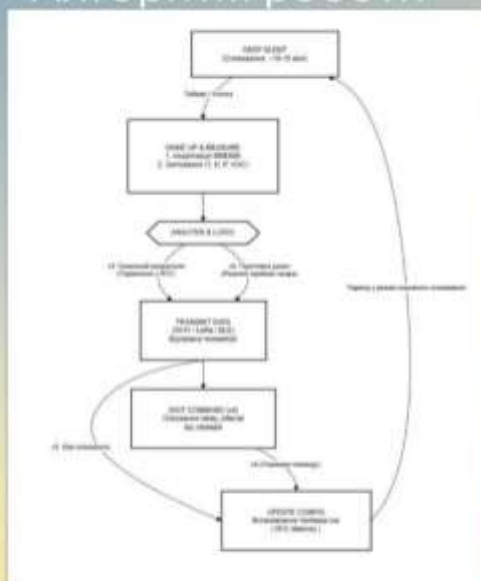
$$I_{avg} = \frac{I_{active} \cdot T_{active} + I_{sleep} \cdot T_{sleep}}{T_{active} + T_{sleep}}$$

де I_{active} та T_{active} - струм і тривалість активної фази, а I_{sleep} та T_{sleep} - струм і тривалість фази сну.

$$T_{autonomy} = \frac{C_{battery} \cdot K_{eff}}{I_{avg}}$$

де $C_{battery}$ - ємність акумулятора та K_{eff} - коефіцієнта ефективності розряду

Алгоритм роботи



Концепція гібридного керування

На пристрої

Для кого: LoRaWAN, BLE.

Суть: "Рішення приймає контролер" (порівняння з RTC).

Ефект: Економія трафіку та часу в ефірі.

У хмарі

Для кого: Wi-Fi (MQTT, HTTP).

Суть: "Рішення приймає сервер" (AWS Lambda).

Ефект: Глибока аналітика без розряду батареї сенсора.



Об'єднання цих підходів дозволяє отримати
максимальну автономність для кожного
сценарію.

Програмна реалізація

```
void enterDeepSleep(int seconds) {
    Serial.printf("Mepexia ☹ Deep Sleep на %d секунд...\n", seconds);
    WiFi.disconnect(true);
    http.end();
    esp_sleep_enable_ext0_wakeup(GPIO_NUM_4, 0);
    esp_sleep_enable_timer_wakeup(seconds * 1000000ULL);
    esp_deep_sleep_start();
}
```

```
void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("--- HTTP v2 ---");

    pinMode(BUTTON_PIN, INPUT_PULLUP);

    if (!Wire.begin()) {
        Serial.println("Тимчасово ініціалізувати DPM000!");
        enterDeepSleep(MAIN_INTERVAL / 1000);
    }

    DPM.setTemperatureSensorSampling(DPM600_CN_8X);
    DPM.setHumiditySensorSampling(DPM600_CN_2X);
    DPM.setPressureSensorSampling(DPM600_CN_4X);
    DPM.setFilterSize(DPM600_FILTER_SIZE_3);
    DPM.setGsmWater(320, 150);

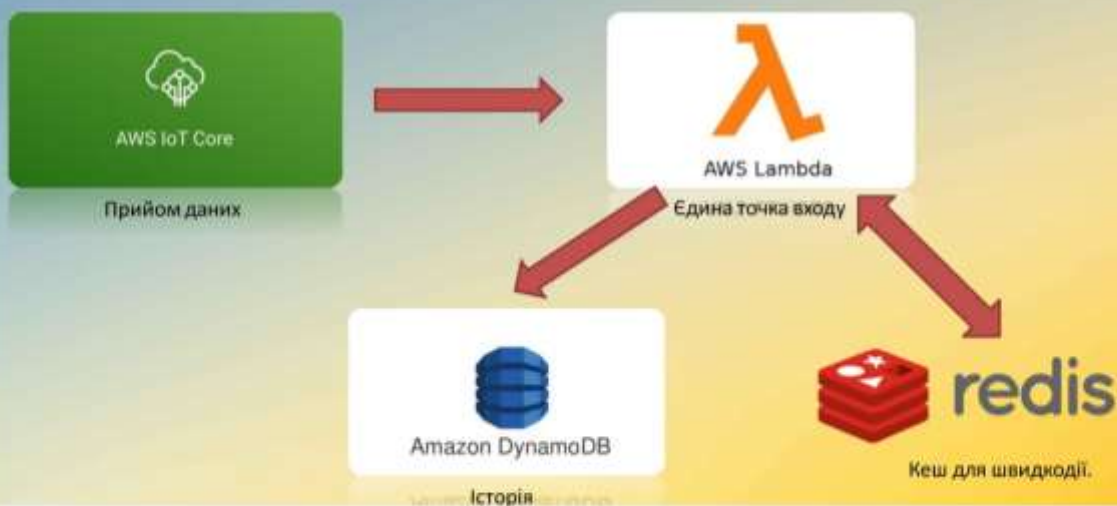
    connectWifi();
    syncTime();
    transmitData();

    delay(100);

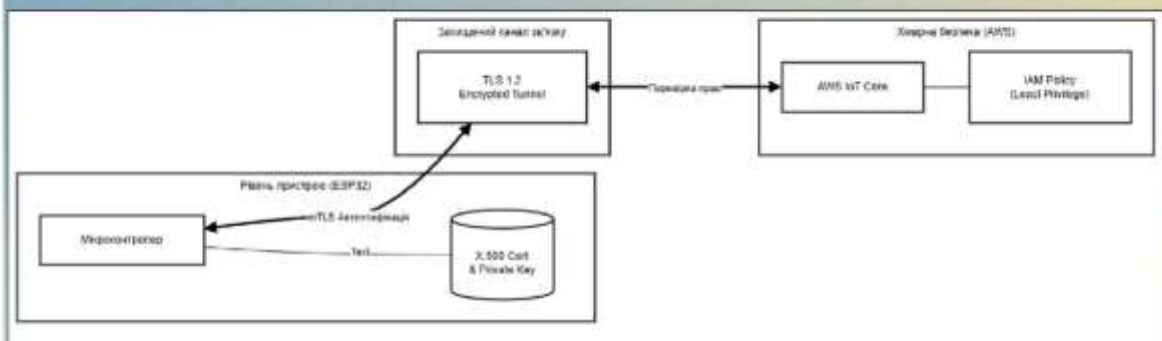
    enterDeepSleep(MAIN_INTERVAL / 1000);
}

void loop() {}
//
}
```

Хмарна реалізація



Забезпечення безпеки



Експериментальний стенд



Результати: Протоколи Wi-Fi

Протокол	Режим	$T_{\text{акт}}$ (с)	$I_{\text{сер}}$ (мА)	Зниження відн. v1
HTTP	v1	4,332	59,607	0%
	v2	8,964	24,678	58,60%
	v3	8,782	17,171	71,19%
	v4	5,048	16,590	72,17%
MQTT	v1	7,385	63,434	0%
	v2	5,690	22,077	65,20%
	v3	5,618	16,114	74,60%
	v4	10,430	15,344	75,81%
CoAP	v1	2,630	60,464	0%
	v2	4,543	19,893	67,10%
	v3	4,456	12,606	79,15%
	v4	4,495	12,469	79,38%

Результати: Енергоефективні мережі

Протокол	Режим	$T_{\text{акт}}$ (с)	$I_{\text{сер}}$ (мА)	Зниження відн. v1
BLE	v1	2,976	56,879	0%
	v2	6,926	22,429	60,57%
	v3	3,556	14,380	74,72%
LoRa	v1	3,623	96,779	0%
	v2	3,317	19,204	80,16%
	v3	2,790	12,629	86,95%

Ефективність адаптації



Оцінка автономності

Протокол	Версія	Середній струм I_{avg} (мА)	Час активності T_{act} (с)	Автономність (годин)	Автономність (днів)
HTTP	v4 (Cloud)	16.59	5.05	51.2	2.1
MQTT	v4 (Cloud)	15.34	10.43	55.4	2.3
BLE	v3 (Local)	14.38	3.56	59.1	2.5
LoRa	v3 (Local)	12.63	2.79	67.3	2.8
CoAP	v4 (Cloud)	12.47	4.49	68.2	2.8

Наукова новизна

Удосконалено: Метод керування енергоспоживанням.

Набуло подальшого розвитку: Застосування протоколу CoAP у зв'язці з проміжним шлюзом.

Вперше отримано: Комплексні порівняльні характеристики 5 протоколів на єдиній платформі.

Практичне значення

Технічне: Створено універсальну платформу (ESP32+AWS) та методику тестування.

Економічне: Зменшення витрат на заміну батарей у 3-5 разів.

Впровадження: Готовність до використання в системах «Розумне місто» та агромоніторингу.

Апробація та Публікації

Публікації:

- Стаття у фаховому виданні «Центральноукраїнський науковий вісник» (прийнято до друку).

Конференції:

- XVIII науково-практична конференція магістрантів та аспірантів "Прикладна математика та комп'ютинг" (2025).
- II International Final R&D online conference "Advances in Science and Technology (2025).

Висновки

1. Виконано аналіз та обґрунтування платформи.
2. Розроблено гібридний адаптивний алгоритм.
3. Експериментально підтверджено зниження енергоспоживання на 72–87%.
4. Визначено CoAP (для Wi-Fi) та LoRa (для Long Range) як найефективніші технології.



Дякую за увагу!